Arculus Recovery v1.6.0

User Guide and Technical Reference

Offline BIP39/BIP32 recovery, encrypted seed backup, QR export, file-format reference, and operational security procedures.

Primary GUI	Standalone HTML application
Desktop GUI	PySide6 WebEngine wrapper
Desktop Package	Tauri wrapper with native export bridge
CLI	Python derivation and export interface
Generated	June 14, 2026

Arculus Recovery v1.6.0 - Byte-Level User and Operator Guide

Source of Truth

This guide describes the standalone browser application implemented by the repository root file Arculus_Recovery.html with APP_VERSION equal to v1.6.0. The HTML file is the authoritative implementation for the UI, mnemonic normalization, key derivation, address construction, ARC seed-file encryption, QR generation, export formatting, and runtime clearing behavior.

The application is designed to run fully offline in a browser. The v1.6.0 HTML embeds its cryptographic and serialization dependencies directly, including:

```
BIP39 English word list
PBKDF2-HMAC-SHA512
HMAC-SHA512
SHA-256
SHA-512
RIPEMD-160
Keccak-256
secp256k1 scalar and point arithmetic
Ed25519 arithmetic
Bech32
Bech32m
CashAddr
Bitcoin Base58Check
XRPL Base58Check
Monero base58 block encoding
QR encoding and PNG canvas rendering
jsPDF PDF export
```

The network indicator reflects navigator.onLine only. It does not create a network request and does not prove that the browser, host operating system, or extensions are isolated. Recovery work should be performed in an environment where the operator controls the machine, browser profile, clipboard, screen capture, and downloaded files.

Supported Recovery Scope

Arculus Recovery v1.6.0 derives keys and addresses. It does not scan balances, query account state, broadcast transactions, import funds, sign transactions, fetch token metadata, check address reuse, infer wallet gap limits, or contact wallet-provider APIs.

The supported visible seed sizes are:

```
12 words
24 words
```

Both sizes are BIP39 English mnemonics. A 12-word mnemonic encodes 128 entropy bits plus 4 checksum bits. A 24-word mnemonic encodes 256 entropy bits plus 8 checksum bits. Other BIP39-valid word counts such as 15, 18, and 21 are not accepted by v1.6.0.

All listed coins are mainnet only:

```
Bitcoin
Bitcoin Cash
Litecoin
Dogecoin
Ethereum / ERC-20
Solana
Stellar
Monero
Cardano
XRP
```

Initial UI State

On load, the title row displays "Arculus Recovery" and v1.6.0. The default input state is:

```
mnemonic textarea: empty
seed length radio: 12 words
numbered word grid: 24 cells present; active cells follow seed length
passphrase input: empty, type=password
coin selector: Bitcoin
derivation path: m/0'
script selector: P2WPKH
```

```
address count: 5
range: empty
export format: PDF
root fingerprint display: Root Fingerprint:
status: Ready
```

Changing the coin selector resets the derivation path to the coin default and also updates the script selector:

```
Bitcoin: P2WPKH
Bitcoin Cash: P2PKH
Litecoin: P2WPKH
Dogecoin: P2PKH
Ethereum / ERC-20: Auto
Solana: Auto
Stellar: Auto
Monero: Auto
Cardano: Auto
XRP: Auto
```

The derivation-path info marker is displayed when the active path is exactly m/0'. That marker exists because the Arculus default path is a single hardened account level rather than the BIP44-style m/44'/coin_type'/account' path used by many other wallets.

Mnemonic Input Model

Seed input can be supplied through either:

```
a single textarea containing whitespace-separated words
the numbered word grid with one word per cell
```

The active mnemonic used by validation, derivation, seed export, and seed copy is selected as follows:

```
If an imported, generated, or masked seed state exists, use that hidden
internal mnemonic.

Otherwise, use the visible mnemonic textarea value.
```

The normalization algorithm is:

```
input_string = input or empty string
trimmed = input_string.trim()
tokens = trimmed.split(/\s+/).filter(Boolean)
joined = tokens.join(" ")
normalized = joined.normalize("NFKD")
words = normalized === "" ? [] : normalized.split(" ")
```

Important byte-level properties:

```
The join separator is one ASCII space, byte 0x20 after UTF-8 encoding.
The mnemonic is normalized with Unicode NFKD before BIP39 seed derivation.
Word lookup is against the embedded lowercase BIP39 English list.
The visible text area placeholder is not part of the mnemonic.
The passphrase is never appended to the displayed mnemonic.
```

Validation rejects:

```
word counts other than 12 or 24
words absent from the embedded BIP39 English list
valid words whose checksum bits do not match SHA-256(entropy)
```

When the visible mnemonic is successfully derived for the first time, v1.6.0 promotes it into the protected hidden seed state. The textarea and word cells are replaced by mask tokens, and later derivations use the hidden internal seed unless Clear All is pressed.

Generated Seed Workflow

Generate Random Seed uses crypto.getRandomValues to sample entropy. The seed length radio controls the entropy length:

```
12 words: 16 random bytes, 128 entropy bits, 4 checksum bits
24 words: 32 random bytes, 256 entropy bits, 8 checksum bits
```

Generation steps:

- Sample entropy bytes with crypto.getRandomValues.
- Compute SHA-256(entropy).

- Take the leftmost ENT / 32 checksum bits.
- Concatenate entropy bits || checksum bits.
- Split into 11-bit unsigned indexes.
- Map each index to BIP39_WORDS[index].
- Store the mnemonic in hidden seed state.
- Replace the visible textarea and word grid with mask tokens.

The generated mnemonic is ready for validation, derivation, copy, QR-free export, and ARC export without revealing it on screen. Hold Show Seed to reveal the words temporarily. Release the button, move the pointer away, or end the touch event to re-mask the phrase.

Passphrase Handling

The Passphrase field is the optional BIP39 passphrase, sometimes called the 25th word in wallet UI language. It is not a BIP39 word and is not validated against the BIP39 word list.

The BIP39 seed calculation is:

```
password bytes = UTF8(NFKD(normalized_mnemonic))
salt bytes = UTF8(NFKD("mnemonic" + passphrase))
PBKDF2 algorithm = PBKDF2-HMAC-SHA512
PBKDF2 iterations = 2048
output length = 64 bytes
```

An empty passphrase is a distinct wallet from any non-empty passphrase. A mistyped passphrase can produce valid-looking derived addresses that do not match the intended wallet. v1.6.0 shows a passphrase warning in validation output when the mnemonic is valid and the passphrase field is empty.

The passphrase visibility button toggles the HTML input type between password and text. Toggling visibility does not change the stored value, derived seed, root fingerprint, or export bytes.

Validate Mnemonic

Validate Mnemonic runs analyzeMnemonic against the active mnemonic and the current passphrase. It writes a JSON diagnostic object to the raw output area. On success or structured failure the object contains:

```
validation
word_count
wordlist_validity
entropy_bits
checksum_bits
checksum_match
bip39_compliance
bip39_seed_512_bit
master_private_key
master_chain_code
root_fingerprint
keystore_type
seed_format_detection
passphrase_warning
message
```

For a valid mnemonic, the BIP39 seed is 64 bytes and is rendered as lowercase hex. The secp256k1 master private key and master chain code are produced by:

```
HMAC-SHA512(key = ASCII("Bitcoin seed"), data = BIP39 seed)
master_private_key = output bytes 0..31
master_chain_code = output bytes 32..63
```

The root fingerprint is:

```
HASH160(compressed_master_public_key)[0..3]
```

The displayed root fingerprint is also refreshed when the passphrase changes.

Derivation Path Rules

The path parser accepts:

```
m/0'
0'
m/44h/60h/0h
m/84H/0H/0H
m/1852'/1815'/0'/0/0
```

The path parser behavior is:

```
Split the path on "/".
If the first component is "m", skip it.
Empty components are ignored by the parser.
A hardened suffix is "'", "h", or "H".
The numeric component must match decimal digits only.
The pre-hardened value must satisfy 0 <= n < 2147483648.
Hardened values are encoded as n | 0x80000000.
normalizePath renders the path back with leading m and apostrophes.
```

Auto script selection is based on the first path component after removing the hardened bit:

```
purpose 86: p2tr
purpose 84: p2wpkh
purpose 49: p2wpkh-p2sh
all other purposes: p2pkh
```

Ethereum and XRP ignore the selected UTXO script type and use their own address families. Solana, Stellar, Monero, and Cardano use their own Ed25519 family derivation and do not emit UTXO script addresses.

Default Derivation Paths

The v1.6.0 default account paths are:

```
bitcoin: m/0'
bitcoincash: m/0'
litecoin: m/84'/2'/0'
dogecoin: m/44'/3'/0'
ethereum: m/44'/60'/0'
solana: m/44'/501'/0'
stellar: m/44'/148'/0'
monero: m/44'/128'/0'
cardano: m/1852'/1815'/0'/0/0
xrp: m/44'/144'/0'
```

For secp256k1 account-style outputs, displayed receiving rows append /0/i and displayed change rows append /1/i to the selected account path.

For Solana and Stellar, displayed rows are hardened account rows. For Monero, the selected account path directly produces a spend/view account address. For Cardano, the selected path is the payment key path and the staking key path is m/1852'/1815'/0'/2/0.

Address Count and Range

Without a range, the count field controls how many indexes are derived per branch. The raw count is:

```
countRaw = Math.max(1, Number(count.value || 5))
```

With count 5, UTXO-style and secp256k1 account-style coins derive 5 receiving rows and 5 change rows. Single-branch Ed25519 account families derive the visible receiving/account rows defined by their coin-specific function.

When the Range field is populated, it overrides count if and only if it matches:

```
^(\d+)\s*-\s*(\d+)$
```

Range behavior:

```
startIdx = Math.max(0, parsed_start)
endIdx = Math.max(startIdx, parsed_end)
count = endIdx - startIdx + 1
```

When the range field is non-empty, the count input is blanked, disabled, and visually dimmed. Clearing the range restores the saved manual count or 5.

Derive Keys + Addresses

Derive Keys + Addresses runs the full derivation pipeline and updates:

```
progress bar
lastDerivedOutput object
raw JSON output
table view
extended keys pane
root seed masking state
status line
```

The top-level output shape is:

```
{
  "coin": string,
  "network": "mainnet",
  "word_count": 12 or 24,
  "accounts": [
    account_object
  ]
}
```

For Bitcoin-family UTXO coins, Ethereum, and XRP, account_object contains:

```
derivation
account_script_type_used
root_xprv
root_xpub
optional root_yprv/root_ypub
optional root_zprv/root_zpub
optional root_trprv/root_trpub
account_xprv
account_xpub
optional account_yprv/account_ypub
optional account_zprv/account_zpub
optional account_trprv/account_trpub
receiving array
change array
```

For each UTXO row:

```
path = derivation + "/" + branch + "/" + index
branch 0 = receiving
branch 1 = change
public_key_hex = compressed secp256k1 public key, 33 bytes
private_key_hex = secp256k1 scalar, 32 bytes
private_key_wif = Base58Check WIF when the coin defines WIF
```

For Ethereum rows:

```
address = EIP-55 address
public_key_hex = compressed secp256k1 public key, 33 bytes
ethereum_public_key_hex = uncompressed x||y public key without 0x04, 64 bytes
private_key_hex = secp256k1 scalar, 32 bytes
erc20_address = same string as address
```

For XRP rows:

```
address = XRPL classic r-address
xrp_classic_address = same string as address
public_key_hex = compressed secp256k1 public key, 33 bytes
private_key_hex = secp256k1 scalar, 32 bytes
destination tags are not derived
```

For Taproot rows:

```
address = Bech32m v1 output key address
taproot_internal_public_key_hex = x-only internal key, 32 bytes
taproot_internal_private_key_hex = internal scalar, 32 bytes
taproot_internal_private_key_wif = WIF for internal scalar
taproot_tweak_hex = tagged TapTweak scalar bytes
taproot_output_public_key_hex = x-only output key, 32 bytes
taproot_output_private_key_hex = tweaked scalar, 32 bytes
taproot_output_private_key_wif = WIF for tweaked scalar
taproot_output_key_parity = output y-coordinate parity
```

Coin-Specific Output Notes

Bitcoin:

```
P2PKH uses version byte 0x00 and Base58Check display.
P2WPKH-P2SH uses redeem script 0x00 0x14 HASH160(pubkey), P2SH version 0x05.
```

```
  P2WPKH uses Bech32 HRP "bc", witness version 0.
  P2TR uses Bech32m HRP "bc", witness version 1.
  WIF uses version byte 0x80 and compressed-key suffix 0x01.
```

Bitcoin Cash:

```
  Default display is CashAddr with prefix "bitcoincash".
  Advanced legacy mode displays Base58Check P2PKH using version byte 0x00.
  P2WPKH, P2WPKH-P2SH, and P2TR are not supported for Bitcoin Cash in v1.6.0.
```

Litecoin:

```
  P2PKH uses address version 0x30.
  P2SH uses address version 0x32.
  P2WPKH uses Bech32 HRP "ltc".
  WIF uses version byte 0xB0.
  Litecoin P2WPKH extended display keys use zpub/zprv version bytes.
```

Dogecoin:

```
  P2PKH uses address version 0x1E.
  P2SH uses address version 0x16 when relevant.
  WIF uses version byte 0x9E.
  P2TR is not supported and must fail before output.
```

Ethereum / ERC-20:

```
  The private key is a secp256k1 scalar.
  The address is the last 20 bytes of Keccak-256(uncompressed_public_key_x_y).
  Display casing follows EIP-55.
  ERC-20 tokens use the same account address; token contract addresses are not derived.
```

Solana:

```
  Derivation uses SLIP-0010 Ed25519 hardened child derivation.
  Address bytes are the 32-byte Ed25519 public key.
  Display is Bitcoin Base58 over raw public-key bytes with no checksum.
```

Stellar:

```
  Derivation uses SLIP-0010 Ed25519 hardened child derivation.
  Public account display uses Stellar StrKey with version byte 0x30.
  Secret seed display uses Stellar StrKey with version byte 0x90.
  StrKey checksum is CRC16-XModem stored little-endian before base32 encoding.
```

Monero:

```
  Derivation uses the v1.6.0 Monero Ed25519 path logic.
  Output includes private spend key, private view key, public spend key, and public view
key.
  Standard mainnet address construction uses network byte 0x12 and a 4-byte Keccak checksum.
  The tool does not scan subaddresses or account balances.
```

Cardano:

```
  Derivation uses CIP-1852/Icarus Ed25519-BIP32 logic.
  The payment path comes from the Derivation Path field.
  The staking path for the displayed base address is m/1852'/1815'/0'/2/0.
  The address is a Shelley mainnet base address using Bech32 HRP "addr".
```

Output Views

After a successful derivation, the output tabs become available:

```
  Table View
  Raw JSON
  Extended Keys
  Expand
```

Table View renders flattened address rows. Raw JSON is the complete lastDerivedOutput object with two-space indentation. Extended Keys displays root and account extended keys available for the active account and script family.

The Expand control enlarges the output area for review. It does not modify the derived object or exported bytes.

Export Derived Keys and Addresses

The Export button exports lastDerivedOutput. If no derivation has completed, the status line reports that there is nothing to export.

JSON export:

```
filename: Arculus_Derived_Keys_Addresses.json
MIME type: application/json;charset=utf-8
bytes: JSON.stringify(lastDerivedOutput, null, 2) || 0a
```

CSV export:

```
filename: Arculus_Derived_Keys_Addresses.csv
MIME type: text/csv;charset=utf-8
row separator: LF, byte 0x0a
final byte: LF, byte 0x0a
quoting: fields containing comma, quote, CR, or LF are wrapped in quotes
quote escaping: " becomes ""
```

TXT export:

```
filename: Arculus_Derived_Keys_Addresses.txt
MIME type: text/plain;charset=utf-8
row separator: LF, byte 0x0a
final byte: LF, byte 0x0a
```

PDF export:

```
filename: Arculus_Key_Export_<CoinDisplayName>.pdf
library: embedded jsPDF
orientation: landscape
unit: points
page size: A4
title: Arculus Key Export
metadata line: coin, Arculus v1.6.0, UTC timestamp, optional root fingerprint
```

PDF column selection:

```
branch
path
address
private_key_wif for Bitcoin-family UTXO coins
private_key_hex for Ethereum, Solana, Stellar, Cardano, XRP
private_spend_key_hex for Monero
```

The PDF also renders an Extended Keys section when account extended keys are present in lastDerivedOutput.

ARC Seed Export Overview

Encrypt/Export Seed writes:

```
filename: Arculus_Encrypted_Seed.arc
MIME type: text/plain;charset=utf-8
armor line 1: ARCULUS-ARC-V2
armor line 2: base64(UTF8(JSON.stringify(bundle)))
final byte: LF
```

The export dialog supports four protection choices:

```
Password
Keyfile
Keyfile + Password
Advanced: no key or password
```

Seed export always validates that the active mnemonic is a valid 12-word or 24-word BIP39 mnemonic before creating the .arc file.

ARC V2 Password Mode

Password mode uses the password text from the export dialog as the .arc encryption secret. The password is NFKD-normalized inside the ARC V2 KDF.

New ARC V2 encrypted seed bundles use:

```
magic: ARCULUS-ARC
```

```
format: arculus-encrypted-seed-v2
version: 2
kdf.name: PBKDF2
kdf.hash: SHA-512
kdf.iterations: 1000000
kdf.salt_b64: 32 random bytes encoded as base64
cipher.name: HMAC-SHA512-CTR
cipher.nonce_b64: 24 random bytes encoded as base64
ciphertext_b64: encrypted plaintext JSON
mac_b64: 64-byte HMAC-SHA512 tag encoded as base64
credential_mode: password
```

Plaintext JSON before encryption contains:

```
mnemonic
word_count
created_at
```

The plaintext mnemonic is normalized to single-space BIP39 words before encryption. The BIP39 passphrase field is not stored in the .arc seed file.

ARC V2 Keyfile Mode

Keyfile mode generates or loads at least 32 bytes of key material. The normal generated keyfile contains 64 random bytes.

Generated raw keyfile:

```
filename: Arculus_Recovery_Keyfile.key
MIME type: application/json;charset=utf-8
magic: ARCULUS-KEYFILE
format: arculus-recovery-keyfile-v1
version: 1
key_b64: 64 random bytes encoded as base64
```

The .arc encryption secret is:

```
"arc-keyfile-v1:" || base64(keyfile_bytes)
```

Accepted import forms for raw keyfiles:

```
JSON object with magic ARCULUS-KEYFILE and format arculus-recovery-keyfile-v1
JSON object with legacy magic KEYFILE and key field
plain base64 key material text
```

A raw keyfile must decode to at least 32 bytes. Generated v1.6.0 keyfiles are 64 bytes.

ARC V2 Keyfile + Password Mode

Keyfile + Password mode uses both possession of keyfile bytes and knowledge of the password. For new exports, v1.6.0 generates 64 random keyfile bytes and writes them in encrypted keyfile form.

Generated encrypted keyfile:

```
filename: Arculus_Recovery_Keyfile.enc.key
MIME type: application/json;charset=utf-8
magic: ARCULUS-KEYFILE-ENC
format: arculus-recovery-keyfile-enc-v1
version: 1
kdf.algo: PBKDF2-SHA512
kdf.iterations: 200000
kdf.salt_b64: 16 random bytes encoded as base64
cipher.name: HMAC-SHA512-CTR
cipher.nonce_b64: 16 random bytes encoded as base64
ciphertext_b64: encrypted raw keyfile bytes
mac_b64: HMAC-SHA512 tag
```

Encrypted keyfile KDF:

```
password bytes = UTF8(NFKD(password))
salt = decoded kdf.salt_b64
PBKDF2-HMAC-SHA512 iterations = 200000
output length = 64 bytes
encKey = bytes 0..31
macKey = bytes 32..63
```

The .arc encryption secret for combined mode is:

```
material = UTF8("arc-combined-v1\n" || NFKD(password) || "\n" || keyfile_secret)
digest = SHA-256(material)
secret = "arc-combined-v1:" || base64(digest)
```

The password encrypts the .enc.key file and also participates in the combined .arc secret. Losing either the password or the matching keyfile prevents import.

Unprotected ARC Export

The Advanced option "Save .arc file without key or password" writes a plaintext seed bundle. It exists for controlled offline workflows and test data only.

Plain seed bundle fields:

```
magic: ARCULUS-ARC
format: arculus-plain-seed-v1
version: 1
created_at: ISO-8601 timestamp
mnemonic: normalized mnemonic words
word_count: 12 or 24
encrypted: false
```

Even unprotected seed bundles are armored with ARCULUS-ARC-V2 line 1 and base64 JSON line 2. The protection difference is inside the decoded JSON bundle.

Import Seed Workflow

Import Seed accepts:

```
.arc files
JSON files containing supported ARC bundle JSON
application/json files selected by the browser file picker
```

Import parsing:

```
If text begins with ARCULUS-ARC-V2, remove the first line, concatenate the
remaining lines, base64-decode them, UTF-8 decode the result, and parse JSON.

Otherwise, parse the selected file text directly as JSON.
```

If the bundle is plaintext ARCULUS-ARC / arculus-plain-seed-v1, no credential dialog is needed. If the bundle is encrypted, the dialog asks for password, keyfile, or keyfile + password. If the bundle has a credential_mode hint, the matching mode is selected and locked.

After decryption:

```
The mnemonic is normalized.
The word_count field must match the normalized word count.
The mnemonic must pass BIP39 word-list and checksum validation.
The seed is stored in hidden imported seed state.
Visible seed fields are replaced by mask tokens.
The output area reports imported_seed, source, word_count, and hidden_on_screen.
```

The BIP39 passphrase is not stored in the .arc file. If the recovered wallet requires a BIP39 passphrase, enter it in the Passphrase field after import and before validation or derivation.

Copy Seed and Clipboard Handling

Copy Seed copies the active normalized mnemonic words to the system clipboard only after a browser confirmation prompt. The copied text is:

```
words.join(" ")
```

The separator is ASCII space, byte 0x20. No trailing newline is added by the application.

After a successful copy, the status line starts a 60-second countdown. Browser and operating-system clipboard APIs do not provide a universal reliable wipe mechanism. Treat the clipboard as exposed until the operating environment is cleared.

Show Seed Behavior

Show Seed is a hold-to-reveal control. It reveals hidden generated, imported, or post-derive manually entered seed words only while the control is active.

Reveal starts on:

```
mousedown
touchstart
```

Reveal ends on:

```
mouseup
mouseleave
touchend
```

When reveal ends, the visible fields are replaced with mask tokens again.

Clear All and Idle Timeout

Clear All runs handleClearAll. It clears:

```
imported or generated hidden seed state
mnemonic textarea
numbered word grid values
word valid/invalid styling
passphrase value
raw output textarea
table and tab view state
root fingerprint display
lastDerivedOutput
```

Clear All does not delete files already downloaded by the browser and does not wipe host clipboard history.

The idle watcher listens for:

```
mousedown
keydown
touchstart
scroll
```

Idle timing:

```
4 minutes after last watched event: show warning banner
5 minutes after last watched event: run Clear All
5 seconds after clearing: hide the idle-cleared banner
```

The idle warning text is:

```
Idle timeout in 60 seconds. All fields will be cleared.
```

The post-clear text is:

```
Session cleared due to inactivity.
```

Settings

Theme settings:

```
Light
Dark
Dark+
Terminal
```

Theme selection is stored in localStorage under:

```
arculusTheme
```

If localStorage is unavailable, the theme still applies to the current page but may not persist.

Advanced settings:

```
Auto-Derive on Settings Change
Legacy Address Format (Bitcoin Cash)
```

Auto-Derive only runs after at least one successful derivation has populated lastDerivedOutput. It is debounced by 600 ms. It is triggered by changes to:

```
coin
script type
derivation path
count
range
```

Legacy Address Format applies only to Bitcoin Cash. When enabled, Bitcoin Cash P2PKH addresses are displayed with legacy Base58Check instead of CashAddr.

QR Export

QR Export opens a modal with an address input. It does not require a derived output object; any pasted address-like string can be encoded.

For most addresses, Generate immediately renders a QR. XRP-like r-addresses open a Destination Tag / Memo prompt first. The operator may enter a numeric destination tag or skip.

QR rendering:

```
normal QR canvas: 256 x 256 CSS/display pixels
XRP QR canvas: 320 x 320 CSS/display pixels
quiet zone: 4 modules
dark color: #000000
light color: #ffffff
saved image MIME type: image/png
filename prefix: qr_
```

Filename sanitization for saved QR PNG:

```
safeName = address.replace(/[^a-zA-Z0-9]/g, "_").slice(0, 40) || "address"
filename = "qr_" || safeName || ".png"
```

QR Copy Address copies only the address input value, not the URI label and not the XRP destination tag.

QR URI Construction

If the address input already begins with a URI scheme matching /^[a-z]+:/i, the input is used unchanged.

Otherwise v1.6.0 generates these QR payload strings:

```
Solana:
  "solana:" || address

Stellar:
  "web+stellar:pay?destination=" || encodeURIComponent(address)

Monero:
  "monero:" || address

Cardano:
  address

Bitcoin:
  "bitcoin:" || address

Bitcoin Cash with bitcoincash: prefix:
  address

Bitcoin Cash without prefix:
  "bitcoincash:" || address

Litecoin:
  "litecoin:" || address

Dogecoin:
  "dogecoin:" || address

Ethereum:
  "ethereum:" || address

XRP:
  address || "?dt=" || destination_tag
```

For XRP, an empty destination tag is encoded as:

```
?dt=00000
```

This behavior is scanner-focused and does not imply that every receiving service accepts an empty or zero destination tag. If a service requires a tag, use the tag supplied by that service.

Python 1.6.0 Automation Surface

The current Python package and launcher are upgraded to APP_VERSION 1.6.0 for scripted derivation. The canonical HTML file remains the interactive source of truth, but Python CLI output is expected to match the v1.6.0 derivation vectors for the supported command-line coin ids:

```
bitcoin
bitcoincash
litecoin
dogecoin
ethereum
solana
stellar
monero
cardano
xrp
```

CLI default behavior:

```
command:
  python Arculus_Recovery.py --mnemonic "<words>" --coin <coin> --output-format json

version string:
  Arculus Recovery 1.6.0

default script type:
  auto

output formats:
  json
  csv
  txt

testnet:
  configured only for coins whose Python network table defines testnet
  parameters.
```

Python CLI JSON/CSV/TXT exports contain private key material when derivation is successful. Treat command output redirected to files exactly like browser JSON, CSV, and TXT exports. Terminal scrollback, shell history, transcript logging, PowerShell history, and CI logs are all out-of-band copies of secret material.

Python GUI behavior is determined by the packaged HTML asset under the Python package. For byte-level derivation conformance, compare Python output against docs/TestVectors.txt. If Python output ever differs from the canonical HTML and the vectors, the Python implementation is wrong until the difference is explicitly documented as an intentional compatibility boundary.

Operational Procedure

Recommended offline recovery procedure:

- Open Arculus_Recovery.html from local storage on a trusted machine.
- Confirm the title shows v1.6.0.
- Disconnect the machine from networks when practical.
- Paste, import, or generate the mnemonic.
- Enter the BIP39 passphrase if the wallet used one.
- Click Validate Mnemonic.
- Confirm word count, checksum, seed status, and root fingerprint.
- Select coin and derivation path.
- Select script type or leave Auto where appropriate.
- Set count or range.
- Click Derive Keys + Addresses.
- Compare the expected receiving address with a known address from the wallet when available.
- Export only the minimum format needed for the recovery workflow.

- Store any .arc, .key, .enc.key, JSON, CSV, TXT, PDF, and QR PNG files according to the sensitivity of private keys.
- Press Clear All.
- Clear clipboard history and browser downloads according to the host operating system.

If imported funds are being recovered into a new wallet, prefer deriving enough addresses to locate funds and then sweep or transfer using a transaction signer that is outside the scope of this tool. Do not treat derived private-key exports as long-term storage unless they are protected by an offline storage process.

File Sensitivity

Exported derived-key files contain private keys unless the operator manually removes them after export. Treat these as seed-equivalent:

```
Arculus_Derived_Keys_Addresses.json
Arculus_Derived_Keys_Addresses.csv
Arculus_Derived_Keys_Addresses.txt
Arculus_Key_Export_<CoinDisplayName>.pdf
```

Encrypted seed bundle:

```
Arculus_Encrypted_Seed.arc
```

Raw keyfile:

```
Arculus_Recovery_Keyfile.key
```

Encrypted keyfile:

```
Arculus_Recovery_Keyfile.enc.key
```

QR PNG files may contain deposit addresses or payment URIs. Address QR files do not contain private keys unless the operator manually pastes private material into the QR dialog.

Failure Handling

Common failure states:

```
Unknown mnemonic word:
  At least one token is absent from the embedded BIP39 English list.

Invalid checksum:
  All words are valid BIP39 words, but the checksum bits do not match the
  entropy-derived SHA-256 checksum.

Wrong passphrase:
  Validation still succeeds, but derived root fingerprint and addresses differ
  from the intended wallet.

Unsupported script type:
  The selected coin does not define the requested script family.

Wrong .arc password or keyfile:
  Import reports that the password may be incorrect or the file may be
  corrupted. The error intentionally does not distinguish the cause.

Missing keyfile:
  Keyfile mode cannot continue until a keyfile is generated or selected.

Encrypted keyfile wrong password:
  Keyfile decryption fails before the .arc seed can be decrypted.

PDF library unavailable:
  PDF export fails if the embedded jsPDF object is absent or damaged.
```

Byte-Level Boundaries

The following values are hard requirements in v1.6.0:

```
APP_VERSION: 1.6.0
ARC armor header: ARCULUS-ARC-V2
ARC encrypted format: arculus-encrypted-seed-v2
ARC plaintext format: arculus-plain-seed-v1
ARC encrypted version: 2
ARC plaintext version: 1
```

```
  ARC V2 KDF: PBKDF2-HMAC-SHA512
  ARC V2 KDF iterations for new exports: 1000000
  ARC V2 import iteration floor: 600000
  ARC V2 salt length: 32 bytes
  ARC V2 nonce length: 24 bytes
  ARC V2 MAC length: 64 bytes
  ARC V2 cipher name: HMAC-SHA512-CTR
  generated keyfile bytes: 64 bytes
  encrypted keyfile KDF iterations: 200000
  encrypted keyfile salt length: 16 bytes
  encrypted keyfile nonce length: 16 bytes
  BIP39 seed length: 64 bytes
  BIP39 seed PBKDF2 iterations: 2048
  supported word counts: 12 and 24
  idle clear timeout: 5 minutes
  idle warning lead time: 60 seconds
  clipboard countdown: 60 seconds
```

Non-Goals

Arculus Recovery v1.6.0 does not:

```
  create transactions
  sign transactions
  broadcast transactions
  query balances
  discover token contracts
  derive exchange destination tags
  store the BIP39 passphrase in .arc files
  guarantee clipboard erasure
  erase downloaded files
  validate that a receiving address has ever held funds
  validate that a third-party wallet uses the same derivation path
  protect against a compromised browser, extension, operating system, or display capture
path
```

The tool is a deterministic offline recovery and export utility. Its outputs are exact key material and address data; operational security around those outputs remains the responsibility of the recovery environment.

Interface Guide

The screenshots below show the primary user-facing layout and theme variants. The HTML application is the canonical GUI surface. PySide6 and Tauri package the same interface for desktop use, so the screenshots apply to all GUI editions.

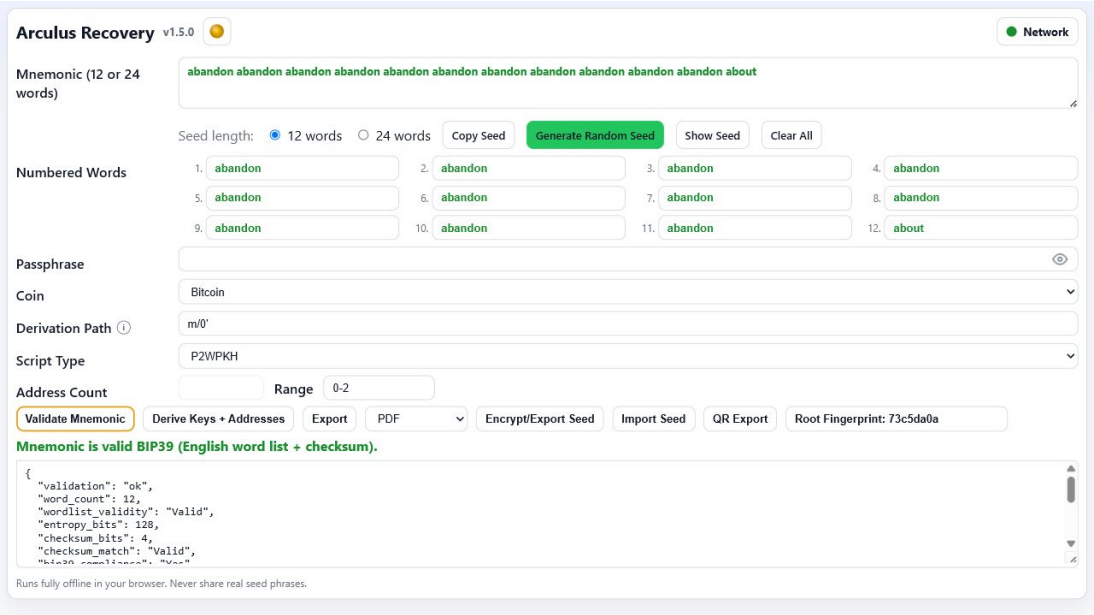

Figure 1. Main recovery workspace with test-vector mnemonic validation controls.

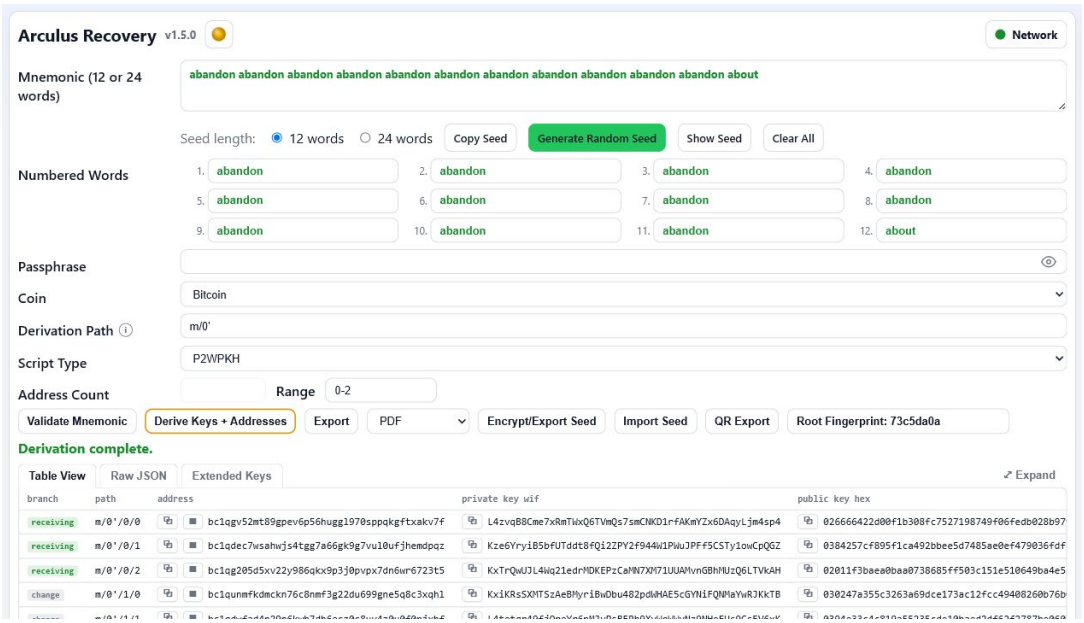

Figure 2. Derived output table after address derivation, with output tabs and export controls.

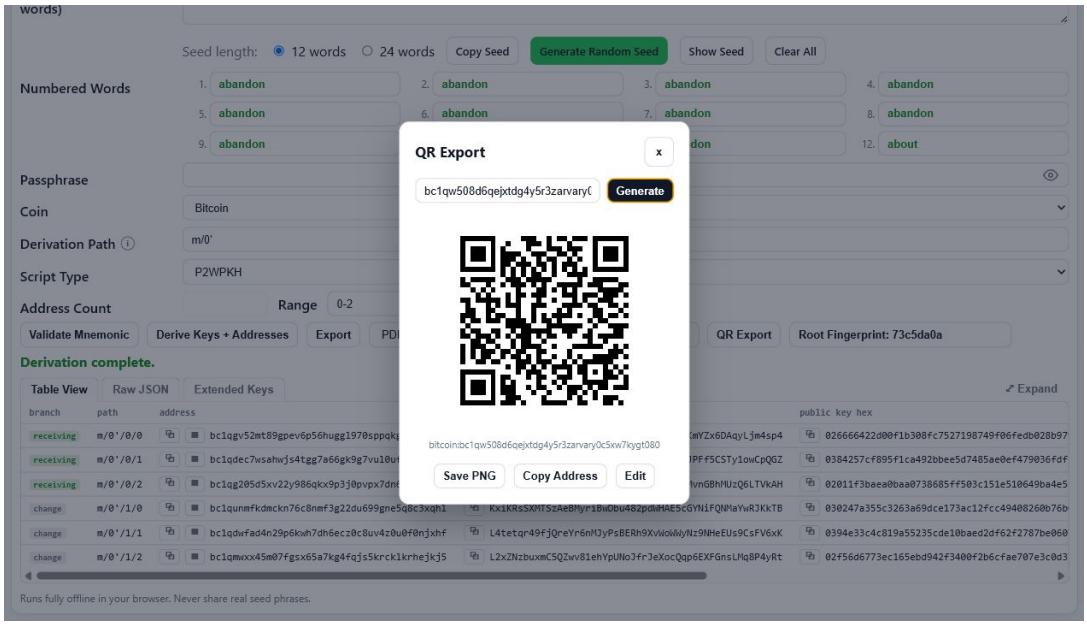

Figure 3. QR Export modal generating an address QR code without external services.

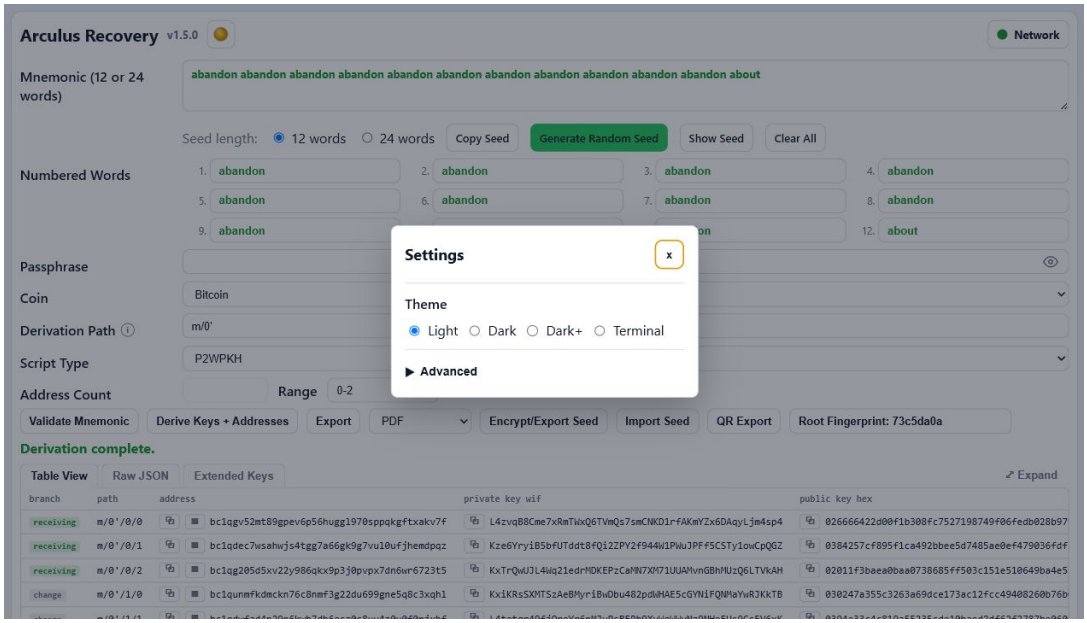

Figure 4. Settings dialog with theme selection and auto-derive preference.

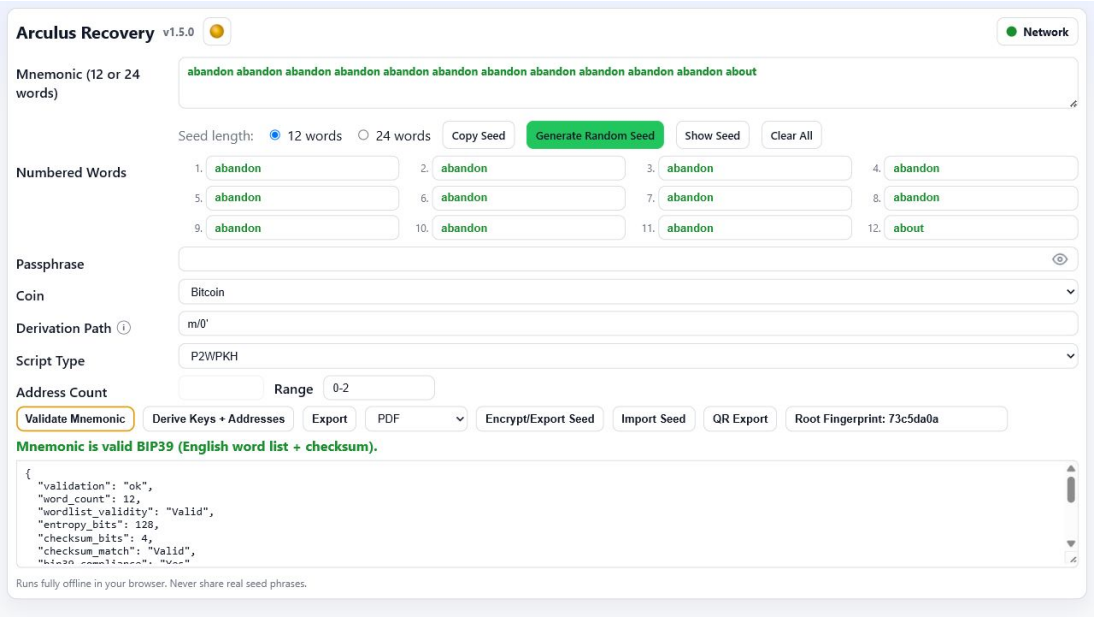

Figure 5. Light theme interface.

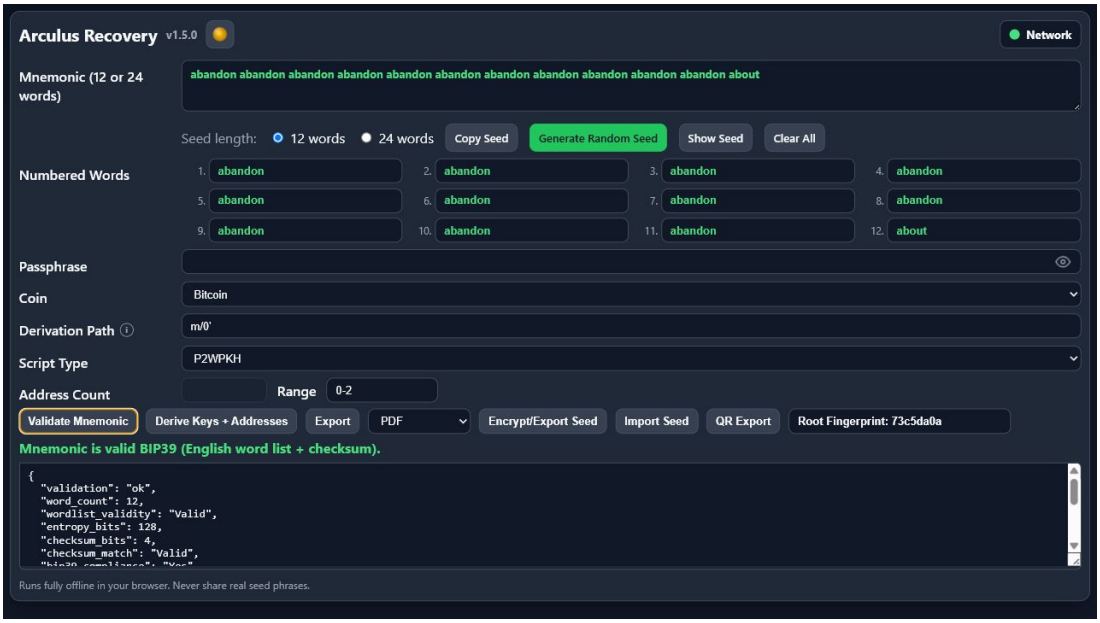

Figure 6. Dark theme interface.

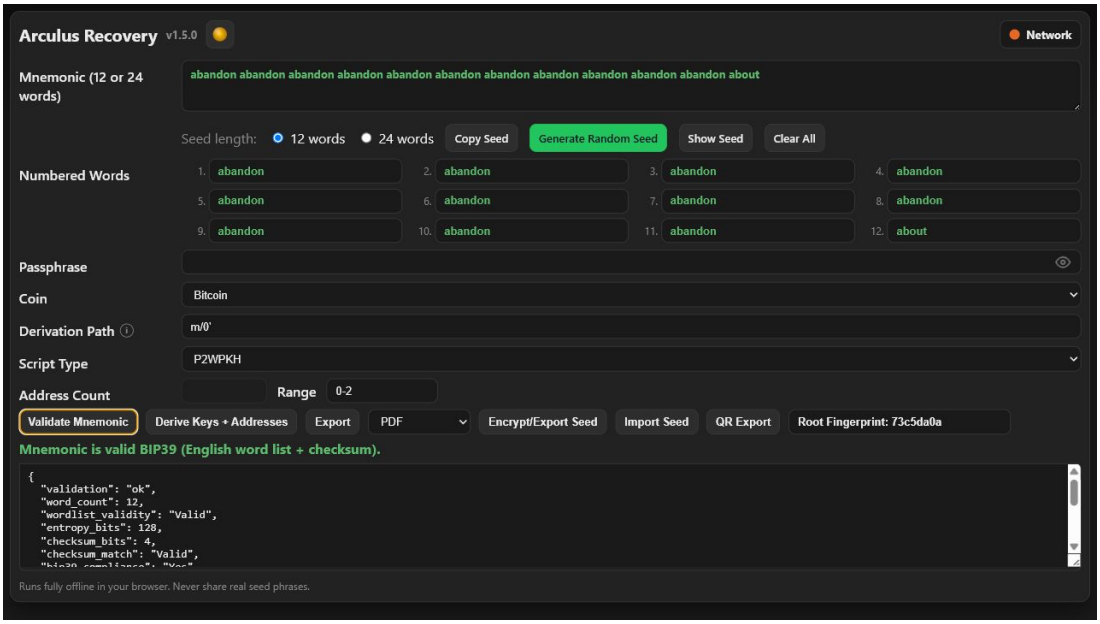

Figure 7. Dark+ theme interface.

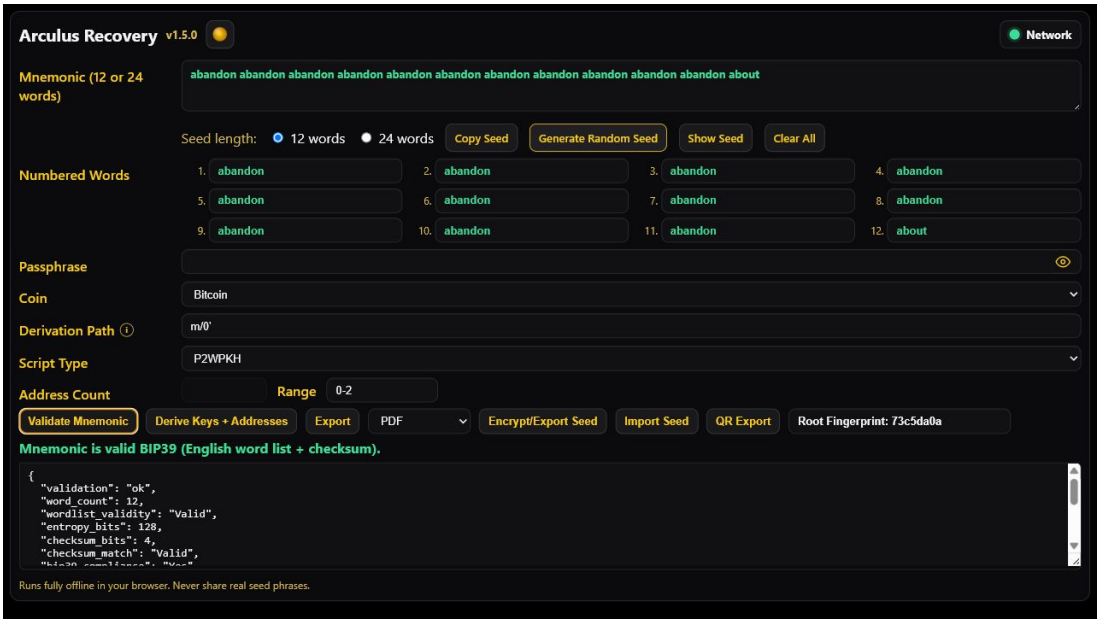

Figure 8. Terminal theme interface.

Arculus Recovery v1.6.0 - Recovery Procedure and Data Boundary Specification

Scope

This document defines recovery procedures for Arculus_Recovery.html v1.6.0. It covers direct mnemonic entry, generated/imported seed handling, protected and unprotected .arc import, credential-mode selection, BIP39 passphrase checks, derivation path recovery, address range selection, output export, QR generation, and session cleanup.

The tool is a deterministic offline recovery engine. It does not sign transactions, broadcast transactions, query balances, discover used addresses, estimate fees, identify token holdings, or validate exchange memo requirements.

Recovery Model

Every recovery result is a deterministic function of local inputs:

```
mnemonic
BIP39 passphrase
coin
derivation path
script type
address count or range
.arc credential if importing a protected .arc file
```

No external service is contacted. If an address does not appear, the tool cannot discover the correct path from chain state. The operator must supply or search the correct derivation settings.

Required Inputs

At minimum:

- 12- or 24-word BIP39 English mnemonic, or importable .arc seed file
- target coin
- intended derivation path
- intended address range

Conditionally required:

- BIP39 passphrase if the wallet used one
- script type for UTXO-like coins if Auto is not sufficient
- .arc password for password-protected .arc
- keyfile for keyfile-protected .arc
- keyfile plus password for combined-mode .arc
- encrypted keyfile password if the keyfile is encrypted

For exchange or custodian deposits:

- XRP destination tag if required by receiver
- Stellar memo if required by receiver

Destination tags and memos are not derived from the seed.

Pre-Session Checklist

Before entering real recovery material:

```
- Confirm local file is Arculus_Recovery.html v1.6.0.
- Disconnect networking.
- Disable browser sync and extensions where practical.
- Use a private physical environment.
- Decide whether the goal is verification, seed backup, address discovery,
  private-key export, QR export, or migration.
```

```
- Know whether a BIP39 passphrase was used.
- Gather any known historical addresses or root fingerprint.
- Prepare storage for any exported artifacts.
```

Do not start recovery on a machine you believe is compromised.

Mnemonic Entry Procedure

Use direct mnemonic entry when the words are available.

Procedure:

- Select 12 or 24 words.
- Enter words in the mnemonic textarea or numbered word grid.
- The internal mnemonic normalization is:

  ```
  NFKD((text || "").trim().split(/\s+/).filter(Boolean).join(" "))
  ```

- Validate word count.
- Validate every word against the BIP39 English list.
- Reconstruct the BIP39 bitstring from 11-bit indexes.
- Split entropy and checksum bits.
- Compute SHA-256(entropy).
- Compare expected checksum bits.
- If checksum is invalid, stop and correct the phrase.

Supported word counts:

- 12 words: 128 entropy bits, 4 checksum bits
- 24 words: 256 entropy bits, 8 checksum bits

v1.6.0 does not support 15-, 18-, or 21-word mnemonic recovery.

Generated Seed Procedure

Generate Seed creates a new mnemonic, not a recovery of an existing wallet.

Procedure:

- Select 12 or 24 words.
- Click Generate Seed.
- Entropy bytes are sampled with crypto.getRandomValues.
- Checksum bits are computed with SHA-256(entropy).
- 11-bit indexes map into BIP39_WORDS.
- The generated mnemonic is validated.
- The seed phrase is hidden on screen.

Use generated seeds only when intentionally creating a new recovery phrase. Generating a seed cannot recover an existing wallet.

BIP39 Passphrase Procedure

If the wallet used a BIP39 passphrase, enter it in the main Passphrase field before validation/derivation.

BIP39 seed:

```
PBKDF2-HMAC-SHA512(
    password   = UTF-8(NFKD(normalized mnemonic)),
    salt       = UTF-8("mnemonic" + NFKD(passphrase)),
    iterations = 2048,
```

```
    dkLen      = 64
)
```

Rules:

- Empty passphrase is valid.
- Wrong passphrase gives no error.
- Wrong passphrase gives valid wrong keys.
- The .arc file does not store the BIP39 passphrase.
- The .arc password is a different secret.

Verification:

- Compare root fingerprint when known.
- Compare known addresses.
- Do not rely on checksum alone.

.arc Import Procedure

Seed file import reads the selected file as text and calls parseSeedBundle.

Armor path:

```
if trimmed file starts with "ARCULUS-ARC-V2":
    remove first line
    join remaining lines
    base64-decode
    parse decoded UTF-8 JSON
```

Raw JSON path:

```
if no armor header:
    parse file text as JSON
```

Plain .arc:

```
magic == "ARCULUS-ARC"
format == "arculus-plain-seed-v1"

The mnemonic is read directly, normalized, and BIP39-validated. No credential
is requested.
```

Protected .arc:

```
magic == "ARCULUS-ARC"
format == "arculus-encrypted-seed-v2"
version == 2

The user supplies the required credential mode. MAC verification occurs before
decryption. If MAC verification fails, the password/keyfile is wrong or the
file is corrupted/tampered.
```

Credential hint:

```
credential_mode, credentialMode, or protection can select/lock the import
dialog mode. This hint is operational and must not be treated as proof of
cryptographic policy.
```

After import:

```
- The mnemonic is hidden on screen.
- Seed state remains available internally for validation, copy, export, and
  derivation.
- The root fingerprint refresh is scheduled.
```

Protected .arc Credential Decision

Password mode:

```
Need exact ARC password.
```

Keyfile mode:

```
Need exact keyfile bytes.
```

Combined mode:

```
Need exact keyfile bytes and exact combined-mode password.
```

Encrypted keyfile:

```
Need exact encrypted-keyfile password to recover keyfile bytes.
```

If import fails:

```
- Check that the credential mode is correct.
- Check that the selected keyfile is the original one.
- Check password spacing and capitalization.
- Check whether the keyfile itself is encrypted.
- Do not assume the BIP39 passphrase can open the .arc; it cannot unless it
  was deliberately reused as the ARC credential.
```

Default Derivation Paths

v1.6.0 default paths:

```
bitcoin       m/0'
bitcoincash   m/0'
litecoin      m/84'/2'/0'
dogecoin      m/44'/3'/0'
ethereum      m/44'/60'/0'
solana        m/44'/501'/0'
stellar       m/44'/148'/0'
monero        m/44'/128'/0'
cardano       m/1852'/1815'/0'/0/0
xrp           m/44'/144'/0'
```

The Bitcoin and Bitcoin Cash default m/0' reflects Arculus-specific recovery behavior. Other wallets often use BIP44/BIP49/BIP84/BIP86 account paths.

Common Path Reference

Bitcoin:

```
Legacy P2PKH:
    m/44'/0'/0'
    addresses begin 1

Wrapped SegWit:
    m/49'/0'/0'
    addresses begin 3

Native SegWit:
    m/84'/0'/0'
    addresses begin bc1q

Taproot:
    m/86'/0'/0'
    addresses begin bc1p

Arculus native:
    m/0'
```

Bitcoin Cash:

```
Arculus/native default:
    m/0'

Common BIP44:
    m/44'/145'/0'
```

Litecoin:

```
Legacy:
    m/44'/2'/0'

Wrapped SegWit:
    m/49'/2'/0'

Native SegWit:
    m/84'/2'/0'
```

Dogecoin:

```
Default:
    m/44'/3'/0'
```

Ethereum:

```
m/44'/60'/0'
```

Solana:

```
m/44'/501'/0'
```

Stellar:

```
m/44'/148'/0'
```

Monero:

```
m/44'/128'/0'
```

Cardano:

```
m/1852'/1815'/0'/0/0
```

XRP:

```
m/44'/144'/0'
```

Path Parser Rules

parsePath(path):

- trims path
- splits on "/"
- skips leading "m" if present
- ignores empty path components
- hardened suffixes: "'", "h", "H"
- indexes must match /^\d+$/
- index must be >= 0 and < 0x80000000 before hardening
- hardened index adds 0x80000000

normalizePath(path):

- parses the path
- emits "m" for empty path
- emits "m/<index>'" for hardened components and "m/<index>" otherwise

Invalid path components stop derivation.

Auto Script Resolution

purposeToScript(path):

- parse path
- if no components: p2pkh
- first component without hardened bit determines purpose

Mapping:

```
86 -> p2tr
84 -> p2wpkh
49 -> p2wpkh-p2sh
otherwise -> p2pkh
```

Therefore:

```
m/0' maps to p2pkh
m/44'/... maps to p2pkh
m/49'/... maps to p2wpkh-p2sh
m/84'/... maps to p2wpkh
m/86'/... maps to p2tr
```

If the original wallet used a non-standard script type, select it explicitly instead of relying on Auto.

Address Count and Range

Without range:

```
count input determines indexes:

  0 through count - 1
```

With range A-B:

```
start = max(0, A)
end = max(start, B)
count = end - start + 1
```

UTXO-style secp256k1 output:

```
For each requested index, derive:

  branch 0 receiving
  branch 1 change
```

Account-style output:

```
Solana, Stellar, Monero, and Cardano use coin-specific row generation
specified in Derivation.txt.
```

Recovery rule:

```
If a known address is missing, increase range and check both branches before
concluding the path/passphrase is wrong.
```

Verification Procedure

Before using private key material:

- Confirm mnemonic checksum is valid.
- Confirm the BIP39 passphrase state: empty or exact non-empty string.
- Compare root fingerprint if known.
- Confirm coin.
- Confirm derivation path.
- Confirm script type.
- Confirm receive address at expected index.
- For UTXO coins, confirm change branch when relevant.
- Confirm address encoding form.
- Export only after verification.

Minimum evidence:

```
At least one known historical address should match. For stronger confidence,
match multiple addresses across expected indexes/branches.
```

Address Location Procedure

If original settings are unknown:

```
1. Start with empty BIP39 passphrase unless documentation says otherwise.
2. Try the wallet's known default path if known.
3. For Bitcoin:
     m/0'
     m/44'/0'/0'
     m/49'/0'/0'
```

```
     m/84'/0'/0'
     m/86'/0'/0'
4. For Bitcoin Cash:
     m/0'
     m/44'/145'/0'
5. For Litecoin:
     m/44'/2'/0'
     m/49'/2'/0'
     m/84'/2'/0'
6. For account coins, try the documented default first.
7. Increase range gradually.
8. Check receiving and change branches for UTXO coins.
9. If no match and a passphrase may have been used, test documented
   passphrase candidates carefully.
```

Do not brute-force personal passphrases on the recovery workstation unless that is an intentional incident-response activity. Every candidate derives a real wallet tree and may create confusion if exported.

Bitcoin Cash Display Choice

v1.6.0 default Bitcoin Cash display/export uses CashAddr when the legacy toggle is off.

Operational issue:

```
Some older systems recorded legacy Base58 Bitcoin Cash addresses. The same key
can correspond to CashAddr and legacy display forms.
```

Recovery rule:

```
If a BCH address appears not to match, verify whether the historical record is
legacy Base58 or CashAddr before changing the seed/path.
```

XRP Recovery Boundary

XRP derives a classic r-address and private key material.

Destination tags:

- not derived
- not stored in the seed
- not inferable from private key
- assigned by exchanges/custodians

If recovering funds sent to an exchange, obtain the required destination tag from that exchange. The QR subsystem can include a user-entered tag in QR payloads, but derivation does not produce it.

Stellar Memo Boundary

Stellar account keys and addresses derive from the seed. Exchange memo fields do not. A Stellar address alone may not be sufficient for custodial deposits.

Confirm memo requirements with the receiving service before moving funds.

Ethereum and ERC-20 Boundary

Ethereum derivation yields the account address and private key. ERC-20 tokens share the same address/private key relationship as ETH on the same chain.

The tool does not:

- identify token balances
- identify chain IDs
- identify L2 networks
- verify contract addresses
- estimate gas

Verify the intended chain/network in the wallet used for sweeping.

Monero Boundary

Monero output includes spend/view material as specified in Derivation.txt.

The tool does not scan the Monero blockchain. It cannot determine balance or transaction history. Wallet restore height and chain scanning happen in Monero wallet software, not in Arculus Recovery.

Cardano Boundary

Cardano output uses the v1.6.0 Icarus/CIP-1852 behavior specified in Derivation.txt.

The tool derives address/key material. It does not query staking state, rewards, UTxOs, or wallet account discovery. Confirm address compatibility with the wallet you use for recovery.

Export Selection

Derived JSON:

```
Most complete machine-readable export. Contains nested account fields and all
generated row fields.
```

CSV:

```
Flattened rows for spreadsheet review. Not lossless.
```

TXT:

```
Human-readable deterministic text. Not lossless.
```

PDF:

```
Visual report with selected columns and extended key section. Not encrypted.
```

.arc:

```
Seed backup. Protected .arc restores mnemonic only. It does not store BIP39
passphrase.
```

QR PNG:

```
Address payload image. Not seed backup.
```

Export only the artifact required for the recovery. Plaintext private-key exports are often more sensitive than a protected .arc file.

Sweeping Procedure

Arculus Recovery does not sign or broadcast transactions. To move funds:

1. Derive and verify the target private key or account secret offline.
2. Transfer only the minimum private material needed to a trusted signing wallet.
3. Sweep funds to a new wallet generated outside the compromised context.
4. Verify transaction details on the signing device/software.
5. Broadcast from an online machine that does not retain broader seed material.

Prefer sweeping to a new mnemonic rather than continuing to use a seed that was entered into a general-purpose environment.

Cleanup Procedure

After recovery:

1. Close QR modal if open.
2. Export or record only required artifacts.
3. Delete unneeded plaintext exports.
4. Click Clear All.
5. Confirm mnemonic, word grid, passphrase, output, and root fingerprint are cleared.
6. Close browser or desktop wrapper.
7. Power off live OS or reboot before reconnecting network.

Clear All is not memory erasure. Treat full power-off of a live environment as the stronger cleanup step.

Common Failure Modes

Valid mnemonic, no funds:

```
Possible causes:
    wrong BIP39 passphrase
    wrong path
    wrong script type
    wrong branch
    wrong account index
    wrong coin
    address range too narrow
```

.arc will not open:

```
Possible causes:
    wrong ARC password
    wrong keyfile
    wrong combined-mode password
    encrypted keyfile not decrypted
    corrupted/tampered file
    unsupported legacy file
```

QR accepted but payment fails:

```
Possible causes:
    missing XRP destination tag
    missing Stellar memo
    wrong network/chain
    receiving wallet does not accept URI form
    address copied without memo/tag metadata
```

PDF/CSV missing fields:

```
Use JSON for complete machine-readable output. PDF/CSV/TXT are projections.
```

Out-of-Scope Behavior

The recovery tool does not:

- broadcast transactions
- sign transactions
- query balances
- discover used addresses
- estimate fees
- identify token balances
- validate XRP destination tags
- validate Stellar memos
- validate exchange deposit policies
- recover forgotten BIP39 passphrases
- recover forgotten .arc passwords
- recover missing keyfiles

Final Recovery Rule

Do not act on a private key until the derived address has been verified against known wallet history or a known fingerprint/path record. Correct-looking output is not enough; deterministic recovery requires all inputs to be correct at the same time.

Procedure: Direct Mnemonic Recovery

Use this procedure when the mnemonic is available on paper or another offline record.

```
1. Prepare the offline environment according to OpSec.txt.
2. Open Arculus_Recovery.html locally.
3. Confirm v1.6.0.
```

```
4. Select 12 or 24 words.
5. Enter the mnemonic.
6. Enter BIP39 passphrase if used. Leave empty only if no passphrase was used
   or if testing the empty-passphrase candidate.
7. Validate the mnemonic.
8. Compare root fingerprint if known.
9. Select coin.
10. Confirm derivation path.
11. Confirm script type.
12. Set a small address count or range.
13. Derive.
14. Compare known address.
15. Expand range or test alternate path only if no known address matches.
16. Export only what is required.
17. Clear all and close session.
```

Do not export private keys before at least one known address matches.

Procedure: Protected .arc Recovery

Use this procedure when a protected .arc file is the mnemonic backup.

- Prepare offline environment.
- Place .arc file on local/removable media.
- Open Arculus_Recovery.html locally.
- Click Import Seed.
- Select .arc file.
- If credential hint selects a mode, verify it matches your backup notes.
- Enter password, select keyfile, or provide both factors.
- If keyfile is encrypted, enter encrypted-keyfile password.
- Import.
- If import succeeds, seed is hidden on screen.
- Enter BIP39 wallet passphrase if the wallet used one.
- Validate root fingerprint/address as usual.

If import fails, do not keep trying random passwords in the same session without notes. Record the failure, verify mode and files, and restart cleanly if needed.

Procedure: Backup Migration

Use this when converting an older or uncertain backup into current v1.6.0 form.

- Import the old .arc or mnemonic offline.
- Validate mnemonic checksum.
- Enter BIP39 passphrase if used.
- Confirm root fingerprint or known address.
- Export a new protected .arc.
- Choose password, keyfile, or combined mode intentionally.
- Save all required factors.
- Clear session.
- Re-import the new .arc using the saved factors.
- Confirm the same root fingerprint.
- Only then retire the old backup.

Never delete the only known-good backup before verifying the replacement.

Procedure: Address Discovery

Use this when the mnemonic is known but the original wallet path is unknown.

For each candidate:

- Set BIP39 passphrase candidate.
- Set coin.
- Set path.
- Set script type or Auto.
- Derive a modest range, for example 0-20.
- Check receiving branch.
- Check change branch for UTXO coins.
- Record candidate result without exporting private keys.

Candidate order should prioritize documented wallet defaults over guesses.

Stop when:

```
- a known historical address matches
- root fingerprint matches and path-specific address evidence matches
- all documented candidates are exhausted and further work becomes passphrase
  recovery rather than path recovery
```

Procedure: Single Private Key Recovery

Use this when the target is one known address.

```
1. Recover mnemonic/passphrase.
2. Set coin/path/script.
3. Use Range around the suspected index if known.
4. Derive.
5. Locate exact address match.
6. Export JSON or copy the single required private field manually from the
   offline display.
7. Avoid full-account PDF/JSON export if one key is enough.
8. Sweep funds to a new wallet.
9. Delete temporary exports.
```

Single private key recovery reduces blast radius compared with exporting an entire account key set.

Procedure: Full Account Recovery

Use this when the target wallet software needs an account-level import or when many addresses must be recovered.

```
1. Verify mnemonic/passphrase with root fingerprint.
2. Verify path/script with known address.
3. Export JSON if machine-readable account fields are needed.
4. Export PDF only for human review/printing.
5. Treat account xprv fields as Class B secrets.
6. Import/sweep in trusted wallet software.
7. Move funds to a newly backed-up wallet if the seed was exposed beyond the
   intended offline environment.
```

An account extended private key can derive many child private keys. Handle it with broader caution than a single WIF key.

Decision Tree: No Known Address Match

If no known address matches:

```
1. Is mnemonic checksum valid?
     no -> fix mnemonic
     yes -> continue

2. Is BIP39 passphrase known?
     no -> test empty and documented candidates
```

```
     yes -> continue

3. Does root fingerprint match?
     no -> mnemonic/passphrase mismatch
     yes -> path/script/range issue likely

4. Is coin correct?
     no -> switch coin
     yes -> continue

5. Is path correct?
     unknown -> test common paths
     yes -> continue

6. Is script type correct?
     unknown -> test P2PKH, P2WPKH-P2SH, P2WPKH, P2TR where supported
     yes -> continue

7. Is range wide enough?
     no -> expand range
     yes -> continue

8. For UTXO, was change branch checked?
     no -> check change
     yes -> reassess source records
```

Documentation to Preserve

For future recovery, preserve non-secret metadata:

- app version used for backup
- root fingerprint
- coin
- derivation path
- script type
- first receive address
- whether BIP39 passphrase was used
- .arc credential mode
- storage locations of .arc/keyfile/password records

Do not put mnemonic words or private keys in the same metadata note unless that note is stored as a full secret.

Recovery Artifact Sensitivity

Protected .arc:

```
Contains encrypted mnemonic. Needs credential.
```

Unprotected .arc:

```
Contains plaintext mnemonic. Treat as mnemonic.
```

Raw keyfile:

```
Can unlock Keyfile-mode .arc. Treat as Class A when paired with .arc.
```

Encrypted keyfile:

```
Needs password. Still sensitive.
```

Derived JSON:

```
Most complete private-key export. Treat as highly sensitive.
```

CSV/TXT:

```
Plaintext private-key material if exported after derivation. Treat as highly
sensitive.
```

PDF:

```
Plaintext visual private-key material. Treat as highly sensitive.
```

QR PNG:

```
Address payload only if used correctly. Privacy-sensitive but not a private
key artifact.
```

Post-Recovery Seed Retirement

Retire the original seed when:

- mnemonic was entered on a machine that may have been online or compromised
- private-key exports were copied to unsafe media
- derived JSON/PDF was exposed
- protected .arc and credential were exposed together
- raw keyfile and .arc were exposed together
- screen capture or camera exposure may have recorded seed/private keys

Retirement procedure:

- Generate a new wallet in a trusted environment.
- Back up the new seed and passphrase if used.
- Move funds from old wallet to new wallet.
- Confirm receipt.
- Stop using the old seed.

Implementation Compatibility Checklist

A v1.6.0-compatible recovery implementation must:

```
- Support 12- and 24-word BIP39 English mnemonics.
- Normalize mnemonic whitespace before BIP39 seed derivation.
- Apply NFKD to mnemonic and BIP39 passphrase.
- Use PBKDF2-HMAC-SHA512 with 2048 iterations for BIP39 seed.
- Use current default paths listed in this document.
- Parse paths with "'", h, and H hardened suffixes.
- Map Auto script type by first purpose component.
- Derive UTXO receiving and change branches.
- Route Solana/Stellar/Monero/Cardano to their current v1.6.0 derivation
  families.
- Import ARCULUS-ARC-V2 protected and plain bundles.
- Preserve the distinction between BIP39 passphrase and .arc credentials.
```

Arculus Recovery v1.6.0 - Operational Security Specification

Scope

This document defines the operational security requirements for using Arculus_Recovery.html v1.6.0. It is tied to the actual implementation behavior: mnemonic generation and entry, BIP39 passphrase use, .arc export/import, password and keyfile credential modes, derived-output exports, QR generation, clipboard behavior, idle clearing, browser runtime limits, and native Tauri save behavior.

This document is not a cryptographic proof. Encryption.txt defines the cryptographic envelope. Derivation.txt defines how secrets are transformed into keys and addresses. FileFormat.txt defines the serialized artifacts. This document defines how an operator should handle those artifacts and runtime surfaces so the byte-level protections are not defeated by the environment.

Primary Rule

Treat every recovery session as handling live private key material.

Even if the UI currently shows only an address, the same session can contain:

- mnemonic words
- BIP39 wallet passphrase
- ARC password
- keyfile bytes
- combined credential material
- BIP39 seed
- root private keys
- account extended private keys
- address private keys
- Monero spend/view keys
- Stellar secret seeds
- Cardano private keys
- exported files containing any of the above

The correct operational stance is that the browser, operating system, display, clipboard, filesystem, removable media, and room are all part of the security boundary.

Threat Model

In scope:

```
Offline file acquisition:
    An attacker obtains a protected .arc file or encrypted keyfile after the
    session. Protection depends on credential entropy and the implemented KDF
    and MAC. Weak passwords remain weak even with PBKDF2.

File tampering:
    An attacker modifies protected .arc metadata, salt, nonce, ciphertext, or
    MAC. ARC v2 detects this through MAC verification before decryption.

Wrong-path recovery error:
    The mnemonic is correct, but the operator uses the wrong coin, BIP39
    passphrase, derivation path, script type, branch, or address index range.
    The tool will derive valid but wrong keys without an error.

Clipboard exposure:
    Mnemonic or address data is copied to the OS clipboard and observed by
    clipboard history, sync, remote desktop, or malware.

Export mishandling:
    JSON, CSV, TXT, PDF, raw keyfile, or unprotected .arc artifacts are stored
    on unsafe media or copied to a networked environment.
```

```
Browser/runtime residue:
    JavaScript strings, ArrayBuffers, object URLs, downloads, caches, swap,
    crash dumps, or hibernation files retain sensitive data after the UI is
    cleared.
```

Out of scope:

- A compromised operating system before the session begins.
- A malicious browser or browser extension with access to page memory.
- Firmware compromise, hardware implants, or physical keyloggers.
- Cameras or observers with screen/keyboard visibility.
- Coercion.
- An attacker who already knows the mnemonic or correct .arc credential.
- A malicious copy of Arculus_Recovery.html.

For high-value recovery, assume an ordinary daily-use online computer is not an acceptable secret-handling environment.

Secret Classes

Class A: whole-wallet recovery secrets

- BIP39 mnemonic
- BIP39 wallet passphrase when one was used
- protected .arc credential that unlocks the mnemonic
- raw keyfile used as sole .arc credential
- combined-mode password plus keyfile together

Class B: subtree or account secrets

- root xprv/yprv/zprv/tprv
- account xprv/yprv/zprv/tprv
- Cardano account private material
- Monero private spend/view key pair
- Stellar secret seed

Class C: single-address secrets

- private_key_hex
- private_key_wif
- taproot output private key
- Solana private key material
- XRP private key material

Class D: privacy-sensitive public data

- addresses
- xpub/ypub/zpub/tpub
- root fingerprint
- QR PNG files
- derivation paths and account ranges

Class E: operational metadata

- filenames

- created_at timestamps
- credential_mode hints
- app version

Class A, B, and C must never be copied to networked storage or ordinary cloud sync. Class D is public on-chain once used, but still links wallet activity and should be handled deliberately.

Offline Workstation Requirement

The safest operating mode is a machine disconnected from all networks before any secret is entered and kept disconnected until after the session is cleared and the application is closed.

Disable:

- Wi-Fi
- Ethernet
- Bluetooth
- cellular modem
- cloud sync clients
- remote desktop
- browser profile sync
- browser extensions where practical

The v1.6.0 UI uses navigator.onLine for the network indicator:

```
online:
    networkStatus class "online"
    title "Network route detected"

offline:
    title "No network connection detected"
```

This indicator is a convenience signal, not proof of isolation. Physical disconnection and OS-level controls are authoritative.

Recommended Session Procedure

Before the session:

1. Obtain the intended Arculus_Recovery.html build from trusted local media.
2. Verify the visible app version is v1.6.0.
3. Use a fresh live OS or a dedicated offline machine for high-value work.
4. Disable networking before opening the file.
5. Close unrelated applications.
6. Disable clipboard managers and screenshot tools.
7. Confirm no cameras or observers can see the screen or keyboard.
8. Decide the exact recovery goal: verify mnemonic, export .arc, derive a specific path, produce QR for an address, or recover a private key.

During the session:

1. Enter or import only the required mnemonic.
2. Enter the BIP39 wallet passphrase only if the original wallet used one.
3. Validate the mnemonic before deriving.
4. Compare the root fingerprint if a known fingerprint exists.
5. Select the coin and derivation path intentionally.
6. Derive the smallest address range that solves the task.
7. For UTXO wallets, check both receiving and change branches if funds are missing.
8. Export the minimum artifact required.
9. Prefer protected .arc over unprotected .arc for mnemonic storage.
10. Avoid clipboard use for seeds and private keys.

After the session:

- Use Clear All.
- Confirm derived output is no longer displayed.

- Close the browser or Tauri wrapper.
- Move required artifacts to controlled storage.
- Delete temporary files from unsafe locations.
- Power off the live OS; do not hibernate.
- For high-value work, reboot before reconnecting networks.

Mnemonic Handling

Mnemonic input and generated mnemonics are Class A secrets.

v1.6.0 behavior:

```
- Generated seeds use browser CSPRNG.
- Generated/imported seeds are hidden after generation/import.
- Manually entered seeds are promoted to importedSeedState and hidden after
  successful derivation.
- Hidden seed fields are displayed as bullet masks.
- "Show Seed" temporarily reveals the seed while held.
```

Operational rules:

- Do not type a real mnemonic into a networked browser.
- Do not paste a mnemonic from a clipboard populated on another machine.
- Do not photograph the word grid.
- Do not screen-share a session.
- Do not leave the session unattended while Show Seed is active.
- Do not rely on UI masking as memory erasure.

Masking prevents casual shoulder viewing. It does not remove the mnemonic from JavaScript memory.

BIP39 Wallet Passphrase Boundary

The passphrase input next to the mnemonic is the BIP39 wallet passphrase. It is not the .arc encryption password.

Operational consequences:

- Same mnemonic + different BIP39 passphrase = different wallet tree.
- Incorrect BIP39 passphrase does not produce an error.
- Incorrect BIP39 passphrase produces valid-looking wrong addresses.
- The BIP39 passphrase is not stored in protected .arc plaintext.
- The BIP39 passphrase must be backed up separately if used.

Before moving funds, verify known addresses under the intended passphrase.

Root Fingerprint Discipline

The UI displays:

```
Root Fingerprint: <hex>
```

Use it as a compact verification artifact when available. It is not a secret on the level of a private key, but it links a session to a wallet root.

Procedure:

- Record a root fingerprint from a known-good wallet before emergency use.
- After entering/importing a mnemonic and passphrase, compare the fingerprint.
- If the fingerprint differs, stop before exporting or sweeping keys.

A matching fingerprint does not prove path correctness. It only verifies the mnemonic/passphrase root.

Derivation Path and Script-Type Risk

The tool derives exactly what the UI requests. A wrong path is not a runtime error. It is a valid derivation of a different subtree.

High-risk inputs:

- coin selector
- derivation path
- script type
- address count
- range input
- BIP39 passphrase

Verification rules:

```
- Compare the first known address before using private keys.
- For Bitcoin-like wallets, check both receiving branch 0 and change branch 1.
- For restored exchange deposit addresses, verify destination tag or memo
  requirements separately.
- For Cardano, verify Shelley-era address expectations.
- For Monero, recognize that view/spend material is sensitive even when only
  one address is shown.
```

.arc Export Security

Protected .arc files contain the mnemonic encrypted under one credential mode:

```
password
keyfile
keyfile + password
```

Unprotected .arc files contain the plaintext mnemonic inside base64 armor.

Protected .arc requirements:

```
- Use high-entropy credentials.
- Verify import immediately after export.
- Store credential material separately from the .arc file.
- Do not upload .arc files to cloud storage unless the threat model explicitly
  accepts offline password guessing risk.
```

Unprotected .arc requirements:

- Use only for test data or controlled offline transfer.
- Treat as equivalent to the mnemonic.
- Do not store on general-purpose disks.

Default saved filename:

```
Arculus_Encrypted_Seed.arc
```

The filename does not prove the file is encrypted. v1.6.0 uses this filename for both protected and unprotected seed exports. Inspect the decoded bundle format if protection status matters:

```
Protected:
    format == "arculus-encrypted-seed-v2"

Unprotected:
    format == "arculus-plain-seed-v1"
```

Password Mode

Password mode passes the user-entered password directly to ARC v2 encryption.

Minimum operational requirements:

```
- Do not reuse an account password.
- Do not use a PIN, date, name, address, phone number, phrase from personal
  history, or single dictionary word.
- Prefer a randomly generated multi-word passphrase or long random string.
```

```
- Record the password offline.
- Store it separately from the .arc file.
```

Losing the password makes the protected .arc file unrecoverable.

Keyfile Mode

Keyfile mode converts keyfile bytes to:

```
"arc-keyfile-v1:" + base64(keyfile_bytes)
```

and uses that string as the .arc credential.

Generated raw keyfiles contain 64 random bytes and are saved as:

```
Arculus_Recovery_Keyfile.key
```

Operational requirements:

- Treat the raw keyfile as a Class A secret.
- Store it separately from the .arc file.
- Do not leave a raw keyfile on a networked machine.
- Keep at least one verified backup copy.
- Verify that the .arc imports with the stored keyfile before relying on it.

If an attacker obtains both the protected .arc and raw keyfile, password guessing is unnecessary for Keyfile mode.

Keyfile + Password Mode

Combined mode requires both:

- keyfile bytes
- password

The app derives:

```
"arc-combined-v1:" + base64(SHA-256(material))
```

and uses that as the .arc credential.

Generated encrypted keyfiles are saved as:

```
Arculus_Recovery_Keyfile.enc.key
```

Operational requirements:

```
- Store the .arc, encrypted keyfile, and password separately.
- Verify the full import path immediately after export.
- Do not assume the encrypted keyfile password is recoverable from the .arc.
- Do not assume the .arc password can open the encrypted keyfile unless that
  was the password used when generating it.
```

Losing any required component prevents recovery.

Encrypted Keyfile Security

Encrypted keyfiles protect keyfile bytes with PBKDF2-HMAC-SHA512 and an HMAC-SHA512 stream/MAC construction. They are safer to store than raw keyfiles but still require a high-entropy password.

Operational notes:

```
- The encrypted keyfile MAC authenticates salt, nonce, and ciphertext.
- The encrypted keyfile metadata is less extensively MAC-bound than ARC v2
  seed metadata.
- The decrypted keyfile bytes exist in browser memory during import.
- The encrypted keyfile password is separate from the BIP39 wallet passphrase.
```

Derived Output Exports

Derived JSON, CSV, TXT, and PDF exports are plaintext secret-bearing artifacts.

They can contain:

- root extended private keys
- account extended private keys
- private_key_hex
- private_key_wif
- taproot private keys
- Stellar secret material
- Monero private spend/view keys
- Cardano payment and stake private keys
- XRP private keys

JSON is the most complete export. CSV and TXT flatten or display fields. PDF is optimized for human reading and clips long cells in the table, while extended keys are printed separately when present.

Operational rules:

- Export only the format needed.
- Prefer deriving a narrow range.
- Do not email, cloud-sync, or chat-share exports.
- Delete temporary exports after use.
- Remember that PDF files may be indexed by desktop search tools.
- Remember that browser downloads may persist in download history.

QR Export Boundary

QR export is designed for address payloads, not seeds or private keys.

v1.6.0 behavior:

- QR modal accepts a user-entered address.
- XRP r-address input prompts for an optional destination tag.
- XRP QR payload is address + "?dt=" + tag-or-00000.
- Non-XRP QR payloads may add coin URI schemes.
- Cardano QR payload is the raw address.
- Save PNG writes qr_<sanitized-address>.png.
- Copy Address copies the address input, not necessarily the full QR label.

Operational risks:

```
- Address QR codes can link wallet activity when scanned by online services.
- XRP destination tags are operational payment metadata and must be verified
  with the receiving service.
- QR PNG files can persist in downloads, photo libraries, or synced folders.
```

Never encode seed words or private keys in the QR modal.

Clipboard Boundary

Copy Seed:

- Shows a confirmation prompt.
- Copies normalized mnemonic words joined by spaces.
- Displays a 60-second countdown.

- Attempts to clear the clipboard by writing an empty string.

Constants:

```
CLIPBOARD_CLEAR_SECS = 60
```

Fallback copy path:

```
If navigator.clipboard.writeText is unavailable, the app creates a temporary
textarea, selects it, calls document.execCommand("copy"), and removes it.
```

Limitations:

- Clipboard managers can retain history.
- Cloud clipboard sync can copy secrets to other devices.
- Remote desktop software can observe clipboard data.
- Some operating systems or browsers may reject clipboard clearing.
- Clearing the clipboard does not erase prior clipboard-manager records.

Operational rule:

```
Treat clipboard use as an explicit secret export.
```

Idle Timeout and Clear All

Constants:

```
IDLE_TIMEOUT_MS = 5 * 60 * 1000
warning time = 60 seconds before timeout
```

Events that reset the timer:

```
mousedown
keydown
touchstart
scroll
```

At warning:

```
Banner:
    Idle timeout in 60 seconds. All fields will be cleared.
```

At timeout:

```
handleClearAll()
Banner:
    Session cleared due to inactivity.
```

Clear All behavior:

- clears imported seed state
- clears mnemonic textarea
- clears word-grid inputs
- removes valid/invalid classes from mnemonic inputs
- clears BIP39 passphrase input
- clears raw output textarea
- clears root fingerprint display
- resets output view
- sets lastDerivedOutput = null
- reports "All fields cleared."

Limitations:

```
Clear All does not guarantee erasure from JavaScript engine internals,
browser process memory, WebCrypto internals, OS swap, hibernation files,
crash dumps, GPU/compositor memory, clipboard managers, download history, or
```

```
already-saved files.
```

Browser Runtime Risk

Arculus_Recovery.html is a local offline browser application, but browser runtimes are not hardened secret vaults.

Risks:

```
- JavaScript strings cannot be reliably zeroized.
- Browser extensions may observe page content.
- Developer tools can inspect memory and DOM state.
- Autofill, history, screenshots, accessibility tools, and crash reporters
  can retain data depending on environment.
- Downloads and object URLs are explicit egress paths.
- The browser profile may sync settings or metadata.
```

Controls:

- Use a clean browser profile.
- Disable extensions.
- Disable sync.
- Avoid developer tools.
- Prefer a live OS for high-value sessions.
- Close the browser after use.

Tauri Native Save Risk

When running inside a Tauri wrapper, saveBlob can pass file content to a native save bridge as base64:

```
save_export(filename, contentBase64)
```

Operational consequences:

- Secret-bearing exports cross from the webview into native code.
- The native layer may write to paths outside browser downloads.
- The UI may display "Export saved to <path>".
- Native export failure is reported and logged to console.

Treat Tauri native save paths as normal filesystem writes of the underlying secret-bearing artifact.

Storage Media

Recommended:

- encrypted removable media
- write-once archival media for finalized backups where appropriate
- physically separate storage for .arc, keyfile, and password
- printed or engraved password backups for high-value offline storage

Avoid:

- cloud-synced folders
- email attachments
- chat uploads
- unencrypted USB drives
- daily-use Downloads folders
- network shares
- managed corporate endpoints

SSD warning:

```
Secure deletion on SSDs is not reliable because of wear leveling and spare
blocks. Avoid writing plaintext secrets to SSDs rather than relying on later
deletion.
```

Verification Before Funds Movement

Before sweeping, importing, or moving funds:

- Confirm coin.
- Confirm network is mainnet if intended.
- Confirm BIP39 passphrase.
- Confirm derivation path.
- Confirm script type.
- Confirm address index range.
- Compare at least one known historical address.
- For UTXO assets, check receiving and change branches.
- For XRP, confirm destination tag requirements separately.
- For Stellar, confirm memo requirements separately.
- For Ethereum/ERC-20, confirm chain/network in the destination wallet.

Never conclude that a seed is empty until plausible alternate paths and change branches have been checked.

Incident Response

If a mnemonic was exposed:

- Assume all assets controlled by that mnemonic are at risk.
- Use a clean offline environment to derive needed keys.
- Move funds to a new seed generated on trusted hardware/software.
- Do not reuse the exposed seed.

If an account extended private key was exposed:

- Treat all descendants below that account as compromised.
- Move funds from affected paths.
- Do not keep using that account subtree.

If a single private key was exposed:

- Move funds from that address.
- For UTXO wallets, consider address reuse and change-output exposure.

If a protected .arc file was exposed:

- Evaluate credential entropy.
- If the credential was weak or reused, treat the mnemonic as compromised.
- If using Keyfile mode, determine whether the keyfile was also exposed.
- If using combined mode, determine whether both factors were exposed.
- Re-export with stronger credentials if the mnemonic remains trusted.

If a raw keyfile was exposed:

```
1. Treat any .arc protected only by that keyfile as exposed if the .arc file
   is also accessible to the attacker.
```

```
2. Re-export affected .arc files with a new keyfile or combined mode.
```

If an encrypted keyfile was exposed:

- Evaluate encrypted-keyfile password entropy.
- If weak, generate a new keyfile and re-export.

Disposal Procedure

After recovery:

```
- Close the QR modal.
- Clear visible seed and output state.
- Delete unneeded derived exports.
- Empty browser downloads only if doing so is meaningful for the storage
  medium.
- Remove temporary files from removable media.
- Power down the offline system.
- Store required backups in separate physical locations.
```

For high-value sessions, prefer destroying the live session by full power-off rather than trusting in-app clearing.

Minimum Acceptable Handling by Artifact

Protected .arc:

```
Store on encrypted removable media. Store password/keyfile separately.
```

Unprotected .arc:

```
Store only like a raw mnemonic. Avoid except for controlled test/offline use.
```

Raw keyfile:

```
Store like a high-value secret. Separate from .arc.
```

Encrypted keyfile:

```
Store separately from .arc and password. Use a high-entropy password.
```

Derived JSON/CSV/TXT/PDF:

```
Treat as plaintext private-key export. Delete after use unless there is a
deliberate archival reason.
```

QR PNG:

```
Treat as address metadata. Avoid cloud photo sync and public sharing if wallet
privacy matters.
```

Printed notes:

```
Store in tamper-evident physical controls. Separate mnemonic, passphrase,
.arc credential, and keyfile where possible.
```

Operator Checklist

Before entering a real seed:

- v1.6.0 build confirmed
- offline state confirmed
- clean browser/profile confirmed
- no extensions or sync
- private physical environment
- intended path/coin/range known
- output destination selected

Before exporting:

- mnemonic checksum valid

- root fingerprint checked if available
- BIP39 passphrase checked
- credential mode chosen intentionally
- export type understood

Before ending:

- import-tested any new .arc backup
- copied required keyfile/password material
- deleted unneeded plaintext exports
- Clear All executed
- app closed
- offline machine powered down

Non-Negotiable Warnings

```
- A protected .arc is only as strong as the credential needed to open it.
- A raw keyfile plus its .arc is equivalent to the ability to recover the
  mnemonic.
- A derived-output JSON file can be more immediately dangerous than a
  protected .arc because it may contain private keys in plaintext.
- The BIP39 wallet passphrase is not saved in the .arc file.
- UI masking is not memory erasure.
- Clipboard clearing is best effort.
- Browser offline operation is necessary but not sufficient on a compromised
  machine.
- Wrong derivation settings produce valid wrong addresses.
```

Airgap Preparation Detail

For high-value sessions, perform these checks before opening the HTML file:

- Remove Ethernet cables physically.
- Disable Wi-Fi in firmware or hardware controls where possible.
- Disable Bluetooth in firmware or hardware controls where possible.
- Remove USB network adapters.
- Disable cellular modems.
- Use a local copy of Arculus_Recovery.html, not a remote URL.
- Confirm the browser did not restore prior tabs into online services.
- Confirm password managers and form-fill extensions are disabled.
- Confirm no cloud storage client is syncing the working directory.

The navigator.onLine indicator can be wrong in both directions. It can report offline when a specialized route exists, and it can report online due to local network conditions. Treat it as a reminder, not an attestation.

Live OS Guidance

The preferred high-value environment is a live operating system that does not write session state to persistent storage.

Recommended properties:

- no persistent home directory
- no swap partition
- hibernation disabled
- browser profile created fresh for the session

- no browser sync
- no extensions
- no network autoconnect

If a live OS is not available, use a dedicated offline machine. A dedicated machine is still weaker than a live OS if it has persistent storage, because secrets can survive in swap, browser cache, crash dumps, filesystem journals, thumbnails, or search indexes.

Password Entropy Guidance

Use high-entropy credentials for protected .arc files and encrypted keyfiles.

Unacceptable:

- one dictionary word
- names
- dates
- phone numbers
- addresses
- keyboard patterns
- reused website passwords
- short phrases from memory
- PINs

Acceptable:

- randomly generated multi-word passphrase from a large wordlist
- randomly generated long character string
- password-manager-generated secret recorded offline

The KDF slows each guess. It does not make a predictable password safe. A captured protected .arc file can be tested offline without interacting with the original machine.

Backup Separation Patterns

Password mode:

```
Location A:
    protected .arc

Location B:
    password record

Optional Location C:
    BIP39 wallet passphrase record if used
```

Keyfile mode:

```
Location A:
    protected .arc

Location B:
    raw keyfile

Optional Location C:
    BIP39 wallet passphrase record if used
```

Combined mode:

```
Location A:
    protected .arc

Location B:
    keyfile or encrypted keyfile
```

```
Location C:
    combined-mode password

Optional Location D:
    BIP39 wallet passphrase record if used
```

Do not store all factors in one bag, one cloud account, one password manager entry, one USB drive, or one safe if the threat model includes theft of that container.

Validation of New Backups

Every newly created seed backup must be tested before it is trusted.

Protected .arc validation:

- Export protected .arc.
- Clear the current session or open a fresh offline session.
- Import the .arc with the intended credential.
- Confirm BIP39 validation succeeds.
- Confirm the root fingerprint matches the pre-export session.
- Confirm at least one expected address derives.

Keyfile validation:

- Verify the stored keyfile can be selected and read.
- Verify encrypted keyfiles decrypt with the recorded password.
- Verify the keyfile opens the protected .arc.

Failure to test a backup is equivalent to not having a backup.

Handling Legacy Files

v1.6.0 can import older seed formats. Legacy import support is for migration, not for continued archival use.

After importing a legacy .arc:

```
1. Confirm the mnemonic and root fingerprint.
2. Export a new protected v1.6.0 .arc.
3. Test import of the new file.
4. Retire the legacy file according to the sensitivity of the original
   storage medium.
```

Legacy files may have lower KDF cost or weaker metadata authentication. Keeping them indefinitely preserves the older risk profile.

Printing and Paper Handling

Printing derived outputs or recovery material changes the risk from digital storage to physical custody.

Rules:

- Do not print from a networked printer.
- Do not use office or shared printers.
- Do not print through cloud print services.
- Verify printer memory and job history behavior if possible.
- Retrieve pages immediately.
- Destroy misprints.
- Store paper in tamper-evident physical controls.

PDF export is not encrypted. Printing a PDF does not erase the PDF file.

Camera and Optical Risks

Seed words, private keys, QR codes, and addresses can be captured optically.

Controls:

- Cover laptop cameras if not needed.
- Remove phones and smart watches from the workspace.
- Avoid rooms with security cameras.
- Avoid reflective surfaces facing the display.
- Do not use video calls during recovery.
- Do not create screenshots as notes.

QR images are easy to scan at a distance when shown full-size. Treat QR display as public disclosure of the payload.

Multi-Asset Recovery Planning

When recovering wallets that may contain multiple coins:

- Work one coin at a time.
- Record derivation settings used for each coin.
- Verify addresses before exporting private keys.
- Avoid exporting all coins if only one asset needs recovery.
- Avoid mixing test mnemonics and real mnemonics in the same session.
- Clear state between unrelated recoveries.

The blast radius of a full JSON export is larger than the blast radius of a single address private key. Choose the smallest artifact that completes the recovery.

Audit Trail Without Secret Leakage

For operational records, keep non-secret metadata separate from secrets.

Safe-ish metadata to record:

- app version v1.6.0
- date of backup creation
- coin
- derivation path
- script type
- address count or range
- root fingerprint
- storage location labels

Do not include:

- mnemonic
- BIP39 passphrase
- .arc password
- keyfile bytes
- private keys
- xprv values

- screenshots of the app

Even public addresses can be privacy-sensitive. If privacy matters, record only the minimum needed to reproduce the recovery.

End-State Decision

After an emergency recovery, decide whether the original seed remains trusted.

Keep using the seed only if:

- it was never exposed outside the intended offline environment
- no plaintext private-key exports were mishandled
- no weak .arc credential was exposed with the .arc file
- no malware or physical observer risk was discovered

Retire the seed if:

- mnemonic was copied to a networked clipboard
- derived JSON/PDF with broad private material was uploaded or synced
- a raw keyfile and protected .arc were both exposed
- a weak password protected an exposed .arc
- the recovery was performed on a machine later believed compromised

Seed retirement means moving funds to a newly generated wallet under a new mnemonic and new backup set.

Arculus Recovery v1.6.0 - Passphrase and Secret Handling Specification

Scope

This document specifies the exact role of passphrases and user-entered secrets in Arculus_Recovery.html v1.6.0. It distinguishes the BIP39 wallet passphrase, the protected .arc credential, raw keyfile material, encrypted keyfile passwords, and Keyfile + Password combined secrets.

The most important distinction:

```
BIP39 passphrase:
    Changes wallet derivation.

.arc credential:
    Opens or creates a protected seed-backup file.

encrypted keyfile password:
    Opens or creates an encrypted keyfile.
```

These secrets are independent unless the operator deliberately reuses the same string. Reuse is allowed by the UI but is not recommended as an operational default.

Secret Classes

v1.6.0 handles these secret classes:

```
mnemonic:
    12- or 24-word BIP39 English phrase.

BIP39 passphrase:
    Optional passphrase field on the main screen. Sometimes called a "25th
    word", but technically an arbitrary Unicode string.

ARC password:
    Password entered in the Encrypt / Export Seed dialog when Password mode
    is selected.

raw keyfile bytes:
    64 random bytes generated by v1.6.0 keyfile export, or imported key
    material of at least 32 bytes.

encrypted keyfile password:
    Password used to protect raw keyfile bytes in
    Arculus_Recovery_Keyfile.enc.key.

combined ARC secret:
    ASCII string derived from combined password plus keyfile secret:
    "arc-combined-v1:" + base64(SHA-256(material)).

BIP39 seed:
    64-byte PBKDF2-HMAC-SHA512 output from mnemonic and BIP39 passphrase.

derived private material:
    BIP32 private keys, WIF keys, Ed25519 keys, Monero private spend/view
    keys, Cardano private keys, and related account secrets.
```

BIP39 Passphrase Definition

The BIP39 passphrase is entered in the main Passphrase field:

```
<input id="passphrase" type="password">
```

It is consumed by bip39Seed(mnemonic, passphrase):

```
m = NFKD((mnemonic || "").trim().split(/\s+/).filter(Boolean).join(" "))
p = NFKD(passphrase || "")

seed = PBKDF2-HMAC-SHA512(
    password   = UTF-8(m),
    salt       = UTF-8("mnemonic" + p),
    iterations = 2048,
    dkLen      = 64
)
```

The passphrase is part of the PBKDF2 salt, prefixed by the literal ASCII string "mnemonic". The mnemonic is the PBKDF2 password. This follows BIP39.

The empty string is a valid passphrase:

```
p = ""
salt = UTF-8("mnemonic")
```

Every distinct passphrase string, including the empty string, creates a different 64-byte seed and therefore a different wallet tree.

What the BIP39 Passphrase Is Not

The BIP39 passphrase is not:

- the .arc password
- the encrypted keyfile password
- a UI login password
- stored in protected .arc plaintext
- stored in unprotected .arc plaintext
- included in derived JSON/CSV/TXT/PDF exports as a field
- included in QR payloads
- validated by the BIP39 checksum
- recoverable from the mnemonic
- recoverable from the .arc file

If the original wallet used a BIP39 passphrase, recovering the same wallet requires the mnemonic and the exact passphrase. Decrypting a protected .arc file recovers the mnemonic only.

Mnemonic Normalization Before BIP39 PBKDF2

The mnemonic passed to bip39Seed is normalized as:

```
(mnemonic || "")
    .trim()
    .split(/\s+/)
    .filter(Boolean)
    .join(" ")
    .normalize("NFKD")
```

Consequences:

```
- Leading and trailing whitespace around the mnemonic is removed.
- Runs of whitespace between words collapse to one U+0020 SPACE.
- Empty tokens are removed.
- Unicode NFKD normalization is applied after whitespace normalization.
- Words are not case-folded by bip39Seed itself; the UI normalization and
  word-entry behavior should produce BIP39 English lowercase words.
```

The normalized mnemonic string is UTF-8 encoded and used as PBKDF2 password.

Passphrase Normalization Before BIP39 PBKDF2

The passphrase is normalized as:

```
p = (passphrase || "").normalize("NFKD")
```

No trimming is performed on the passphrase.

Consequences:

```
- "secret" and " secret" are different.
- "secret" and "secret " are different.
- "secret" and "Secret" are different.
- Tab, newline, and ordinary space are significant if entered into a field
  that permits them.
```

```
- Visually similar Unicode strings may normalize to the same NFKD byte string.
```

Recommendation:

```
Use ASCII-only passphrases unless there is a documented reason to use
non-ASCII text. ASCII is unaffected by NFKD and is easiest to reproduce across
keyboards and platforms.
```

Unicode NFKD Effects

NFKD is Unicode Normalization Form Compatibility Decomposition.

Examples of effects:

```
- Precomposed accented characters decompose into base character plus combining
  mark.
- Compatibility ligatures decompose into component letters.
- Full-width characters decompose to ordinary width equivalents.
- Superscript and similar compatibility characters may decompose to ordinary
  characters.
```

For ASCII strings:

```
NFKD(s) == s
```

For non-ASCII strings:

```
The visible characters and the actual PBKDF2 bytes may differ from what a
non-BIP39 system would use. BIP39-conformant implementations must apply NFKD.
```

Operational rule:

```
Record passphrases in a form that can be reproduced exactly under NFKD. If a
non-ASCII passphrase was used historically, test recovery before relying on a
new backup.
```

Cryptographic Effect of Passphrase Errors

A BIP39 passphrase error produces no checksum failure. It creates a different valid wallet tree.

One-byte differences produce unrelated seeds:

```
passphrase = "secret"
passphrase = "Secret"
passphrase = "secret "
passphrase = "secret\n"
```

All are distinct after NFKD unless normalization maps them to the same code point sequence.

Symptoms of a wrong passphrase:

- mnemonic checksum still passes
- root fingerprint differs from known value
- derived addresses do not match the original wallet
- balances appear absent when checked elsewhere
- private keys are valid but control the wrong addresses

There is no local mathematical test that says "wrong passphrase" unless a known root fingerprint or address is available for comparison.

Root Fingerprint Effect

The root fingerprint displayed by v1.6.0 is computed after BIP39 passphrase application.

Process:

```
seed = bip39Seed(mnemonic, passphrase)
root = masterFromSeed(seed)
root_pub = compressed secp256k1 public key of root.k
fingerprint = RIPEMD-160(SHA-256(root_pub))[0:4]
display = lowercase hex of those 4 bytes
```

Display field:

```
Root Fingerprint: <8 hex chars>
```

Changing the BIP39 passphrase changes the seed and therefore the fingerprint. Use a known root fingerprint as the fastest check that mnemonic and BIP39 passphrase are both correct.

Limit:

```
A matching root fingerprint does not verify derivation path, script type,
branch, or address index.
```

Validation and Analysis Fields

analyzeMnemonic returns details including:

```
word_count
wordlist_validity
entropy_bits
checksum_bits
checksum_match
bip39_compliance
bip39_seed_512_bit
master_private_key
master_chain_code
root_fingerprint
keystore_type
seed_format_detection
passphrase_warning
message
```

Supported word counts:

```
12
24
```

For other counts:

```
message = "Only 12-word and 24-word mnemonics are supported."
```

When checksum is valid and passphrase is empty:

```
passphrase_warning = "This seed may require a passphrase"
```

This warning is not evidence that a passphrase was used. It is a reminder that a valid mnemonic can have hidden passphrase-protected wallets.

Mnemonic Checksum Boundary

BIP39 checksum validation covers only the mnemonic entropy/checksum bits.

It detects:

- unsupported word count
- words not in BIP39 English word list
- checksum mismatch for many word errors

It does not detect:

- wrong BIP39 passphrase
- omitted BIP39 passphrase
- wrong derivation path
- wrong script type
- wrong coin selection
- use of another valid mnemonic
- transposition errors that happen to produce a valid checksum

Do not treat "Mnemonic is valid BIP39" as "wallet recovered".

ARC Password Mode

Password mode uses the password entered in the Encrypt / Export Seed dialog as the protected .arc credential.

Before ARC v2 PBKDF2:

```
arc_password_bytes = UTF-8(NFKD(dialog_password))
```

ARC v2 KDF:

```
master_key = PBKDF2-HMAC-SHA512(
    password   = arc_password_bytes,
    salt       = 32 random bytes from file,
    iterations = 1000000 for new exports,
    dkLen      = 64
)
```

Export requirements:

- password must be non-empty
- confirmation must match

Import requirements:

- same password string after NFKD normalization

The ARC password opens the protected .arc file. It does not reproduce the wallet tree unless the BIP39 passphrase is also known when one was used.

Raw Keyfile Mode

Raw keyfile mode uses random bytes rather than a typed password as the protected .arc credential source.

Generated keyfile bytes:

```
64 random bytes from crypto.getRandomValues
```

Generated raw keyfile object:

```
magic = "ARCULUS-KEYFILE"
format = "arculus-recovery-keyfile-v1"
key_b64 = base64(64 bytes)
```

Credential string:

```
arc_secret = "arc-keyfile-v1:" + base64(keyfile_bytes)
```

Validation:

```
imported keyfile bytes must be at least 32 bytes
```

The credential string is passed to ARC v2 key derivation as if it were the password string. Because it is ASCII prefix plus standard base64, NFKD does not change v1.6.0-generated keyfile secret strings.

Encrypted Keyfile Password

Encrypted keyfile password protects the raw keyfile bytes inside Arculus_Recovery_Keyfile.enc.key.

It is used by decryptArcKeyfileObject and encryptArcKeyfileBytes:

```
password_bytes = UTF-8(NFKD(password))

derived = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = 16 random bytes from encrypted keyfile,
    iterations = 200000 for generated keyfiles,
    dkLen      = 64
)

enc_key = derived[0:32]
mac_key = derived[32:64]
```

The encrypted keyfile password is not the BIP39 passphrase unless reused. It is not the ARC password unless reused through a workflow. It decrypts the keyfile; the decrypted keyfile bytes then participate in .arc credential construction.

Keyfile + Password Mode

Combined mode has two secrets:

- password
- keyfile bytes

First, keyfile bytes become:

```
keyfile_secret = "arc-keyfile-v1:" + base64(keyfile_bytes)
```

Then combinedArcSecret constructs:

```
material = UTF-8(
    "arc-combined-v1\n" +
    NFKD(password) +
    "\n" +
    keyfile_secret
)

digest = SHA-256(material)

arc_secret = "arc-combined-v1:" + base64(digest)
```

arc_secret is passed to ARC v2 as the protected .arc credential string.

Recovery requires both original factors. The password alone cannot reconstruct arc_secret. The keyfile alone cannot reconstruct arc_secret.

Unprotected Export

The advanced "Export without key or password" path creates:

```
format = "arculus-plain-seed-v1"
encrypted = false
mnemonic = normalized words
word_count = 12 or 24
```

It uses the same ARCULUS-ARC-V2 armor wrapper as protected seed exports, but the inner bundle is plaintext JSON after base64 decoding.

No password, keyfile, combined secret, or encrypted keyfile password protects this file.

The BIP39 passphrase is not stored in unprotected .arc either.

Password Reuse Boundaries

The UI cannot prevent a user from entering the same string for:

- BIP39 wallet passphrase
- ARC password
- encrypted keyfile password
- combined-mode password

However, the implementation treats them as separate inputs:

```
BIP39 passphrase:
    PBKDF2 salt suffix after "mnemonic", 2048 iterations.

ARC password:
    PBKDF2 password input, 1000000 iterations for current .arc exports.

encrypted keyfile password:
    PBKDF2 password input, 200000 iterations for current encrypted keyfiles.

combined-mode password:
    SHA-256 material input with keyfile secret, then ARC v2 PBKDF2 over the
    resulting ASCII combined secret string.
```

Recommendation:

```
Do not reuse the same string across these roles. Separate roles limit blast
radius when one backup component is exposed.
```

Whitespace Rules

Mnemonic:

- Leading/trailing whitespace is stripped.
- Internal whitespace runs collapse to one space.
- Empty tokens are removed.

BIP39 passphrase:

- Not trimmed.
- Whitespace is significant.
- Empty string is valid.

ARC password:

- Not trimmed by the cryptographic function.
- Whitespace is significant.
- Empty password rejected by UI for password mode.

Encrypted keyfile password:

- Not trimmed by the cryptographic function.
- Whitespace is significant.
- Empty password rejected for encrypted keyfile creation/decryption.

Combined-mode password:

- NFKD normalized before SHA-256 material construction.
- Whitespace is significant.
- Empty password rejected by UI.

Visibility and UI Controls

Main passphrase field:

- type="password"
- visibility toggle exists
- value participates in root fingerprint refresh and derivation

ARC credential dialog:

- password input and confirmation input for export
- password meter estimates length * log2(unique character count)
- password confirmation hidden during import
- mode tabs: password, keyfile, both

Generated/imported seed display:

```
- mnemonic is hidden with bullet masks
- Show Seed reveals while active
- generated/imported seeds remain available internally for validation, copy,
  export, and derivation
```

Clipboard:

```
Copy Seed copies normalized mnemonic words after confirmation and starts a
60-second best-effort clipboard-clear countdown.
```

Memory Handling

JavaScript strings cannot be reliably zeroized.

High-risk strings:

- mnemonic
- BIP39 passphrase
- ARC password
- encrypted keyfile password
- keyfile secret string
- combined secret string
- JSON plaintext strings

Some Uint8Array buffers are cleared with fill(0), especially around encrypted keyfile generation and decryption. This is best-effort cleanup only. It does not guarantee erasure from browser internals, garbage-collected copies, WebCrypto state, OS swap, hibernation files, crash dumps, or screenshots.

Recovery Requirements by Mode

Mnemonic-only recovery without BIP39 passphrase:

```
Need:
    mnemonic
```

Mnemonic with BIP39 passphrase:

```
Need:
    mnemonic
    exact BIP39 passphrase
```

Protected .arc password mode:

```
Need:
    protected .arc
    exact ARC password
    exact BIP39 passphrase if wallet used one
```

Protected .arc keyfile mode:

```
Need:
    protected .arc
    exact keyfile bytes
    exact BIP39 passphrase if wallet used one
```

Protected .arc combined mode:

```
Need:
    protected .arc
    exact keyfile bytes
    exact combined-mode password
    exact BIP39 passphrase if wallet used one
```

Encrypted keyfile:

```
Need:
    encrypted keyfile
    exact encrypted-keyfile password
```

Operational Checklist

Before relying on a passphrase-backed recovery:

```
- Verify mnemonic checksum.
- Enter the suspected BIP39 passphrase exactly.
- Compare root fingerprint if available.
- Compare known receive addresses.
- Check change branch for UTXO wallets.
- Try empty passphrase only as a separate candidate, not as a fallback after a
  failed non-empty passphrase.
```

Before relying on a protected .arc backup:

- Import-test it.
- Confirm root fingerprint after import.
- Confirm the BIP39 passphrase is separately backed up if used.
- Confirm required keyfile and password factors are separately stored.

Failure Modes

Wrong BIP39 passphrase:

```
No error. Valid wrong wallet.
```

Missing BIP39 passphrase:

```
No error. Empty-passphrase wallet.
```

Wrong ARC password:

```
MAC failure or decryption failure:
    "Unable to decrypt seed file. The password may be incorrect or the file may be
corrupted."
```

Wrong keyfile:

```
Same protected .arc failure, because reconstructed credential differs.
```

Wrong combined-mode password:

```
Same protected .arc failure, because combined secret differs.
```

Wrong encrypted-keyfile password:

```
"Unable to decrypt keyfile. The password may be incorrect or the keyfile may be corrupted.
"
```

Wrong derivation path with correct passphrase:

```
No error. Valid wrong address set.
```

Non-Negotiable Rules

- The BIP39 passphrase is never optional if it was used at wallet creation.
- The .arc file does not store the BIP39 passphrase.
- A valid BIP39 checksum does not validate the passphrase.
- A wrong passphrase can look perfectly normal.
- Store mnemonic, BIP39 passphrase, .arc credentials, and keyfiles separately.
- Test backups before relying on them.
- Avoid non-ASCII passphrases unless you have tested exact recovery.

Byte-Level Comparison Table

Mnemonic normalization:

```
Input field:
    mnemonic textarea and word grid

Transform:
    trim outer whitespace
    split on one or more whitespace characters
    remove empty tokens
    join with one ASCII space
    NFKD normalize

Consumed by:
    BIP39 checksum validation
    BIP39 seed derivation
    protected/unprotected .arc plaintext mnemonic field
```

```
Stored in .arc:
    yes, as normalized words in protected plaintext or unprotected bundle
```

BIP39 passphrase:

```
Input field:
    main Passphrase field

Transform:
    NFKD normalize only

Consumed by:
    BIP39 seed derivation
    root fingerprint calculation
    address/key derivation

Stored in .arc:
    no
```

ARC password:

```
Input field:
    Encrypt / Export Seed dialog password field

Transform:
    NFKD normalize before ARC v2 PBKDF2

Consumed by:
    protected .arc encryption/decryption

Stored in .arc:
    no
```

Raw keyfile bytes:

```
Input/source:
    generated keyfile or selected file

Transform:
    base64 encode into "arc-keyfile-v1:" credential string

Consumed by:
    protected .arc encryption/decryption in Keyfile mode

Stored in .arc:
    no
```

Combined password:

```
Input field:
    Encrypt / Export Seed dialog password field in Keyfile + Password mode

Transform:
    NFKD normalize inside combinedArcSecret material
    SHA-256 material
    base64 digest
    prefix with "arc-combined-v1:"

Consumed by:
    protected .arc encryption/decryption in combined mode

Stored in .arc:
    no
```

Passphrase Candidate Testing

If the operator is unsure whether a BIP39 passphrase was used, test candidates without exporting private keys until a known verification artifact matches.

Recommended order:

- Empty passphrase.
- Exact passphrase recorded in wallet documentation.
- Exact passphrase variants known to have been used by the operator.
- Stop and reassess before trying guesses not grounded in documentation.

For each candidate:

- enter the candidate in the Passphrase field
- validate mnemonic
- compare root fingerprint if known
- derive the smallest useful address range
- compare known receive address
- check UTXO change branch if applicable

Do not export a private-key file for every candidate. Candidate exports create many valid but wrong private-key artifacts and increase the chance of later operator error.

Known Address Verification

When no root fingerprint exists, known addresses are the practical verification source.

Strong match:

- same coin
- same address encoding family
- same derivation path
- same branch/index
- same known historical address

Weak match:

- same first few characters
- same address prefix only
- plausible address type only

Only a full address match is useful. Prefix similarity is not evidence of a correct passphrase.

For Bitcoin-like wallets:

- Compare receive branch first if you have deposit addresses.
- Compare change branch if you have transaction change outputs.
- If no match, vary path and script type before assuming funds are absent.

For Ethereum-like wallets:

- Compare the full 20-byte address.
- Remember ERC-20 tokens share the same address.

For XRP/Stellar:

- Address match does not validate exchange destination tag or memo.

Human Factors and Transcription

Passphrases fail operationally more often than cryptographic systems fail.

Dangerous ambiguities:

- uppercase vs lowercase
- trailing spaces
- multiple internal spaces
- hyphen vs dash in old notes
- apostrophe vs quote in old notes

- zero vs letter O
- one vs letter l or uppercase I
- non-breaking space copied from rich text
- mobile keyboard smart punctuation
- non-ASCII lookalike characters

Mitigations:

```
- Prefer generated ASCII passphrases.
- Record passphrase in a plain-text physical representation.
- Avoid smart quotes and typographic punctuation.
- Store a root fingerprint or first receive address alongside non-secret
  metadata so future recovery can verify correctness.
```

Backup Separation Examples

Example A: mnemonic plus BIP39 passphrase:

```
Storage 1:
    mnemonic words

Storage 2:
    BIP39 passphrase

Storage 3:
    non-secret recovery note containing coin/path/root fingerprint
```

Example B: protected .arc password mode:

```
Storage 1:
    Arculus_Encrypted_Seed.arc

Storage 2:
    ARC password

Storage 3:
    BIP39 passphrase if used
```

Example C: protected .arc keyfile mode:

```
Storage 1:
    Arculus_Encrypted_Seed.arc

Storage 2:
    Arculus_Recovery_Keyfile.key

Storage 3:
    BIP39 passphrase if used
```

Example D: protected .arc combined mode:

```
Storage 1:
    Arculus_Encrypted_Seed.arc

Storage 2:
    Arculus_Recovery_Keyfile.enc.key or raw keyfile

Storage 3:
    combined-mode password

Storage 4:
    BIP39 passphrase if used
```

Compatibility Notes

BIP39 passphrase behavior should match any BIP39-conformant wallet:

- NFKD mnemonic
- NFKD passphrase
- salt is "mnemonic" + passphrase
- PBKDF2-HMAC-SHA512

- 2048 iterations
- 64-byte output

If another wallet does not reproduce the same result:

- verify that the wallet is using BIP39, not another mnemonic scheme
- verify passphrase support was enabled
- verify Unicode entry behavior
- verify derivation path
- verify script/address type
- verify account index

The passphrase algorithm is standard. Most mismatches come from path, script type, account index, or transcription errors.

Incident Response

If the BIP39 passphrase was exposed with the mnemonic:

```
Treat the wallet as compromised. Move funds to a new seed.
```

If the BIP39 passphrase was exposed without the mnemonic:

```
The passphrase alone does not recover funds. Still consider rotating if the
mnemonic may also be exposed or if the passphrase was reused elsewhere.
```

If the ARC password was exposed with the protected .arc:

```
Treat the mnemonic as exposed. Move funds or re-secure depending on whether
the BIP39 passphrase remains secret and whether the attacker can derive it.
```

If a raw keyfile was exposed with its protected .arc:

```
Treat the mnemonic as exposed for Keyfile mode.
```

If one factor of combined mode was exposed:

```
The .arc remains protected against that exposure alone, but re-export with new
factors if long-term custody is affected.
```

Implementation Checklist

A compatible passphrase implementation must:

- Support only 12- and 24-word mnemonics if matching v1.6.0 UI behavior.
- Normalize mnemonic whitespace exactly before BIP39 seed derivation.
- Apply NFKD to mnemonic and BIP39 passphrase.
- Use UTF-8 bytes.
- Use salt UTF-8("mnemonic" + passphrase).
- Use PBKDF2-HMAC-SHA512 with 2048 iterations and 64-byte output.
- Compute root fingerprint from BIP32 master public key hash160 first 4 bytes.
- Keep BIP39 passphrase separate from .arc credential inputs.
- Never infer passphrase correctness from mnemonic checksum alone.

Arculus Recovery v1.6.0 - Byte-Level File Format Specification

Scope

This document specifies every file format produced or consumed by Arculus_Recovery.html v1.6.0. It covers armored seed bundles, protected seed bundles, unprotected seed bundles, raw keyfiles, encrypted keyfiles, derived JSON, CSV, TXT, PDF, and QR PNG exports.

This document specifies structure and serialization. Encryption.txt specifies the cryptographic construction inside protected .arc and encrypted keyfile objects. Derivation.txt specifies the meaning of key and address fields inside derived-output files. QR.txt specifies the QR payload encoder in deeper detail.

All non-PDF formats emitted by v1.6.0 are UTF-8 text unless explicitly stated otherwise. The HTML implementation writes no byte-order mark. Plain text exports use LF line endings. Importers should tolerate CRLF in user-edited text, but v1.6.0 producers write LF.

Format Inventory

v1.6.0 writes these file types:

```
Arculus_Encrypted_Seed.arc
    ARCULUS-ARC-V2 armored seed bundle. The internal bundle may be protected
    or unprotected depending on credential selection.

Arculus_Recovery_Keyfile.key
    JSON raw keyfile containing 64 random key bytes in base64.

Arculus_Recovery_Keyfile.enc.key
    JSON encrypted keyfile containing an encrypted 64-byte keyfile payload.

Arculus_Derived_Keys_Addresses.json
    Pretty-printed derived-output object.

Arculus_Derived_Keys_Addresses.csv
    Flattened row export.

Arculus_Derived_Keys_Addresses.txt
    Deterministic human-readable text export.

Arculus_Key_Export_<display coin>.pdf
    PDF key export generated by embedded jsPDF.

qr_<sanitized-address>.png
    QR code PNG generated from the QR canvas.
```

v1.6.0 reads these file families:

- ARCULUS-ARC-V2 armored protected seed bundles
- ARCULUS-ARC-V2 armored unprotected seed bundles
- raw JSON current protected seed bundles
- raw JSON current unprotected seed bundles
- legacy PBKDF2-SHA256 stream seed bundles
- legacy AES-GCM seed bundles
- current raw keyfile JSON
- current encrypted keyfile JSON
- base64-only raw key material
- legacy KEYFILE and KEYFILE-ENC JSON

Common Encodings

UTF-8:

```
All text fields are UTF-8 strings. File text is decoded by browser File.text
during import. TextEncoder and TextDecoder are used for cryptographic byte
conversion.
```

JSON:

```
Seed bundles:
    JSON.stringify(bundle)

Derived JSON export:
    JSON.stringify(data, null, 2) + "\n"

Keyfiles:
    JSON.stringify(obj, null, 2) + "\n"
```

Base64:

```
Alphabet:
    A-Z a-z 0-9 + /

Padding:
    "=" padding as produced by btoa.

Line wrapping:
    None in v1.6.0-produced fields.
```

Hex:

```
Lowercase hex, two characters per byte, no separators and no "0x" prefix.
```

Dates:

```
JavaScript Date.toISOString strings.

Example:
    2026-06-14T18:22:13.456Z
```

Newlines:

```
Text producers write LF, byte 0x0a.
```

Binary export:

```
PDF and PNG exports are Blob objects. In browser mode they are downloaded by
object URL. In Tauri mode they are base64-encoded and passed to save_export.
```

Save Path Abstraction

All file saves route through saveBlob(blob, filename).

Browser path:

```
downloadBlobInBrowser(blob, filename):
    a = document.createElement("a")
    a.href = URL.createObjectURL(blob)
    a.download = filename
    a.click()
    URL.revokeObjectURL(a.href)
```

Tauri path:

```
getTauriSaveExport detects one of:

  window.arculusTauriSaveExport(filename, contentBase64)
  window.__TAURI__.core.invoke("save_export", { filename, contentBase64 })
  window.__TAURI__.invoke("save_export", { filename, contentBase64 })
  window.__TAURI_INTERNALS__.invoke("save_export", { filename, contentBase64 })

When Tauri save is available:

  bytes = new Uint8Array(await blob.arrayBuffer())
  contentBase64 = bytesToBase64(bytes)
  savedPath = await save_export(filename, contentBase64)

If native save fails, the UI reports the error and does not automatically
fall back to browser download in that code path.
```

.arc Armor

Every v1.6.0 seed export, protected or unprotected, is serialized by serializeSeedBundle(bundle):

```
json = JSON.stringify(bundle)
json_bytes = UTF-8(json)
body = base64(json_bytes)
```

```
file_text = "ARCULUS-ARC-V2" + "\n" + body + "\n"
```

Byte layout:

```
byte 0..13:       ASCII "ARCULUS-ARC-V2"
byte 14:          LF, 0x0a
byte 15..N-2:     base64(UTF-8(JSON.stringify(bundle)))
byte N-1:         LF, 0x0a
```

The armor does not distinguish protected from unprotected seed bundles. The decoded JSON bundle determines protection type.

Armor parser:

```
trimmed = text.trim()
if trimmed.startsWith("ARCULUS-ARC-V2"):
    encoded = trimmed.split(/\r?\n/).slice(1).join("").trim()
    bundle = JSON.parse(bytesToText(base64ToBytes(encoded)))
else:
    bundle = JSON.parse(text)
```

Parser consequences:

- Leading and trailing whitespace outside the file body is ignored for armor.
- CRLF and LF are accepted between armor lines.
- Wrapped armor body lines are joined without separators.
- The trailing LF is optional for import.
- Files without the armor header are parsed directly as JSON.
- The filename and extension are not used for dispatch.

Protected .arc Bundle

Current protected seed export is produced by encrypt_v2 and then optionally decorated with credential_mode.

Internal JSON object:

```
{
  "magic": "ARCULUS-ARC",
  "format": "arculus-encrypted-seed-v2",
  "version": 2,
  "created_at": "<Date.toISOString>",
  "kdf": {
    "name": "PBKDF2",
    "hash": "SHA-512",
    "iterations": 1000000,
    "salt_b64": "<base64 32 bytes>"
  },
  "cipher": {
    "name": "HMAC-SHA512-CTR",
    "nonce_b64": "<base64 24 bytes>"
  },
  "ciphertext_b64": "<base64 N bytes>",
  "mac_b64": "<base64 64 bytes>",
  "credential_mode": "password" | "keyfile" | "both"
}
```

Required cryptographic fields:

```
magic:
    Fixed "ARCULUS-ARC".

format:
    Fixed "arculus-encrypted-seed-v2".

version:
    Integer 2.

created_at:
    Date.toISOString string. This field is in the ARC v2 MAC transcript.

kdf.name:
    Fixed "PBKDF2".
```

```
kdf.hash:
    Fixed "SHA-512".

kdf.iterations:
    Integer. v1.6.0 export writes 1000000. Import requires >= 600000.

kdf.salt_b64:
    Base64 of exactly 32 bytes.

cipher.name:
    Fixed "HMAC-SHA512-CTR".

cipher.nonce_b64:
    Base64 of exactly 24 bytes.

ciphertext_b64:
    Base64 of encrypted plaintext JSON. Length equals plaintext UTF-8 byte
    length. There is no fixed minimum or maximum at the format layer.

mac_b64:
    Base64 of exactly 64 bytes.
```

credential_mode:

```
Added by handleEncryptExportSeed after encryptSeedBundle returns. The value
is a UI hint and is not included in the v1.6.0 MAC transcript when produced by
v1.6.0. Import also accepts credentialMode and protection as aliases.
```

Protected .arc Plaintext

The encrypted plaintext JSON is:

```
{
  "mnemonic": "<normalized words joined by U+0020 SPACE>",
  "word_count": 12 or 24,
  "created_at": "<same Date.toISOString string as outer bundle>"
}
```

Serialization:

```
plaintext = UTF-8(JSON.stringify(payload))
```

No indentation, no trailing LF, and no BOM are included. The encrypted payload contains only the mnemonic, word_count, and created_at. It does not contain the BIP39 wallet passphrase, derivation path, coin, addresses, or derived private keys.

Unprotected .arc Bundle

Unprotected seed export is produced by makePlainSeedBundle and serialized through the same ARCULUS-ARC-V2 armor as protected exports.

Internal JSON object:

```
{
  "magic": "ARCULUS-ARC",
  "format": "arculus-plain-seed-v1",
  "version": 1,
  "created_at": "<Date.toISOString>",
  "mnemonic": "<normalized 12 or 24 words>",
  "word_count": 12 or 24,
  "encrypted": false
}
```

Security:

```
This is not encrypted and not authenticated. Anyone who base64-decodes the
armor body can read the mnemonic.
```

Import dispatch:

```
bundle.magic == "ARCULUS-ARC"
bundle.format == "arculus-plain-seed-v1"
```

The password parameter is ignored for this path.

.arc Import Dispatch

Seed import dispatch is implemented by parseSeedBundle and decrypt:

- Parse armor or raw JSON into bundle.

```
2. If:
     bundle.magic == "ARCULUS-ARC"
     and bundle.format == "arculus-plain-seed-v1"
   then:
     return normalizeDecryptedMnemonic(bundle)

3. If:
     bundle.magic == "ARCULUS-ARC"
     or bundle.version == 2
   then:
     decryptV2(bundle, credential_secret)

4. Otherwise:
     decryptV1(bundle, credential_secret)
```

decryptV1 handles:

```
- format "arculus-encrypted-seed-v2" in the legacy PBKDF2-SHA256 stream
  branch when it is not routed to current decryptV2 by magic/version.
- format "arculus-encrypted-seed-python-v1"
- format "arculus-encrypted-seed-v1" AES-GCM legacy branch.
```

New v1.6.0 exports always use ARCULUS-ARC-V2 armor.

Raw Keyfile Format

Raw keyfiles are produced by makeArcKeyfile(password = "") when password is empty.

Generated JSON:

```
{
  "magic": "ARCULUS-KEYFILE",
  "format": "arculus-recovery-keyfile-v1",
  "version": 1,
  "created_at": "<Date.toISOString>",
  "key_b64": "<base64 64 bytes>"
}
```

Serialization:

```
JSON.stringify(obj, null, 2) + "\n"
```

Default filename:

```
Arculus_Recovery_Keyfile.key
```

Generated key length:

```
64 bytes.
```

Import accepts:

```
- Current raw keyfile JSON with magic "ARCULUS-KEYFILE", format
  "arculus-recovery-keyfile-v1", and key_b64.
- Base64-only text. Whitespace is stripped before base64 decode.
- Legacy object with magic "KEYFILE" and field key.
```

Decoded key material must be at least 32 bytes. v1.6.0-generated raw keyfiles are exactly 64 bytes.

Encrypted Keyfile Format

Encrypted keyfiles are produced by makeArcKeyfile(password) when password is non-empty.

Generated JSON:

```
{
  "magic": "ARCULUS-KEYFILE-ENC",
  "format": "arculus-recovery-keyfile-enc-v1",
  "version": 1,
  "created_at": "<Date.toISOString>",
  "kdf": {
    "algo": "PBKDF2-SHA512",
    "iterations": 200000,
    "salt_b64": "<base64 16 bytes>"
  },
  "cipher": {
```

```
    "name": "HMAC-SHA512-CTR",
    "nonce_b64": "<base64 16 bytes>"
  },
  "ciphertext_b64": "<base64 64 bytes>",
  "mac_b64": "<base64 64 bytes>"
}
```

Serialization:

```
JSON.stringify(obj, null, 2) + "\n"
```

Default filename:

```
Arculus_Recovery_Keyfile.enc.key
```

Generated field lengths:

```
kdf.salt_b64:
    base64 of 16 bytes.

cipher.nonce_b64:
    base64 of 16 bytes.

ciphertext_b64:
    base64 of encrypted keyfile bytes. For v1.6.0-generated keyfiles this is
    64 bytes before base64.

mac_b64:
    base64 of 64 bytes.
```

Import accepts aliases:

```
salt:
    obj.kdf.salt_b64 or obj.kdf.s

iterations:
    obj.kdf.iterations or obj.kdf.i or default 200000

nonce:
    obj.cipher.nonce_b64 or obj.n

ciphertext:
    obj.ciphertext_b64 or obj.ct

MAC:
    obj.mac_b64 or obj.mac
```

Import accepts current encrypted keyfile objects and legacy magic "KEYFILE-ENC".

Keyfile Secret String

Keyfile bytes are not used directly as the .arc password. They are converted to a secret string:

```
"arc-keyfile-v1:" + base64(keyfile_bytes)
```

Combined Keyfile + Password mode then hashes:

```
material = UTF-8(
    "arc-combined-v1\n" +
    NFKD(password) +
    "\n" +
    keyfile_secret
)
```

and produces:

```
"arc-combined-v1:" + base64(SHA-256(material))
```

These strings are credential inputs to the ARC v2 seed-file key schedule. They are not file formats by themselves, but implementations must reproduce them exactly to import files protected by keyfile modes.

Derived JSON Export

Function:

```
exportPayload(data, "json")
```

Serialization:

```
JSON.stringify(data, null, 2) + "\n"
```

MIME type:

```
application/json;charset=utf-8
```

Default filename:

```
Arculus_Derived_Keys_Addresses.json
```

Top-level object:

```
{
  "coin": "<internal coin id>",
  "network": "mainnet",
  "word_count": 12 or 24,
  "accounts": [ ... ]
}
```

Supported coin ids:

```
bitcoin
bitcoincash
litecoin
dogecoin
ethereum
solana
stellar
monero
cardano
xrp
```

Python 1.6.0 producer note:

```
The Python CLI writes the same top-level JSON object shape for all supported
v1.6.0 command-line coin ids. It serializes with json.dumps(data, indent=2)
followed by one LF byte. Python CSV is produced from the flattened row list
and Python TXT is produced from the same object graph. The Python CLI does not
write browser localStorage state, DOM state, hidden-seed UI state, or PDF
modal state into JSON/CSV/TXT exports.

Python --coin accepts exactly:

  bitcoin
  bitcoincash
  litecoin
  dogecoin
  ethereum
  solana
  stellar
  monero
  cardano
  xrp

Python --script-type defaults to auto. For Bitcoin Cash this resolves to
P2PKH/CashAddr; unsupported script families must fail rather than emit an
address. For Solana, Stellar, Monero, and Cardano, script type does not select
a UTXO script and the coin-specific derivation family controls output fields.
```

For secp256k1/UTXO families, account objects can include:

```
derivation
account_script_type_used
root_xprv
root_xpub
root_yprv
root_ypub
root_zprv
root_zpub
root_trprv
root_trpub
account_xprv
account_xpub
account_yprv
account_ypub
account_zprv
account_zpub
account_trprv
account_trpub
receiving
```

```
change
```

For account-based secp256k1 families, account objects can include:

```
derivation
account_script_type_used
root extended keys
account extended keys
receiving
change
```

Common row fields:

```
branch
path
address
public_key_hex
private_key_hex
```

UTXO row additions:

```
private_key_wif
```

Taproot row additions:

```
taproot_internal_public_key_hex
taproot_internal_private_key_hex
taproot_internal_private_key_wif
taproot_tweak_hex
taproot_output_public_key_hex
taproot_output_private_key_hex
taproot_output_private_key_wif
taproot_output_parity
```

Ethereum row additions:

```
ethereum_public_key_hex
erc20_address
erc20_note
```

XRP row additions:

```
xrp_classic_address
xrp_note
```

Solana/Stellar row additions:

```
private_key_hex
public_key_hex
coin-specific note fields
```

Monero row additions:

```
private_spend_key_hex
private_view_key_hex
public_spend_key_hex
public_view_key_hex
monero_note
```

Cardano row additions:

```
private_key_hex
stake_private_key_hex
public_key_hex
stake_public_key_hex
cardano_note
```

The derived JSON schema is intentionally sparse. Fields appear only when the selected coin and script type produce them.

CSV Export

Function:

```
derivedToCsv(data)
```

MIME type:

```
text/csv;charset=utf-8
```

Default filename:

```
Arculus_Derived_Keys_Addresses.csv
```

Row source:

```
flattenDerivedRows(data)
```

Flattening algorithm:

```
rows = []
for each account in data.accounts:
    EXCLUDE_FROM_TABLE = Set(
        AK_KEY_ORDER +
        [
          "network",
          "word_count",
          "derivation",
          "account_script_type_used",
          "coin"
        ]
    )

    accountFields = {}
    for each [key, value] in Object.entries(account):
        if key is not "receiving"
           and key is not "change"
           and key is not in EXCLUDE_FROM_TABLE:
            accountFields[key] = value

    for branch in ["receiving", "change"]:
        for each item in account[branch] or []:
            rows.push({
                branch,
                ...accountFields,
                ...item
            })
```

Header order:

```
The first-seen key order across rows, implemented as:

  Array.from(rows.reduce((set, row) => {
    Object.keys(row).forEach(key => set.add(key))
    return set
  }, new Set()))
```

Cell serialization:

```
text = value === null || value === undefined ? "" : String(value)

if text contains DQUOTE, comma, CR, or LF:
    output = DQUOTE + text.replace(/"/g, '""') + DQUOTE
else:
    output = text
```

Address cells:

```
For header "address", displayAddress(row.address, data.coin) is applied
before CSV escaping. This can strip URI prefixes for display for some coin
types.
```

Line endings:

```
Rows are joined with LF. The file ends with LF.
```

Empty output:

```
If flattenDerivedRows returns no rows, derivedToCsv returns an empty string.
```

TXT Export

Function:

```
derivedToText(data)
```

MIME type:

```
text/plain;charset=utf-8
```

Default filename:

```
Arculus_Derived_Keys_Addresses.txt
```

Serialization:

```
Header:

  Arculus Derived Keys and Addresses

  coin: <data.coin or empty>
  network: <data.network or empty>
  word count: <data.word_count or empty>

For each account:

  blank line
  Account <1-based account index>

Account scalar fields:

  appendTextFields(account, ["receiving", "change"])

For each branch receiving/change:

  blank line
  Receiving Addresses
  or
  Change Addresses

  If branch has no items:
      No addresses derived.

  Else for each item:
      blank line
      <branch> #<zero-based item index>
      field lines
```

Field label conversion:

```
key.replace(/_/g, " ")
```

Address field display:

```
displayAddress(value, coin) is applied for key "address".
```

Line ending:

```
LF.
```

Final byte:

```
LF.
```

PDF Export

Function:

```
exportDump()
```

Requirement:

```
window.jspdf.jsPDF must be available inside the HTML file.
```

MIME type:

```
application/pdf
```

Default filename:

```
"Arculus_Key_Export_" + (displayCoin || "export") + ".pdf"
```

Document creation:

```
new jsPDF({
    orientation: "landscape",
    unit: "pt",
    format: "a4"
})
```

Page model:

```
page width and height from jsPDF internal page size
margin: 36 pt
usable width: page width - 72 pt
```

Fonts and sizes:

```
title size: 13
header size: 8
cell size: 7.5
row height: 14
header height: 18
column gap: 6
extended-key label width: 90
extended-key line height: 13
extended-key font size: 7.5
extended-key label size: 7
```

Title block:

```
Text:
    Arculus Key Export

Meta fields:
    display coin
    "Arculus v" + APP_VERSION
    export timestamp as YYYY-MM-DD HH:MM:SS UTC
    optional "Fingerprint: <root fingerprint>"

Page label:
    Page <page number>
```

Extended keys:

```
The first account is inspected. Keys from AK_KEY_ORDER that exist on the
account are rendered under "EXTENDED KEYS" with labels from AK_LABELS. Values
are split to fit the usable page width and rendered in courier.
```

Address table columns:

```
Branch:
    fixed width 52 pt

Path:
    fixed width 110 pt

Address:
    flexible width

Private key column:
    Monero:
        key "private_spend_key_hex"
        label "Private Spend Key Hex"

    Ethereum, Solana, Stellar, Cardano, XRP:
        key "private_key_hex"
        label "Private Key Hex"

    Other coins:
        key "private_key_wif"
        label "Private Key WIF"
```

Rows:

```
Rows come from flattenDerivedRows(lastDerivedOutput). Text is clipped with an
ellipsis if it exceeds the column width. Branch text is color-coded. Pages are
added when the next row would exceed the bottom margin.
```

The PDF is not encrypted by jsPDF. It is a plaintext secret-bearing artifact.

QR PNG Export

QR output is generated by the QR modal. The QR encoder renders to a canvas, and Save PNG serializes that canvas.

Save procedure:

```
canvas.toBlob(resolve, "image/png")
if blob exists:
    saveBlob(blob, "qr_" + safeName + ".png")
```

safeName:

```
safeName = address.replace(/[^a-zA-Z0-9]/g, "_").slice(0, 40)
if safeName == "":
    safeName = "address"
```

Filename:

```
qr_<safeName>.png
```

PNG dimensions:

```
Non-XRP renderer requests 256 px.
XRP renderer requests 320 px.
```

The actual canvas dimension is determined by QR.txt:

```
module_px = floor(requested_size / (matrix_size + quiet_zone * 2))
actual_px = module_px * (matrix_size + quiet_zone * 2)
```

QR payloads:

```
Payload URI construction is specified in QR.txt. FileFormat only specifies
the PNG file serialization and filename.
```

QR Address Copy

The QR modal Copy Address button copies only the trimmed address input, not the rendered QR URI label. For XRP, this means it copies the r-address without the "?dt=" destination-tag component shown in the QR label.

Copy path:

```
copyTextToClipboard(address)
```

Security:

```
Address copying is not secret in the same sense as a private key, but it can
leak wallet association and payment intent.
```

Display Address Transformation

Some export paths call displayAddress(value, coin) before serializing an address for human display. This affects CSV, TXT, and PDF table output for the address column. JSON export preserves the original derived object values.

Consumers that need byte-for-byte derived payloads should prefer JSON. CSV, TXT, and PDF are display exports.

Filename Summary

v1.6.0 default filenames:

```
Arculus_Encrypted_Seed.arc
Arculus_Recovery_Keyfile.key
Arculus_Recovery_Keyfile.enc.key
Arculus_Derived_Keys_Addresses.json
Arculus_Derived_Keys_Addresses.csv
Arculus_Derived_Keys_Addresses.txt
Arculus_Key_Export_<display coin>.pdf
qr_<sanitized-address>.png
```

Filenames are not authenticated and are not used to identify file type during import. They are operator-facing defaults only.

Security Classification by Format

Protected secret-bearing:

- Protected ARCULUS-ARC-V2 .arc file
- Encrypted keyfile JSON

Plaintext secret-bearing:

- Unprotected .arc file
- Raw keyfile JSON

- Base64-only keyfile material
- Derived JSON export
- Derived CSV export
- Derived TXT export
- Derived PDF export

Not inherently private-key-bearing but privacy-sensitive:

- QR PNG exports
- Copied address strings

The fact that a format is text, JSON, CSV, PDF, or PNG says nothing about its sensitivity. Classification depends on fields and payloads.

Parser Robustness Requirements

A compatible importer should:

```
- Accept LF and CRLF in the ARC armor.
- Accept a missing final LF in ARC armor.
- Reject invalid base64 for required binary fields.
- Reject protected ARC v2 salt lengths other than 32 bytes.
- Reject protected ARC v2 nonce lengths other than 24 bytes.
- Reject protected ARC v2 MAC lengths other than 64 bytes.
- Reject current protected .arc files with KDF iterations below 600000.
- Ignore unknown non-security fields in JSON objects where the implementation
  already ignores them.
- Preserve JSON field values as strings exactly where those fields feed a MAC
  transcript.
- Treat credential_mode only as an unauthenticated UI hint.
```

A compatible producer should:

- Write UTF-8 without BOM.
- Write LF line endings.
- Use standard base64 with padding.
- Use JSON.stringify-compatible JSON.
- Use the exact current magic and format strings.
- Use the exact filenames only as defaults, not as security boundaries.

Detailed Derived Object Shapes

The derived JSON object is the only export intended to preserve the full machine-readable result. CSV, TXT, and PDF are projections of the same object.

Common top-level fields:

```
coin:
    Internal lower-case coin id selected by deriveOutput.

network:
    "mainnet" in v1.6.0.

word_count:
    12 or 24, copied from normalized mnemonic word count.

accounts:
    Array. v1.6.0 writes one account object for the selected derivation
    operation.
```

Common account fields for secp256k1 families:

```
derivation:
    Normalized account derivation path.

account_script_type_used:
```

```
    Effective script type after auto script resolution.

receiving:
    Array of branch-0 rows for Bitcoin-like branch derivation, or account-like
    rows for account families when the implementation maps output there.

change:
    Array of branch-1 rows. For several non-UTXO families this array is empty
    or unused.
```

Extended-key display order:

```
The account-key pane and PDF extended-key section use AK_KEY_ORDER, not raw
Object.keys order. File consumers should not assume all possible extended key
fields are present.
```

Bitcoin-like row minimum:

```
{
  "path": "<full child path>",
  "address": "<encoded address>",
  "public_key_hex": "<compressed secp256k1 public key>",
  "private_key_hex": "<32-byte private key hex>",
  "private_key_wif": "<WIF string>"
}
```

Ethereum row minimum:

```
{
  "path": "<full child path>",
  "address": "0x...",
  "public_key_hex": "<compressed secp256k1 public key>",
  "ethereum_public_key_hex": "<uncompressed x||y public key hex>",
  "private_key_hex": "<32-byte private key hex>",
  "erc20_address": "0x...",
  "erc20_note": "<note string>"
}
```

XRP row minimum:

```
{
  "path": "<full child path>",
  "address": "<classic r-address>",
  "xrp_classic_address": "<classic r-address>",
  "public_key_hex": "<compressed secp256k1 public key>",
  "private_key_hex": "<32-byte private key hex>",
  "xrp_note": "<destination-tag note>"
}
```

Solana/Stellar/Monero/Cardano:

```
These families use Ed25519 or Ed25519-derived schemes. Their exact private
key, public key, chain-code, address, and note fields are specified in
Derivation.txt. FileFormat requires only that JSON preserve the produced
object exactly and that flattened exports treat the fields as strings.
```

CSV Lossiness and Ordering

CSV is not a canonical backup format. It is a flattened convenience export.

Loss and transformation points:

```
- Nested account structure is removed.
- Receiving and change rows are combined into one table.
- Extended-key fields in AK_KEY_ORDER are intentionally excluded from row
  flattening.
- network, word_count, derivation, account_script_type_used, and coin are
  excluded from repeated account fields.
- Address cells pass through displayAddress.
- Values are converted with String(value), so type information is not
  preserved.
- Header order depends on first appearance while iterating JavaScript object
  keys.
```

CSV is appropriate for spreadsheet review of derived addresses and private keys. It is not appropriate for lossless re-import into Arculus.

TXT Lossiness and Ordering

TXT export is deterministic but human-oriented.

Loss and transformation points:

```
- Underscores in field names are replaced by spaces.
- Object nesting is represented as headings and paragraphs.
- Values are rendered with String(value).
- Address values pass through displayAddress.
- Field order follows Object.entries order of the JavaScript object at export
  time.
```

TXT is appropriate for printing or manual review. JSON is preferred for any tooling or automated audit.

PDF Lossiness and Ordering

PDF export is a visual report, not a complete machine-readable file.

Loss and transformation points:

```
- Address table cells are clipped to fit column width.
- Only Branch, Path, Address, and one private-key column are rendered in the
  main table.
- The private-key column changes by coin.
- Extended keys are printed only from the first account.
- Some row fields present in JSON do not appear in the PDF table.
- The root fingerprint is read from the UI display, not recomputed inside the
  PDF exporter.
```

PDF should be treated as plaintext key material when it contains private keys, but it should not be treated as a complete archival representation of the derived JSON object.

Import File Input Surfaces

Seed import file input:

```
Element accepts:

  .arc
  .json
  application/json

The accept attribute is a browser chooser hint. It is not a parser security
boundary. parseSeedBundle decides actual support from file contents.
```

Keyfile inputs:

```
Raw and encrypted keyfiles are read with File.text and then parsed as JSON or
base64-only text. The filename extension is not normative.
```

QR PNG import:

```
v1.6.0 does not import QR PNG files.
```

Derived-output import:

```
v1.6.0 does not import derived JSON/CSV/TXT/PDF exports back into the app as
trusted key material.
```

Artifact Equivalence

The following artifacts are operationally equivalent to the ability to recover or spend funds:

```
- mnemonic + BIP39 passphrase if one was used
- unprotected .arc file + BIP39 passphrase if one was used
- protected .arc file + correct credential + BIP39 passphrase if one was used
- protected .arc file + raw keyfile in Keyfile mode + BIP39 passphrase if one
  was used
- protected .arc file + keyfile + combined password in combined mode + BIP39
  passphrase if one was used
- account xprv for all descendants under that account
- single private key for the corresponding address
- Monero private spend key for spend authority
```

File format readers and operators must classify artifacts by contents, not by extension or MIME type.

Version-Specific Notes

v1.6.0 format facts:

- APP_VERSION is "1.6.0".
- Current protected seed format is arculus-encrypted-seed-v2.
- Current plain seed format is arculus-plain-seed-v1.
- The outer seed armor is ARCULUS-ARC-V2 for both protected and plain seed bundles.
- credential_mode is appended after encryption and is therefore a UI hint, not a MAC-bound field in v1.6.0-produced files.
- New protected seed exports use PBKDF2-HMAC-SHA512 with 1000000 iterations.
- Current protected seed imports reject iteration counts below 600000.
- Encrypted keyfiles use PBKDF2-HMAC-SHA512 with 200000 iterations.
- Derived-output JSON/CSV/TXT/PDF files are plaintext.
- QR PNG files are address-payload images, not seed backups.

Arculus Recovery v1.6.0 - Encryption Subsystem Internals

Scope

This document is the byte-level specification of the seed-file and keyfile encryption subsystem implemented by Arculus_Recovery.html v1.6.0. It specifies the protected .arc envelope, the unprotected .arc envelope, raw keyfiles, encrypted keyfiles, the Keyfile + Password combined-secret mode, legacy import formats, key derivation, stream generation, authentication transcripts, import validation, and failure behavior.

The encryption subsystem protects mnemonic storage at rest. It does not specify address derivation, BIP32 trees, Ed25519 trees, QR payloads, derived-output exports, or wallet passphrase semantics except where those concepts touch seed export/import. Those topics are specified in the other files in this directory.

The current v1.6.0 implementation is the single-file browser build:

```
Arculus_Recovery.html
```

Normative references to function names in this document refer to that file.

Design Constraints

The encryption subsystem is built under these constraints:

```
1. Browser-native cryptography only.
   The canonical v1.6.0 HTML build uses WebCrypto primitives exposed through
   crypto.subtle and random bytes from crypto.getRandomValues. It does not
   require an external cryptographic package or network access.

2. File formats must be inspectable and recoverable offline.
   Protected files are text files. The outer armor is an ASCII header plus a
   standard base64 body. Internal binary fields are standard base64 strings.
   A future offline recovery implementation can parse them using only UTF-8,
   JSON, base64, PBKDF2, HMAC, SHA-256, SHA-512, AES-GCM for legacy v1, and
   byte-array operations.

3. New protected seed exports use encrypt-then-MAC.
   v1.6.0 protected .arc exports derive independent encryption and MAC keys
   from PBKDF2-HMAC-SHA512, encrypt with an HMAC-SHA512 counter-mode stream,
   and authenticate all cryptographic metadata before decryption.

4. The current format must reject downgrade attempts.
   The v2 MAC commits to the KDF algorithm, KDF hash, iteration count, cipher
   name, salt, nonce, ciphertext, magic string, format string, version, and
   creation timestamp. v2 imports reject KDF iteration counts below 600000.

5. Credential modes are operational wrappers around the same .arc envelope.
   Password, Keyfile, and Keyfile + Password all ultimately produce a secret
   string. That string is passed to the same ARC v2 seed-file key schedule.

6. The wallet passphrase is separate.
   The BIP39 passphrase field changes wallet derivation. It is not the .arc
   encryption password and is not stored in protected .arc plaintext.
```

Terminology

The following terms are used exactly:

```
mnemonic:
    The BIP39 English recovery phrase, normalized as 12 or 24 lower-case
    words separated by U+0020 SPACE before encryption.

wallet passphrase:
    The optional BIP39 passphrase used by BIP39 seed stretching. This is a
    derivation parameter, not an .arc encryption password.

.arc password:
    The secret string passed into ARC v2 seed-file encryption. Depending on
    UI mode, this may be a human password, a keyfile-derived secret string,
    or a combined keyfile+password secret string.

raw keyfile:
    A JSON or base64 file containing random bytes. v1.6.0-generated raw
    keyfiles contain 64 bytes.

encrypted keyfile:
```

```
    A JSON file containing a password-encrypted raw keyfile payload. It is
    protected independently from the .arc file.

ARC v2:
    The current protected seed-file format with magic "ARCULUS-ARC", format
    "arculus-encrypted-seed-v2", and armor header "ARCULUS-ARC-V2".

plain .arc:
    The unprotected seed-file format with format "arculus-plain-seed-v1".
    It uses the same outer armor but has no encryption and no MAC.
```

Byte Conventions

Unless a section states otherwise:

```
Text encoding:
    UTF-8.

JSON encoding:
    ECMAScript JSON as emitted by JSON.stringify in the HTML implementation.

Newline emitted by producers:
    LF, byte 0x0a.

Newline accepted by armor parser:
    CRLF or LF.

Base64:
    RFC 4648 standard base64, alphabet A-Z a-z 0-9 + /, with "=" padding.
    The implementation uses btoa and atob-compatible standard base64.

Hex:
    Lowercase, two hexadecimal characters per byte, no "0x" prefix.

Integer-to-byte conversion for stream counters:
    Fixed-width unsigned big-endian using intToBytes(value, length).

Password normalization:
    Unicode NFKD, then UTF-8:

      password_bytes = UTF-8(NFKD(password_string))

Separator byte in ARC v2 MAC transcript:
    0x00.
```

Implementation Constants

The v1.6.0 HTML implementation defines these constants:

```
ARC_MAGIC                    = "ARCULUS-ARC"
ARC_ARMOR_HEADER             = "ARCULUS-ARC-V2"
ARC_V2_FORMAT                = "arculus-encrypted-seed-v2"
ARC_PLAIN_FORMAT             = "arculus-plain-seed-v1"
ARC_V2_KDF_ITERATIONS        = 1000000
ARC_V2_MIN_KDF_ITERATIONS    = 600000
ARC_V2_SALT_BYTES            = 32
ARC_V2_NONCE_BYTES           = 24

ARC_KEYFILE_MAGIC            = "ARCULUS-KEYFILE"
ARC_KEYFILE_ENC_MAGIC        = "ARCULUS-KEYFILE-ENC"
ARC_KEYFILE_FORMAT           = "arculus-recovery-keyfile-v1"
ARC_KEYFILE_ENC_FORMAT       = "arculus-recovery-keyfile-enc-v1"
ARC_KEYFILE_BYTES            = 64
ARC_KEYFILE_KDF_ITERATIONS   = 200000
```

Derived string constants:

```
ARC v2 KDF name              = "PBKDF2"
ARC v2 KDF hash              = "SHA-512"
ARC v2 cipher name           = "HMAC-SHA512-CTR"
ARC v2 encryption label      = "Arculus ARC v2 encryption key"
ARC v2 authentication label  = "Arculus ARC v2 authentication key"
ARC v2 stream label          = "Arculus ARC v2 stream\0"
ARC v2 MAC prefix            = "Arculus ARC v2 MAC"
```

Keyfile encrypted format constants:

```
keyfile KDF algo             = "PBKDF2-SHA512"
keyfile cipher name          = "HMAC-SHA512-CTR"
```

Combined credential constants:

```
keyfile secret prefix        = "arc-keyfile-v1:"
combined secret prefix       = "arc-combined-v1:"
combined secret label        = "arc-combined-v1\n"
```

Outer .arc Armor

Every v1.6.0 seed export, encrypted or unprotected, is serialized by serializeSeedBundle(bundle):

```
json_bytes = UTF-8(JSON.stringify(bundle))
body       = base64(json_bytes)
file_text  = "ARCULUS-ARC-V2" || LF || body || LF
```

Byte layout:

```
bytes 0..13:       ASCII "ARCULUS-ARC-V2"
byte 14:           0x0a
bytes 15..N-2:     standard base64 of UTF-8 JSON bundle
byte N-1:          0x0a
```

No byte-order mark is written. No line wrapping is inserted in the base64 body.

The parser parseSeedBundle(text) performs:

```
trimmed = text.trim()
if trimmed.startsWith("ARCULUS-ARC-V2"):
    encoded = trimmed.split(/\r?\n/).slice(1).join("").trim()
    bundle = JSON.parse(bytesToText(base64ToBytes(encoded)))
else:
    bundle = JSON.parse(text)
```

Important parser consequences:

```
- Leading and trailing whitespace outside the armor are ignored.
- CRLF and LF are both accepted between armor lines.
- If the base64 body is manually wrapped across multiple lines, those lines
  are joined without separators before decoding.
- The parser does not require the final LF after trimming.
- Files without the armor header are treated as raw JSON legacy inputs.
- The same outer header is used for encrypted and unprotected .arc bundles.
```

Protected .arc Bundle Structure

The protected seed bundle produced by encrypt_v2(mnemonic, password) is a JSON object with this v1.6.0 insertion order before credential_mode is appended:

```
{
  "magic": "ARCULUS-ARC",
  "format": "arculus-encrypted-seed-v2",
  "version": 2,
  "created_at": "<Date.toISOString string>",
  "kdf": {
    "name": "PBKDF2",
    "hash": "SHA-512",
    "iterations": 1000000,
    "salt_b64": "<base64 32 bytes>"
  },
  "cipher": {
    "name": "HMAC-SHA512-CTR",
    "nonce_b64": "<base64 24 bytes>"
  },
  "ciphertext_b64": "<base64 ciphertext bytes>",
  "mac_b64": "<base64 64 bytes>"
}
```

After encryptSeedBundle returns, handleEncryptExportSeed appends:

```
"credential_mode": "password" | "keyfile" | "both"
```

credential_mode is a UI hint. It is appended after MAC calculation in v1.6.0. Therefore, it is not part of the MAC transcript for files produced by v1.6.0. Import also accepts the aliases credentialMode and protection as UI hints:

```
hintedMode = normalizeArcCredentialMode(
    bundle.credential_mode || bundle.credentialMode || bundle.protection
)
```

The credential hint selects or locks the import dialog mode. It is not a cryptographic policy field and must not be used as proof of protection type.

Protected .arc Plaintext Payload

The encrypted plaintext is produced by:

```
words = normalizeMnemonicWords(mnemonic)
createdAt = new Date().toISOString()

payload = {
  mnemonic: words.join(" "),
  word_count: words.length,
  created_at: createdAt
}

plaintext = UTF-8(JSON.stringify(payload))
```

Properties:

- words.length must be 12 or 24.
- words are joined by exactly one U+0020 SPACE.
- JSON.stringify is called without indentation.
- No trailing LF is included in the plaintext.
- created_at inside the plaintext equals bundle.created_at.

The plaintext does not contain:

- BIP39 wallet passphrase
- derived private keys
- WIF strings
- xprv/yprv/zprv/tprv
- xpub/ypub/zpub/tpub
- addresses
- derivation paths
- coin selection
- output table data
- QR payloads

Mnemonic Normalization on Export and Import

Export uses normalizeMnemonicWords(mnemonic) before encryption. Import uses normalizeDecryptedMnemonic(payload) after decryption.

normalizeDecryptedMnemonic requires:

```
payload is a non-null object
payload.mnemonic normalizes to at least one word
if payload.word_count is present:
    Number(payload.word_count) == normalized word count
```

It returns:

```
normalized_words.join(" ")
```

handleImportSeedFile then checks the returned mnemonic with checkMnemonic using the current UI wallet passphrase field. A protected file can decrypt correctly and still be rejected by the UI if the recovered phrase is not valid BIP39.

ARC v2 Key Schedule

Function:

```
arcV2Keys(password, saltBytes, iterations)
```

Inputs:

```
password:
    JavaScript string. In Password mode, this is the user-entered password.
    In Keyfile mode, this is a keyfile secret string. In combined mode, this
    is a combined secret string.

saltBytes:
    32 bytes for protected .arc files.

iterations:
    Integer KDF iteration count from bundle.kdf.iterations during import or
    ARC_V2_KDF_ITERATIONS during export.
```

Validation:

```
Number.isInteger(iterations) must be true.
iterations >= 600000.
```

Failure message:

```
"Encrypted seed file uses too few KDF iterations."
```

Step 1: normalize password:

```
password_bytes = UTF-8(NFKD(password))
```

Step 2: derive master key:

```
master_key = PBKDF2-HMAC-SHA512(
    password   = password_bytes,
    salt       = saltBytes,
    iterations = iterations,
    dkLen      = 64
)
```

Step 3: derive encryption key:

```
enc_key = HMAC-SHA512(
    key  = master_key,
    data = UTF-8("Arculus ARC v2 encryption key")
)
```

Step 4: derive authentication key:

```
mac_key = HMAC-SHA512(
    key  = master_key,
    data = UTF-8("Arculus ARC v2 authentication key")
)
```

Outputs:

```
enc_key: 64 bytes
mac_key: 64 bytes
```

The master key is not directly used for encryption or authentication. The domain-separated labels prevent reuse of the same PBKDF2 output as both a stream-generation key and a MAC key.

PBKDF2-HMAC-SHA512 Details

Function:

```
pbkdf2Sha512(passwordBytes, saltBytes, iterations, dkLen)
```

Cryptographic primitive:

```
WebCrypto PBKDF2 with hash "SHA-512"
```

Output length:

```
dkLen bytes. ARC v2 uses 64. Encrypted keyfiles also use 64.
```

The effective PBKDF2 input for ordinary Password mode is:

```
UTF-8(NFKD(user_password_string))
```

The effective PBKDF2 input for Keyfile mode is:

```
UTF-8(NFKD("arc-keyfile-v1:" || base64(keyfile_bytes)))
```

The effective PBKDF2 input for Keyfile + Password mode is:

```
UTF-8(NFKD("arc-combined-v1:" || base64(SHA-256(material))))
```

where material is specified in the combined credential section.

ARC v2 HMAC-SHA512 Counter Stream

Function:

```
arcV2StreamCrypt(data, keyBytes, nonceBytes)
```

This transform is symmetric. The same function encrypts plaintext and decrypts ciphertext.

Inputs:

```
data:
    Bytes to XOR with the generated stream.

keyBytes:
    HMAC key. For ARC v2 seed files this is the 64-byte enc_key. For
    encrypted keyfiles this is the 32-byte encKey slice from keyfile PBKDF2.

nonceBytes:
    Nonce bytes. Protected .arc files use 24 bytes. Encrypted keyfiles use
    16 bytes.
```

Internal constants:

```
label = UTF-8("Arculus ARC v2 stream\0")
counter_initial = 0
counter_width = 8 bytes
counter_endian = big-endian
block_size = 64 bytes
```

For each block i:

```
counter_i = uint64_be(i)

block_i = HMAC-SHA512(
    key  = keyBytes,
    data = label || nonceBytes || counter_i
)
```

Keystream:

```
stream = block_0 || block_1 || block_2 || ...
```

Output:

```
out[j] = data[j] XOR stream[j]
```

The final block is truncated to the remaining data length. No padding is applied. A zero-length input produces a zero-length output.

Why the Current Protected .arc Format Is Not AES-GCM

New v1.6.0 protected .arc exports use HMAC-SHA512-CTR plus HMAC-SHA512 authentication, not AES-GCM. AES-GCM appears only in the legacy "arculus-encrypted-seed-v1" import path.

The current construction is:

```
PBKDF2-HMAC-SHA512 -> HMAC-derived enc_key and mac_key
HMAC-SHA512-CTR    -> ciphertext
HMAC-SHA512        -> MAC over metadata and ciphertext
```

This is an encrypt-then-MAC construction. Import verifies the MAC before decrypting. It is not AEAD by API, but the MAC transcript authenticates the same material that an AEAD associated-data field would need to cover.

ARC v2 MAC Transcript

Function:

```
arcV2MacData(bundle, saltBytes, nonceBytes, ciphertext)
```

The MAC transcript is not the raw JSON string. It is an explicitly constructed byte sequence. This avoids dependence on JSON key order or whitespace.

The function builds:

```
sep = 0x00

textParts = [
  "Arculus ARC v2 MAC",
  String(bundle.magic || ""),
  String(bundle.format || ""),
  String(bundle.version || ""),
  String(bundle.created_at || ""),
  String(bundle.kdf.name || ""),
  String(bundle.kdf.hash || ""),
  String(bundle.kdf.iterations || ""),
  String(bundle.cipher.name || "")
]
```

For each text part:

```
encoded.push(UTF-8(part), sep)
```

Then append:

```
saltBytes || sep || nonceBytes || sep || ciphertext
```

Full transcript:

```
UTF8("Arculus ARC v2 MAC")           || 00 ||
UTF8(bundle.magic)                   || 00 ||
UTF8(bundle.format)                  || 00 ||
UTF8(String(bundle.version))         || 00 ||
UTF8(bundle.created_at)              || 00 ||
UTF8(bundle.kdf.name)                || 00 ||
UTF8(bundle.kdf.hash)                || 00 ||
UTF8(String(bundle.kdf.iterations))  || 00 ||
UTF8(bundle.cipher.name)             || 00 ||
saltBytes                            || 00 ||
nonceBytes                           || 00 ||
ciphertext
```

MAC:

```
mac = HMAC-SHA512(
    key  = mac_key,
    data = transcript
)
```

Stored value:

```
mac_b64 = base64(mac)
```

Stored length:

```
64 bytes before base64 encoding.
```

Authenticated fields:

- magic
- format
- version
- created_at
- kdf.name
- kdf.hash
- kdf.iterations
- cipher.name
- salt bytes
- nonce bytes

- ciphertext bytes

Not authenticated by v1.6.0 producer:

- credential_mode

The NUL separators prevent transcript collisions caused by moving bytes between adjacent variable-length fields.

Protected .arc Export Procedure

Function path:

```
handleEncryptExportSeed
  -> requestArcCredential("export")
  -> encryptSeedBundle(mnemonic, credential.secret)
  -> encrypt_v2(mnemonic, password)
  -> serializeSeedBundle(bundle)
```

Procedure:

```
1. Read active mnemonic.
2. Validate BIP39 words and checksum with checkMnemonic.
3. Ask for credential mode:
     password
     keyfile
     both
     or advanced unprotected export
4. If protected, obtain credential.secret.
5. Normalize mnemonic words.
6. Reject word counts other than 12 or 24.
7. Generate salt:
     crypto.getRandomValues(new Uint8Array(32))
8. Generate nonce:
     crypto.getRandomValues(new Uint8Array(24))
9. Generate createdAt:
     new Date().toISOString()
10. Build initial bundle containing magic, format, version, created_at, kdf,
    and cipher objects.
11. Build plaintext payload JSON.
12. Derive enc_key and mac_key with ARC v2 key schedule and 1000000
    iterations.
13. Encrypt plaintext with arcV2StreamCrypt.
14. Store ciphertext_b64.
15. Compute HMAC-SHA512 over ARC v2 MAC transcript.
16. Store mac_b64.
17. Append credential_mode as a UI hint.
18. Armor with ARCULUS-ARC-V2 header.
19. Save as:
     Arculus_Encrypted_Seed.arc
```

For the same mnemonic and same credential secret, repeated exports should produce different protected files because salt and nonce are freshly sampled.

Protected .arc Import Procedure

Function path:

```
handleImportSeedFile(file)
  -> parseSeedBundle(text)
  -> requestArcCredential("import", hintedMode, lockMode)
  -> decryptSeedBundle(bundle, credential.secret)
  -> decrypt(bundle, password)
  -> decryptV2(bundle, password)
```

Procedure:

```
1. Read selected file as text.
2. Parse armor or raw JSON.
3. Read credential hint from credential_mode, credentialMode, or protection.
4. If bundle is plain .arc, skip credential dialog.
5. Otherwise request credential secret.
6. Dispatch:
     if magic == "ARCULUS-ARC" and format == "arculus-plain-seed-v1":
         normalize plain mnemonic
     else if magic == "ARCULUS-ARC" or version == 2:
         decryptV2
```

```
    else:
        decryptV1 legacy path
```

decryptV2 validation:

```
bundle.magic must equal "ARCULUS-ARC"
bundle.version must equal 2
bundle.format must equal "arculus-encrypted-seed-v2"
bundle.kdf must exist
bundle.cipher must exist
bundle.kdf.name must equal "PBKDF2"
bundle.kdf.hash must equal "SHA-512"
bundle.cipher.name must equal "HMAC-SHA512-CTR"
bundle.kdf.iterations must be an integer >= 600000
salt_b64 must decode to exactly 32 bytes
nonce_b64 must decode to exactly 24 bytes
mac_b64 must decode to exactly 64 bytes
ciphertext_b64 must decode successfully; no fixed length is required
```

decryptV2 authentication:

```
expectedMac = decoded mac_b64
{ encKey, macKey } = arcV2Keys(password, salt, iterations)
actualMac = HMAC-SHA512(macKey, arcV2MacData(...))
equalBytes(actualMac, expectedMac) must be true
```

Only after MAC success:

```
decrypted = arcV2StreamCrypt(ciphertext, encKey, nonce)
payload = JSON.parse(TextDecoder().decode(decrypted))
mnemonic = normalizeDecryptedMnemonic(payload)
```

After decryptSeedBundle returns:

```
checkMnemonic(mnemonic, current_wallet_passphrase)
```

If the BIP39 check fails, the UI raises:

```
"Imported file decrypted successfully, but the mnemonic is not valid BIP-39."
```

Constant-Time Comparison

Function:

```
equalBytes(a, b)
```

Behavior:

```
if a.length !== b.length:
    return false

diff = 0
for i in 0..a.length-1:
    diff |= a[i] ^ b[i]

return diff === 0
```

For equal-length arrays, every byte is read before returning. This prevents prefix-length timing leakage during MAC comparison. The length check occurs before the XOR loop. For ARC v2 MACs, the stored MAC length is validated to 64 bytes before equalBytes is called.

Base64 Decoding and Length Checks

General base64 conversion:

```
base64ToBytes(text):
    binary = atob(text)
    out = new Uint8Array(binary.length)
    for each byte position:
        out[i] = binary.charCodeAt(i)
```

Required protected .arc fields use:

```
b64DecodeRequired(value, fieldName, expectedLength = null)
```

Validation:

- value must be a non-empty string

- atob-compatible base64 decoding must succeed
- if expectedLength is not null, decoded length must equal expectedLength

Failure messages are field-specific:

```
"Encrypted seed file is missing <fieldName>."
"Encrypted seed file has invalid <fieldName>."
"Encrypted seed file has invalid <fieldName> length."
```

The ciphertext field is required but not length-constrained:

```
b64DecodeRequired(bundle.ciphertext_b64, "ciphertext_b64")
```

Unprotected .arc Bundle

The unprotected .arc path exists for controlled offline handling and test data. It uses the same ARCULUS-ARC-V2 outer armor but a plaintext internal bundle.

Function:

```
makePlainSeedBundle(mnemonic)
```

Structure:

```
{
  "magic": "ARCULUS-ARC",
  "format": "arculus-plain-seed-v1",
  "version": 1,
  "created_at": "<Date.toISOString string>",
  "mnemonic": "<normalized 12 or 24 words>",
  "word_count": 12 or 24,
  "encrypted": false
}
```

Serialization:

```
serializeSeedBundle(bundle)
```

Import:

```
decrypt(bundle, password) detects:

  bundle.magic == "ARCULUS-ARC"
  bundle.format == "arculus-plain-seed-v1"

It then returns normalizeDecryptedMnemonic(bundle).
```

Security properties:

- No encryption.
- No MAC.
- Anyone with the file can read the mnemonic after base64-decoding the armor.
- The outer armor is only a container, not a protection mechanism.

Credential Modes

The credential dialog supports three protected modes:

```
password:
    User-entered password string is passed directly as ARC v2 password.

keyfile:
    A raw or encrypted keyfile is converted to a secret string:

      "arc-keyfile-v1:" || base64(keyfile_bytes)

    That secret string is passed as ARC v2 password.

both:
    The selected/generated keyfile is converted to the keyfile secret string.
    The UI password and keyfile secret are combined with SHA-256 into a
    combined secret string:

      "arc-combined-v1:" || base64(sha256(material))
```

```
    The combined secret string is passed as ARC v2 password.
```

The credential dialog also has an advanced unprotected export option:

```
unprotected:
    makePlainSeedBundle is used instead of encrypt_v2.
```

Password mode export requirements:

- Password must be non-empty.
- Password confirmation must match.

Keyfile mode export requirements:

```
- A keyfile secret must be available, either by generating a new keyfile or
  choosing an existing keyfile through the advanced keyfile chooser.
```

Combined mode export requirements:

- Password must be non-empty.
- Password confirmation must match.
- A keyfile secret must be available.

Import with a credential hint:

```
- If credential_mode is one of password, keyfile, both, the dialog selects
  and locks that mode.
- If no valid hint exists, the user chooses the mode.
```

Raw Keyfile Secret

Function:

```
arcKeyfileSecretFromBytes(bytes)
```

Validation:

```
bytes must be a Uint8Array.
bytes.length must be at least 32.
```

Failure:

```
"Keyfile must contain at least 256 bits of key material."
```

Secret string:

```
keyfile_secret = "arc-keyfile-v1:" || base64(bytes)
```

Generated keyfiles use:

```
key = crypto.getRandomValues(new Uint8Array(64))
```

Generated keyfile entropy:

```
64 bytes = 512 bits.
```

The keyfile_secret string is not itself stored inside the .arc file. It is reconstructed from the keyfile during export/import and used as the ARC v2 password input.

Raw Keyfile File Format

Function:

```
makeArcKeyfile(password = "")
```

If password is empty, output object:

```
{
  "magic": "ARCULUS-KEYFILE",
  "format": "arculus-recovery-keyfile-v1",
  "version": 1,
  "created_at": "<Date.toISOString string>",
  "key_b64": "<base64 64 bytes>"
}
```

Serialization:

```
JSON.stringify(obj, null, 2) || LF
```

Filename:

```
Arculus_Recovery_Keyfile.key
```

Import accepts:

```
- JSON object with:
    magic == "ARCULUS-KEYFILE"
    format == "arculus-recovery-keyfile-v1"
    key_b64 present

- Base64-only file:
    file text may contain base64 characters and whitespace
    whitespace is removed
    decoded bytes must be at least 32 bytes

- Legacy JSON object:
    magic == "KEYFILE"
    key contains base64 raw key bytes
```

Encrypted Keyfile Format

Function:

```
encryptArcKeyfileBytes(keyBytes, password)
```

Input keyBytes:

```
64 random bytes for v1.6.0-generated keyfiles.
```

Password:

```
Must be non-empty.
```

Failure:

```
"Password cannot be empty when using Keyfile + Password."
```

Random parameters:

```
salt  = crypto.getRandomValues(new Uint8Array(16))
nonce = crypto.getRandomValues(new Uint8Array(16))
```

Key derivation:

```
derived = PBKDF2-HMAC-SHA512(
    password   = UTF-8(NFKD(password)),
    salt       = salt,
    iterations = 200000,
    dkLen      = 64
)
```

Split:

```
encKey = derived[0:32]
macKey = derived[32:64]
```

Encryption:

```
ciphertext = arcV2StreamCrypt(keyBytes, encKey, nonce)
```

Although the function name is arcV2StreamCrypt, encrypted keyfiles use:

```
keyBytes = 32-byte encKey slice
nonceBytes = 16 bytes
stream label = "Arculus ARC v2 stream\0"
counter = uint64_be starting at 0
HMAC-SHA512 block output = 64 bytes
```

Authentication:

```
mac = HMAC-SHA512(
    key  = macKey,
    data = salt || nonce || ciphertext
)
```

Stored object:

```
{
  "magic": "ARCULUS-KEYFILE-ENC",
  "format": "arculus-recovery-keyfile-enc-v1",
  "version": 1,
  "created_at": "<Date.toISOString string>",
  "kdf": {
    "algo": "PBKDF2-SHA512",
    "iterations": 200000,
    "salt_b64": "<base64 16 bytes>"
  },
  "cipher": {
    "name": "HMAC-SHA512-CTR",
    "nonce_b64": "<base64 16 bytes>"
  },
  "ciphertext_b64": "<base64 ciphertext bytes>",
  "mac_b64": "<base64 64 bytes>"
}
```

Serialization:

```
JSON.stringify(obj, null, 2) || LF
```

Filename:

```
Arculus_Recovery_Keyfile.enc.key
```

Memory cleanup:

```
encryptArcKeyfileBytes calls fill(0) on derived, encKey, and macKey after
computing the encrypted keyfile object. This is best-effort JavaScript heap
cleanup and does not guarantee erasure of copies held internally by the
browser engine or WebCrypto implementation.
```

Encrypted Keyfile Import

Function:

```
decryptArcKeyfileObject(obj, password)
```

Password:

```
Must be non-empty.
```

Failure:

```
  "Enter the same password used with Keyfile + Password before loading this encrypted
keyfile."
```

Accepted field names:

```
salt:
    obj.kdf.salt_b64 or obj.kdf.s

nonce:
    obj.cipher.nonce_b64 or obj.n

ciphertext:
    obj.ciphertext_b64 or obj.ct

MAC:
    obj.mac_b64 or obj.mac

iterations:
    obj.kdf.iterations or obj.kdf.i or 200000 default
```

Derivation:

```
derived = PBKDF2-HMAC-SHA512(
    password   = UTF-8(NFKD(password)),
    salt       = decoded salt,
    iterations = Number(iterations),
    dkLen      = 64
)
```

Split:

```
encKey = derived[0:32]
macKey = derived[32:64]
```

Authentication:

```
actualMac = HMAC-SHA512(macKey, salt || nonce || ciphertext)
equalBytes(actualMac, expectedMac) must be true
```

Failure:

```
"Unable to decrypt keyfile. The password may be incorrect or the keyfile may be corrupted.
"
```

Only after MAC success:

```
plain = arcV2StreamCrypt(ciphertext, encKey, nonce)
```

The caller then converts plain bytes into:

```
"arc-keyfile-v1:" || base64(plain)
```

and zeroes the plain Uint8Array with fill(0).

Accepted encrypted keyfile objects:

```
- magic == "ARCULUS-KEYFILE-ENC"
  format == "arculus-recovery-keyfile-enc-v1"
```

- legacy magic == "KEYFILE-ENC"

Combined Keyfile + Password Secret

Function:

```
combinedArcSecret(password, keyfileSecret)
```

Inputs:

```
password:
    UI password string.

keyfileSecret:
    String produced by arcKeyfileSecretFromBytes:

      "arc-keyfile-v1:" || base64(keyfile_bytes)
```

Material:

```
material = UTF-8(
    "arc-combined-v1\n" ||
    NFKD(password) ||
    "\n" ||
    keyfileSecret
)
```

Digest:

```
digest = SHA-256(material)
```

Combined secret:

```
"arc-combined-v1:" || base64(digest)
```

The combined secret is the password argument passed to encryptSeedBundle and decryptSeedBundle. It is then normalized again with NFKD by arcV2Keys before PBKDF2. The string consists of ASCII prefix and standard base64 output, so NFKD does not alter generated combined secrets.

Security consequence:

```
The protected .arc file cannot be opened with only the password or only the
keyfile. Both are required to reconstruct the same combined secret.
```

Operational consequence:

```
The same password used for combined mode also encrypts the generated encrypted
keyfile when the user generates a password-protected keyfile during export.
Losing either the .arc file, the keyfile, or the password makes recovery
impossible for combined-mode backups.
```

Credential Mode Normalization

Function:

```
normalizeArcCredentialMode(mode)
```

Accepted values:

```
"password"             -> "password"
"keyfile"              -> "keyfile"
"both"                 -> "both"
"keyfile + password"   -> "both"
"keyfile-password"     -> "both"
```

Any other value returns:

```
""
```

This function is used for UI hint normalization only.

Legacy Seed Import Family: HMAC-SHA256 Stream

Function:

```
decryptV1(bundle, password)
```

This branch accepts:

```
bundle.format == "arculus-encrypted-seed-v2"
or
bundle.format == "arculus-encrypted-seed-python-v1"
```

This is a legacy compatibility branch and is distinct from current ARC v2.

Legacy inputs:

```
salt       = base64ToBytes(bundle.kdf.salt_b64 || "")
nonce      = base64ToBytes(bundle.cipher.nonce_b64 || "")
ciphertext = base64ToBytes(bundle.ciphertext_b64 || "")
expected   = base64ToBytes(bundle.mac_b64 || "")
iterations = Number(bundle.kdf.iterations || 250000)
```

Password:

```
passwordBytes = UTF-8(NFKD(password))
```

KDF:

```
derived = PBKDF2-HMAC-SHA256(
    password   = passwordBytes,
    salt       = salt,
    iterations = iterations,
    dkLen      = 64
)
```

Split:

```
encKey = derived[0:32]
macKey = derived[32:64]
```

MAC:

```
actualMac = HMAC-SHA256(macKey, nonce || ciphertext)
```

Comparison:

```
equalBytes(actualMac, expectedMac)
```

Stream:

```
xorStreamCrypt(ciphertext, encKey, nonce)
```

Legacy stream block:

```
block_i = SHA-256(encKey || nonce || uint32_be(i))
```

where i begins at 0 and is encoded as 4 bytes big-endian. The block size is 32 bytes because SHA-256 output is 32 bytes.

Plaintext:

```
JSON.parse(TextDecoder().decode(decrypted))
normalizeDecryptedMnemonic(payload)
```

New v1.6.0 exports do not produce this format.

Legacy Seed Import Family: AES-GCM v1

Function:

```
decryptV1(bundle, password)
```

This branch accepts:

```
bundle.format == "arculus-encrypted-seed-v1"
```

Legacy inputs:

```
salt       = base64ToBytes(bundle.kdf.salt_b64 || "")
iv         = base64ToBytes(bundle.cipher.iv_b64 || "")
ciphertext = base64ToBytes(bundle.ciphertext_b64 || "")
iterations = Number(bundle.kdf.iterations || 250000)
```

Password:

```
passwordBytes = UTF-8(NFKD(password))
```

KDF:

```
WebCrypto PBKDF2 with:
    hash       = "SHA-256"
    salt       = salt
    iterations = iterations
```

Derived key:

```
AES-GCM 256-bit CryptoKey
```

Decrypt:

```
crypto.subtle.decrypt(
    { name: "AES-GCM", iv: iv },
    key,
    ciphertext
)
```

The ciphertext field is the WebCrypto AES-GCM ciphertext including its authentication tag, as returned by the earlier implementation.

Plaintext:

```
JSON.parse(TextDecoder().decode(decrypted))
normalizeDecryptedMnemonic(payload)
```

New v1.6.0 exports do not produce this format.

Dispatch Matrix

decrypt(bundle, password) dispatches as follows:

```
if bundle is not an object:
    reject as unsupported encrypted seed file format

if bundle.magic == "ARCULUS-ARC"
   and bundle.format == "arculus-plain-seed-v1":
    return normalizeDecryptedMnemonic(bundle)

if bundle.magic == "ARCULUS-ARC"
   or bundle.version == 2:
    return decryptV2(bundle, password)

otherwise:
    return decryptV1(bundle, password)
```

Compatibility notes:

```
- Current protected v2 exports use magic "ARCULUS-ARC" and version 2.
- Current plain exports use magic "ARCULUS-ARC" and format
  "arculus-plain-seed-v1".
- Legacy raw JSON files without the armor header are still parsed as JSON.
```

```
- A malformed object with version 2 but without current v2 fields is routed
  to decryptV2 and rejected by v2 validation.
```

Format Compatibility Matrix

Seed file formats:

```
ARCULUS-ARC-V2 armored protected v2
    Export: yes
    Import: yes
    Protection: PBKDF2-HMAC-SHA512, HMAC-SHA512-CTR, HMAC-SHA512 MAC

ARCULUS-ARC-V2 armored plain v1
    Export: yes, advanced option
    Import: yes
    Protection: none

Raw JSON current protected v2 object
    Export: no, but JSON parser accepts it
    Import: yes if fields validate
    Protection: same as current protected v2

Legacy HMAC-SHA256 stream object
    Export: no
    Import: yes
    Protection: PBKDF2-HMAC-SHA256, SHA256 XOR stream, HMAC-SHA256 MAC

Legacy AES-GCM v1 object
    Export: no
    Import: yes in browser
    Protection: PBKDF2-HMAC-SHA256, AES-GCM-256
```

Keyfile formats:

```
ARCULUS-KEYFILE raw JSON
    Export: yes
    Import: yes

Base64-only raw key material
    Export: no
    Import: yes

ARCULUS-KEYFILE-ENC encrypted JSON
    Export: yes
    Import: yes

KEYFILE legacy JSON
    Export: no
    Import: yes

KEYFILE-ENC legacy JSON
    Export: no
    Import: yes
```

Protected Fields and Unprotected Fields

Protected by current ARC v2 seed-file encryption:

- mnemonic plaintext confidentiality at rest
- mnemonic plaintext integrity after MAC success and JSON parse
- magic string
- protected format identifier
- version integer
- created_at timestamp
- KDF name
- KDF hash
- KDF iteration count
- salt bytes

- cipher name
- nonce bytes
- ciphertext bytes

Not protected by current ARC v2 seed-file encryption:

- credential_mode hint appended after MAC generation
- outer armor header itself
- filesystem filename
- file modification timestamp
- mnemonic while displayed in the browser
- mnemonic while held in JavaScript memory
- .arc password while typed or held in memory
- keyfile bytes while held in memory
- generated raw keyfile if stored unencrypted
- wallet passphrase field
- derived-output JSON/CSV/TXT/PDF exports
- clipboard contents
- screenshots, screen recording, browser extensions, or compromised OS state

Protected by encrypted keyfile format:

```
- keyfile bytes at rest
- salt, nonce, and ciphertext integrity through HMAC-SHA512 over
  salt || nonce || ciphertext
```

Not protected by encrypted keyfile format:

- filename
- created_at metadata
- kdf/cipher metadata integrity as a structured transcript
- magic and format fields by MAC commitment

The encrypted keyfile MAC is simpler than ARC v2 seed-file MAC. It authenticates the raw salt, nonce, and ciphertext bytes but does not bind the metadata strings through a separate transcript.

Failure Behavior

Protected .arc structural failures:

- unsupported magic/version/format
- missing kdf or cipher object
- unsupported KDF name or hash
- unsupported cipher name
- KDF iterations below 600000
- invalid or missing base64 fields
- invalid decoded salt length
- invalid decoded nonce length
- invalid decoded MAC length

Protected .arc authentication failure:

```
  If equalBytes(actualMac, expectedMac) is false, decryptV2 throws:

    "Unable to decrypt seed file. The password may be incorrect or the file may be
corrupted."
```

No decryption is attempted before this check succeeds.

Protected .arc plaintext failures:

- decrypted bytes are not valid UTF-8 JSON
- parsed plaintext is not an object
- mnemonic is missing or normalizes to empty
- word_count does not match normalized mnemonic length
- final BIP39 check fails

Encrypted keyfile failures:

- missing password
- invalid JSON and not base64-only material
- unsupported keyfile magic/format
- decoded raw key material shorter than 32 bytes
- MAC mismatch

The user-facing messages deliberately avoid distinguishing wrong password from corruption for protected seed files and encrypted keyfiles.

KDF Iteration Rationale

Current protected seed exports use:

```
  PBKDF2-HMAC-SHA512, 1000000 iterations
```

Current protected seed imports require:

```
  iterations >= 600000
```

This import floor allows files from recent hardened builds while preventing an attacker from lowering the stored iteration count to make password guessing cheaper. The iteration count is also included in the ARC v2 MAC transcript, so an attacker cannot modify it without causing MAC failure.

Encrypted keyfiles use:

```
  PBKDF2-HMAC-SHA512, 200000 iterations
```

The keyfile KDF protects random 64-byte key material rather than a mnemonic directly. In combined mode, the keyfile password protects the encrypted keyfile, while the final .arc envelope is protected by the combined secret derived from both password and keyfile material.

PBKDF2 is CPU-hard but not memory-hard. It slows offline guessing linearly with the iteration count. It does not rescue weak passwords. A captured .arc file can be attacked offline by testing candidate credentials until MAC verification succeeds.

Password Entropy Boundary

The cryptographic strength of Password mode is bounded by the entropy of the user password and the PBKDF2 cost. A short password, common phrase, reused password, PIN, or personally meaningful phrase should be treated as weak even with 1000000 PBKDF2 iterations.

The cryptographic strength of Keyfile mode is bounded by possession of the keyfile bytes. v1.6.0-generated keyfiles contain 64 random bytes. If the keyfile is copied to cloud storage, email, screenshots, or a compromised machine, the .arc file protected only by that keyfile should be considered compromised by anyone who obtains both files.

The cryptographic strength of Keyfile + Password mode requires both:

- the password

- the keyfile bytes

This mode provides better loss-of-one-factor resistance, but it also increases backup fragility. Recovery requires preserving both factors exactly.

Operational Backup Procedure

For Password mode:

- Load or generate the mnemonic.
- Export protected .arc with a high-entropy password.
- Import the .arc file immediately on the same offline system.
- Confirm BIP39 validation succeeds.
- Store the .arc and password separately.

For Keyfile mode:

```
1. Generate or select a keyfile.
2. Export protected .arc using Keyfile mode.
3. Import the .arc using the same keyfile.
4. Confirm BIP39 validation succeeds.
5. Store .arc and keyfile separately.
6. If the raw keyfile is not password-encrypted, treat it as a high-value
   secret equivalent to the ability to open the .arc file.
```

For Keyfile + Password mode:

- Generate or select a keyfile.
- Use a high-entropy password.
- Export protected .arc using combined mode.
- Import the .arc using the same keyfile and password.
- Confirm BIP39 validation succeeds.
- Store .arc, keyfile, and password in separate controlled locations.

For unprotected .arc:

```
Use only for test data or tightly controlled offline procedures. The mnemonic
is available to anyone who can base64-decode the armored JSON body.
```

Recovery Requirements

To recover from a protected .arc file:

```
Password mode:
    Need .arc file and exact password string after NFKD normalization.

Keyfile mode:
    Need .arc file and exact keyfile bytes.

Keyfile + Password mode:
    Need .arc file, exact keyfile bytes, and exact password string after NFKD
    normalization.

Plain .arc:
    Need only the .arc file.
```

The BIP39 wallet passphrase is not stored in any .arc file. Recovering the same wallet tree also requires knowing whether a wallet passphrase was used during address derivation.

Implementation Function Map

Core byte helpers:

```
base64ToBytes(text)
    Standard base64 decode via atob into Uint8Array.

bytesToBase64(bytes)
```

```
    Standard base64 encode.

textToBytes(text)
    UTF-8 encode.

bytesToText(bytes)
    UTF-8 decode.

normalizeNFKD(text)
    Unicode NFKD string normalization.

intToBytes(value, length)
    Big-endian fixed-width integer encoding.

concatBytes(...arrays)
    Byte-array concatenation.

equalBytes(a, b)
    Equal-length XOR-accumulation comparison.
```

ARC v2 protected seed functions:

```
arcV2Keys(password, saltBytes, iterations)
    PBKDF2-SHA512 master key plus HMAC-SHA512 domain-separated enc/mac keys.

arcV2StreamCrypt(data, keyBytes, nonceBytes)
    HMAC-SHA512 counter-mode XOR transform.

arcV2MacData(bundle, saltBytes, nonceBytes, ciphertext)
    NUL-separated metadata transcript plus salt, nonce, and ciphertext.

encrypt_v2(mnemonic, password)
    Current protected seed export.

decryptV2(bundle, password)
    Current protected seed import.

encryptSeedBundle(mnemonic, password)
    Wrapper for encrypt_v2.

decryptSeedBundle(bundle, password)
    Wrapper for decrypt.

serializeSeedBundle(bundle)
    ARCULUS-ARC-V2 armor writer.

parseSeedBundle(text)
    Armor/raw JSON parser.
```

Plain .arc functions:

```
makePlainSeedBundle(mnemonic)
    Creates unprotected arculus-plain-seed-v1 bundle.
```

Legacy seed functions:

```
decryptV1(bundle, password)
    Handles PBKDF2-SHA256 stream family and AES-GCM v1 family.

xorStreamCrypt(data, keyBytes, nonceBytes)
    Legacy SHA-256 stream XOR transform.
```

Keyfile functions:

```
arcKeyfileSecretFromBytes(bytes)
    Validates key material and returns arc-keyfile-v1 secret string.

encryptArcKeyfileBytes(keyBytes, password)
    Password-encrypts keyfile bytes.

decryptArcKeyfileObject(obj, password)
    Authenticates and decrypts encrypted keyfile JSON.

makeArcKeyfile(password = "")
    Generates raw or encrypted keyfile text.

parseArcKeyfileText(text, password)
    Parses raw, encrypted, base64-only, or legacy keyfile input.

readArcKeyfile(file, password)
    Reads browser File text and parses it.
```

Credential functions:

```
combinedArcSecret(password, keyfileSecret)
    Creates arc-combined-v1 secret.

normalizeArcCredentialMode(mode)
    Converts credential hint aliases into password, keyfile, both, or empty.

requestArcCredential(context, preferredMode, lockMode)
    Opens the credential dialog and returns selected secret data.

submitArcCredentialDialog()
    Validates the selected credential mode and constructs the correct secret.
```

UI integration:

```
handleEncryptExportSeed()
    Validates mnemonic, obtains credential, writes .arc file.

handleImportSeedFile(file)
    Parses .arc, obtains credential, decrypts or loads mnemonic.
```

Security Classification

Treat these as highly sensitive secrets:

- BIP39 mnemonic
- .arc password
- raw keyfile bytes
- encrypted keyfile password
- combined-mode password
- combined secret string
- decrypted .arc plaintext JSON
- decrypted keyfile plaintext

Treat these as sensitive encrypted artifacts:

- protected .arc files
- encrypted keyfile files

Treat these as plaintext secret-bearing artifacts:

- unprotected .arc files
- raw keyfile files
- derived-output JSON exports
- derived-output CSV exports
- derived-output TXT exports
- derived-output PDF exports
- screenshots showing mnemonics or keys

Treat these as non-secret but integrity-relevant:

- ARC v2 constants
- KDF iteration counts
- MAC transcript field order
- stream labels
- credential mode hint names
- legacy dispatch format strings

Compliance Checklist

An implementation is compatible with Arculus_Recovery.html v1.6.0 protected .arc exports if it satisfies all of the following:

```
- Parses ARCULUS-ARC-V2 armor exactly enough to recover JSON bundle bytes.
- Uses UTF-8 for all text fields.
- Applies NFKD normalization to the credential string before PBKDF2.
- Requires current v2 magic, format, version, KDF name/hash, and cipher name.
- Requires salt length 32 bytes.
- Requires nonce length 24 bytes.
- Requires MAC length 64 bytes.
- Requires iterations >= 600000.
- Derives 64-byte PBKDF2-HMAC-SHA512 master key.
- Derives enc_key and mac_key with the exact label strings.
- Constructs the MAC transcript with the exact field order and 0x00
  separators.
- Verifies HMAC-SHA512 before decrypting.
- Generates HMAC-SHA512 stream blocks from:

      UTF8("Arculus ARC v2 stream\0") || nonce || uint64_be(counter)
```

- XORs ciphertext with the stream to recover plaintext.
- Parses plaintext JSON as UTF-8.
- Normalizes mnemonic words and validates word_count if present.

An implementation is compatible with v1.6.0 encrypted keyfiles if it:

```
- Parses JSON keyfile objects.
- Accepts current encrypted magic/format.
- Accepts documented legacy aliases.
- Applies NFKD password normalization.
- Uses PBKDF2-HMAC-SHA512 with 200000 default iterations for generated files.
- Splits derived bytes into 32-byte encKey and 32-byte macKey.
- Verifies HMAC-SHA512 over salt || nonce || ciphertext before decrypting.
- Decrypts with the same HMAC-SHA512 stream transform.
- Requires recovered key material to be at least 32 bytes before building
  the arc-keyfile-v1 secret string.
```

Non-Goals

The v1.6.0 encryption subsystem does not:

```
- fetch network state
- use remote key escrow
- store the BIP39 wallet passphrase
- encrypt derived-output exports
- encrypt QR payloads
- protect against malware on the active machine
- protect against screen capture
- guarantee JavaScript memory erasure
- prevent offline guessing if an attacker has a protected .arc file and the
  credential has low entropy
```

The encryption subsystem is a local at-rest protection layer for exported seed files and keyfiles. It is not a complete operational-security system by itself.

Arculus Recovery v1.6.0 - Derivation Engine Internals

Scope

This document is the authoritative byte-level specification of the deterministic derivation engine implemented by the canonical Arculus_Recovery.html v1.6.0 application. It defines the complete computational pipeline from CSPRNG entropy through BIP39 mnemonic generation, BIP39 validation, Unicode normalization, PBKDF2 seed stretching, secp256k1 BIP32 derivation, SLIP-0010 Ed25519 derivation, Cardano Icarus Ed25519-BIP32 derivation, script-type resolution, per-coin address construction, Taproot key material, extended key serialization, output-object construction, and export flattening.

The root HTML file is the source of truth for v1.6.0 behavior. The Python CLI is upgraded in the current repository to expose the same v1.6.0 derivation coin set for scripted use: Bitcoin, Bitcoin Cash, Litecoin, Dogecoin, Ethereum, Solana, Stellar, Monero, Cardano, and XRP. This document marks any remaining surface differences between the canonical HTML workflow and Python automation where those differences affect byte output, export shape, or operator behavior.

This is an offline reference. Every value described here is a deterministic function of local input bytes and local CSPRNG output where generation is requested. The engine never queries a blockchain, wallet server, balance API, token contract, exchange, custodian, or chain-state oracle.

Architectural Invariants

For fixed non-random inputs, derivation is deterministic. Given identical:

```
mnemonic
BIP39 passphrase
coin selector
derivation path
script selector
address count or range
Bitcoin Cash legacy-address toggle
```

the canonical HTML engine produces byte-identical output on every conforming browser implementation.

The following are intentionally outside the derivation engine:

- address discovery
- balance lookup
- UTXO scanning
- wallet gap-limit inference
- Ethereum account-state inspection
- ERC-20 token enumeration
- XRP destination-tag discovery
- Cardano staking-state lookup
- Monero subaddress scanning
- Solana token-account enumeration
- fee estimation
- transaction signing
- transaction broadcast
- any network I/O

The high-level pipeline is:

```
input mnemonic or generated entropy
     |
     | BIP39 word index mapping and checksum verification
     v
normalized mnemonic words
     |
     | optional BIP39 passphrase, NFKD
```

```
     v
BIP39 seed or Cardano entropy input
     |
     +--> secp256k1 BIP32 path families
     |
     +--> SLIP-0010 Ed25519 path families
     |
     +--> Cardano Icarus Ed25519-BIP32 path family
     |
     v
per-coin account node and address rows
     |
     v
JSON object, rendered table, CSV, TXT, or PDF export
```

Input Surfaces

The HTML app accepts mnemonic material through four user-facing paths:

- Manual textarea input.
- Numbered 24-cell word grid with 12/24-word selector.
- Generate Random Seed.
- Import Seed from .arc file.

All four paths feed the same downstream validator. A generated or imported seed is held in importedSeedState and displayed as masked bullet groups until the user holds Show Seed. Since v1.6.0, a manually entered seed is promoted into the same hidden-seed state after a successful derivation, so subsequent derivations, exports, and reveal behavior use the same state model as generated and imported seeds.

The canonical normalized word sequence is:

```
words = raw_input
    .trim()
    .toLowerCase()
    .split(/\s+/)
    .filter(Boolean)

mnemonic = words.join(" ")
```

The implementation accepts only 12-word and 24-word English BIP39 mnemonics. No 15, 18, or 21 word BIP39 mnemonics are supported in v1.6.0.

CSPRNG-Based Mnemonic Generation

Generate Random Seed accepts word_count in {12, 24}. It derives entropy length from BIP39:

```
checksum_bits = floor(word_count / 3)
entropy_bits  = word_count * 11 - checksum_bits
entropy_bytes = entropy_bits / 8
```

For 12 words:

```
entropy_bits  = 128
checksum_bits = 4
entropy_bytes = 16
```

For 24 words:

```
entropy_bits  = 256
checksum_bits = 8
entropy_bytes = 32
```

The browser entropy source is:

```
crypto.getRandomValues(new Uint8Array(entropy_bytes))
```

The checksum is the most significant checksum_bits bits of SHA-256(entropy). The generated bit string is:

```
entropy_bits || checksum_bits
```

It is split into word_count groups of 11 bits. Each unsigned 11-bit value is an index into the embedded BIP39 English word list:

```
word_i = BIP39_WORDS[index_i]
```

The generated mnemonic is immediately passed through checkMnemonic. A generated mnemonic that does not validate is rejected and never used.

Word Grid Behavior

The UI constructs 24 word cells. The seed-length radio selector chooses the active span:

```
12-word mode: cells 0..11 active, cells 12..23 inactive and hidden
24-word mode: cells 0..23 active
```

Pasting a full mnemonic into the textarea or grid is normalized through the same word tokenizer. If more than 12 words are detected, the selector is set to 24. If at least one word and no more than 12 words are detected, the selector is set to 12.

The grid and textarea are synchronized through guarded event handlers so the same canonical mnemonic reaches validation regardless of which visible control the user used. When a hidden-seed state is active and the seed is not being revealed, the controls show bullet placeholders and the raw words are not visible in those input controls.

BIP39 Word Mapping and Checksum

The engine embeds the canonical 2048-entry BIP39 English word list. Each word maps to one 11-bit integer:

```
index in [0, 2047]
binary form is exactly 11 bits, big-endian within the concatenated bitstream
```

Validation procedure:

```
words = normalizeMnemonicWords(input)
wc = words.length
require wc == 12 or wc == 24
require every word exists in BIP39_INDEX

acc = 0
for word in words:
    acc = (acc << 11) | BIP39_INDEX[word]

checksum_bits = floor(wc / 3)
entropy_bits  = wc * 11 - checksum_bits

entropy_int = acc >> checksum_bits
checksum_int = acc & ((1 << checksum_bits) - 1)
entropy = uint_be(entropy_int, entropy_bits / 8)

expected_checksum = int_be(SHA-256(entropy)) >> (256 - checksum_bits)
```

The checksum passes when:

```
checksum_int == expected_checksum
```

Word count table:

```
word_count   total_bits   entropy_bits   checksum_bits   entropy_bytes
----------   ----------   ------------   -------------   -------------
12           132          128            4               16
24           264          256            8               32
```

If every word is in the BIP39 list but checksum fails, the engine reports the input as BIP39-like with invalid checksum. It does not derive keys from that input.

BIP39 Seed Stretching

For Bitcoin, Bitcoin Cash, Litecoin, Dogecoin, Ethereum, XRP, Solana, Stellar, and Monero, the BIP39 seed is:

```
m = NFKD((mnemonic || "").trim().split(/\s+/).filter(Boolean).join(" "))
p = NFKD(passphrase || "")

seed = PBKDF2-HMAC-SHA512(
    password   = UTF-8(m),
    salt       = UTF-8("mnemonic" + p),
    iterations = 2048,
    dkLen      = 64
)
```

The output is exactly 64 bytes. An empty passphrase is a valid passphrase and produces a different seed than any non-empty passphrase. The derivation engine cannot know whether the correct passphrase was used. Verification must be done by root fingerprint or known addresses.

Cardano is different in v1.6.0: the canonical HTML uses the BIP39 entropy bytes for the Icarus master-key derivation rather than the 64-byte BIP39 PBKDF2 seed. Cardano behavior is specified in its own section.

Root Fingerprint

The validation panel computes a secp256k1 BIP32 root fingerprint even when the selected coin uses a non-secp256k1 derivation family. This fingerprint is a BIP39 mnemonic/passphrase verification value, not a statement that all coins use the secp256k1 tree.

Procedure:

```
seed = BIP39 seed from mnemonic and passphrase
I = HMAC-SHA512(key = UTF-8("Bitcoin seed"), data = seed)
root_private = int_be(I[0:32])
root_chain_code = I[32:64]
root_pub = compressed_pubkey(root_private * secp256k1_G)
fingerprint = HASH160(root_pub)[0:4]
```

It is displayed as 8 lowercase hex characters.

Path Parser

Path grammar:

```
path = ["m"] [ "/" component ]*
component = decimal_digits [ "'" | "h" | "H" ]
```

The parser behavior is:

```
parts = path.trim().split("/")
if parts[0] == "m":
    parsing starts at parts[1]
else:
    parsing starts at parts[0]
```

Empty components after splitting are skipped. For each non-empty component:

```
if component ends with "'", "h", or "H":
    hardened = true
    decimal_text = component without final suffix
else:
    hardened = false
    decimal_text = component
```

```
require decimal_text matches /^[0-9]+$/
n = Number(decimal_text)
require 0 <= n < 0x80000000
if hardened:
    n = n | 0x80000000
output n as uint32
```

The canonical normalizer emits:

```
"m" for an empty path
otherwise "m/" + components joined by "/"
hardened components use apostrophe notation
```

Examples:

```
m/84h/0H/0'  -> m/84'/0'/0'
44'/60'/0'   -> m/44'/60'/0'
m            -> m
```

All serialized child numbers use ser32, a 4-byte unsigned big-endian integer.

Address Count and Range

The UI has both Address Count and Range inputs. Derivation computes:

```
countRaw = max(1, Number(count_field || 5))
rangeRaw = trim(range_field)
startIdx = 0
endIdx = countRaw - 1
```

If rangeRaw matches:

```
/^(\d+)\s*-\s*(\d+)$/
```

then:

```
startIdx = max(0, parseInt(first, 10))
endIdx   = max(startIdx, parseInt(second, 10))
count    = endIdx - startIdx + 1
```

If rangeRaw is non-empty but does not match this pattern, v1.6.0 silently falls back to the count field. Address indexes are always non-negative.

secp256k1 Domain Parameters

The secp256k1 curve is:

```
y^2 = x^3 + 7 mod P
```

Field prime:

```
P = 0xfffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f
```

Group order:

```
N = 0xfffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141
```

Generator:

```
Gx = 0x79be667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798
Gy = 0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8
```

The valid private scalar domain is:

```
1 <= k < N
```

Point addition is affine-coordinate addition. If p1 is null, p2 is returned. If p2 is null, p1 is returned. If x coordinates are equal and y1 + y2 = 0 mod P, the result is the point at infinity represented by null.

Doubling slope:

```
lambda = (3 * x1^2) * inverse_mod(2 * y1, P) mod P
```

Addition slope:

```
lambda = (y2 - y1) * inverse_mod(x2 - x1, P) mod P
```

Result:

```
x3 = lambda^2 - x1 - x2 mod P
y3 = lambda * (x1 - x3) - y1 mod P
```

Scalar multiplication uses double-and-add over native BigInt arithmetic.

Compressed public key serialization:

```
prefix = 0x03 if y is odd else 0x02
serP(P) = prefix || uint256_be(x)
```

Ethereum uncompressed input to Keccak omits the usual 0x04 prefix:

```
uint256_be(x) || uint256_be(y)
```

BIP32 secp256k1 Master Node

The secp256k1 master node is derived from the 64-byte BIP39 seed:

```
I = HMAC-SHA512(key = UTF-8("Bitcoin seed"), data = seed)
IL = I[0:32]
IR = I[32:64]

k = int_be(IL)
c = IR
```

Reject if:

```
k == 0
k >= N
```

The node object contains:

```
k         private scalar
c         32-byte chain code
depth     0 for root
parentFp  00000000 for root
childNum  00000000 for root
```

BIP32 CKDpriv

For a parent node:

```
parent.k = private scalar
parent.c = chain code
parentPub = parent.k * G
```

For child index idx:

```
if idx has bit 31 set:
    data = 0x00 || ser256(parent.k) || ser32(idx)
else:
    data = serP(parentPub) || ser32(idx)
```

```
I = HMAC-SHA512(parent.c, data)
IL = I[0:32]
IR = I[32:64]
il = int_be(IL)
childK = (il + parent.k) mod N
```

Reject if:

```
il >= N
childK == 0
```

The child node is:

```
k         childK
c         IR
depth     parent.depth + 1
parentFp  HASH160(serP(parentPub))[0:4]
childNum  idx
```

The engine raises an error for invalid edge cases rather than skipping to the next index.

secp256k1 Account and Branch Model

For Bitcoin, Bitcoin Cash, Litecoin, Dogecoin, Ethereum, and XRP:

```
root = masterFromSeed(BIP39 seed)
account = derive(root, account_derivation_path)
```

For Bitcoin, Bitcoin Cash, Litecoin, and Dogecoin, rows are derived under both:

```
receiving branch: account / 0 / address_index
change branch:    account / 1 / address_index
```

For Ethereum and XRP, v1.6.0 still uses the same branch/index layout:

```
receiving branch: account / 0 / address_index
change branch:    account / 1 / address_index
```

This means Ethereum and XRP exports contain both receiving and change arrays. Although many account-wallet conventions use only branch 0, the HTML engine emits both branches for consistency with the secp256k1 row model.

Example:

```
account derivation: m/44'/60'/0'
receiving index 0: m/44'/60'/0'/0/0
change index 0:    m/44'/60'/0'/1/0
```

Coin Selector and Defaults

The UI coin selector maps display names to internal IDs:

```
Bitcoin            -> bitcoin
Bitcoin Cash       -> bitcoincash
Litecoin           -> litecoin
Dogecoin           -> dogecoin
Ethereum / ERC-20  -> ethereum
```

```
Solana             -> solana
Stellar            -> stellar
Monero             -> monero
Cardano            -> cardano
XRP                -> xrp
```

Default account derivation paths:

```
bitcoin      m/0'
bitcoincash  m/0'
litecoin     m/84'/2'/0'
dogecoin     m/44'/3'/0'
ethereum     m/44'/60'/0'
solana       m/44'/501'/0'
stellar      m/44'/148'/0'
monero       m/44'/128'/0'
cardano      m/1852'/1815'/0'/0/0
xrp          m/44'/144'/0'
```

When the coin selector changes, the HTML sets:

```
bitcoincash -> script selector P2PKH
dogecoin    -> script selector P2PKH
ethereum    -> Auto
solana      -> Auto
stellar     -> Auto
monero      -> Auto
cardano     -> Auto
xrp         -> Auto
otherwise   -> P2WPKH
```

The initial page default is Bitcoin with P2WPKH selected.

Network Parameters

Bitcoin mainnet:

```
p2pkh xprv/xpub:       0488ade4 / 0488b21e
p2wpkh-p2sh yprv/ypub: 049d7878 / 049d7cb2
p2wpkh zprv/zpub:      04b2430c / 04b24746
p2tr trprv/trpub:      0488ade4 / 0488b21e
WIF prefix:            80
P2PKH prefix:          00
P2SH prefix:           05
HRP:                   bc
networkName:           mainnet
```

Bitcoin Cash mainnet:

```
p2pkh xprv/xpub:       0488ade4 / 0488b21e
p2wpkh-p2sh:           null
p2wpkh:                null
p2tr:                  null
WIF prefix:            80
P2PKH prefix:          00
P2SH prefix:           05
CashAddr prefix:       bitcoincash
networkName:           mainnet
```

Litecoin mainnet:

```
p2pkh xprv/xpub:       019d9cfe / 019da462
p2wpkh-p2sh yprv/ypub: 01b26792 / 01b26ef6
p2wpkh zprv/zpub:      04b2430c / 04b24746
p2tr trprv/trpub:      019d9cfe / 019da462
WIF prefix:            b0
P2PKH prefix:          30
P2SH prefix:           32
HRP:                   ltc
networkName:           mainnet
```

Dogecoin mainnet:

```
p2pkh dgpv/dgub:       02fac398 / 02facafd
p2wpkh-p2sh:           02fac398 / 02facafd
p2wpkh:                02fac398 / 02facafd
p2tr:                  null
WIF prefix:            9e
```

```
P2PKH prefix:          1e
P2SH prefix:           16
HRP:                   doge
networkName:           mainnet
```

Ethereum mainnet:

```
addressFamily:         ethereum
xprv/xpub versions:    0488ade4 / 0488b21e for all script slots
WIF:                   undefined
networkName:           mainnet
```

XRP mainnet:

```
addressFamily:         xrp
xprv/xpub versions:    0488ade4 / 0488b21e for all script slots
WIF:                   undefined
networkName:           mainnet
```

Solana, Stellar, Monero, and Cardano:

```
addressFamily only
no BIP32 extended-key version bytes
networkName: mainnet
```

Script Type Resolution

Script selector values:

```
Auto          -> auto
P2PKH         -> p2pkh
P2WPKH-P2SH   -> p2wpkh-p2sh
P2WPKH        -> p2wpkh
P2TR          -> p2tr
```

For Ethereum and XRP:

```
script type is forced to addressFamily: ethereum or xrp
```

For Solana, Stellar, Monero, and Cardano:

```
script selector is ignored because these functions dispatch before
secp256k1 script handling.
```

For Bitcoin, Bitcoin Cash, Litecoin, and Dogecoin:

```
if requested == auto:
    first path component with hardening bit masked:
        86 -> p2tr
        84 -> p2wpkh
        49 -> p2wpkh-p2sh
        otherwise -> p2pkh
else:
    script type = requested
```

If the selected network object does not have parameters for the resolved script type, the engine throws "Unsupported script type". This prevents constructing addresses for unsupported combinations such as Bitcoin Cash P2WPKH in v1.6.0.

Hash and Encoding Primitives

SHA-256:

```
Browser WebCrypto SHA-256 digest, 32 bytes.
```

RIPEMD-160:

```
Pure JavaScript RIPEMD-160 implementation, 20 bytes.
```

HASH160:

```
RIPEMD-160(SHA-256(bytes))
```

Keccak-256:

```
Pre-standard Ethereum Keccak, not NIST SHA3-256.
Rate = 136 bytes.
Padding = 0x01 ... 0x80.
Output = 32 bytes.
```

Base58 alphabet:

```
123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
```

XRPL Base58 alphabet:

```
rpshnaf39wBUDNEGHJKLM4PQRST7VWXYZ2bcdeCg65jkm8oFqi1tuvAxyz
```

Bech32 alphabet:

```
qpzry9x8gf2tvdw0s3jn54khce6mua7l
```

Base32 alphabet for Stellar StrKey:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ234567
```

Base58 and Base58Check

Base58 encoding interprets the byte string as one big-endian unsigned integer:

```
x = int_be(bytes)
while x > 0:
    mod = x mod 58
    x = floor(x / 58)
    prepend alphabet[mod]
```

For each leading 0x00 byte in the input, prepend alphabet[0].

Base58Check:

```
checksum = SHA-256(SHA-256(payload))[0:4]
output = Base58(payload || checksum)
```

Used for:

- P2PKH addresses
- P2WPKH-P2SH addresses
- WIF private keys
- extended keys
- XRP classic addresses, with XRPL alphabet

WIF Private Keys

For UTXO-style secp256k1 coins with WIF defined:

```
payload = WIF_prefix || ser256(private_key) || 0x01
wif = Base58Check(payload)
```

The trailing 0x01 is the compressed-public-key flag. Every public key emitted by this engine is compressed for UTXO address construction, so the WIF compression flag is mandatory.

Ethereum and XRP do not use WIF in v1.6.0. Solana, Stellar, Monero, and Cardano also do not use WIF.

Bech32 and Bech32m

The bech32Polymod generator constants are:

```
0x3b6a57b2
0x26508e6d
0x1ea119fa
0x3d4233dd
0x2a1462b3
```

HRP expansion:

```
high bits of each HRP char
zero separator
low five bits of each HRP char
```

Checksum construction:

```
values = hrp_expand(hrp) || data || [0,0,0,0,0,0]
polymod = bech32Polymod(values) XOR constant
```

```
checksum[i] = (polymod >> (5 * (5 - i))) & 31 for i=0..5
```

Constants:

```
Bech32  = 1
Bech32m = 0x2bc830a3
```

segwitAddress(hrp, version, witnessProgram):

```
data = [version] || convertBits(witnessProgram, 8, 5, pad=true)
if version == 0:
    use Bech32
else:
    use Bech32m
```

v1.6.0 therefore uses:

```
P2WPKH version 0 -> Bech32
P2TR version 1  -> Bech32m
```

CashAddr

Bitcoin Cash default address rendering uses CashAddr unless the legacy toggle is enabled. The CashAddr alphabet is the same 32-character string as Bech32:

```
qpzry9x8gf2tvdw0s3jn54khce6mua7l
```

cashaddrPolymod uses five 40-bit generator constants:

```
0x98f2bc8e61
0x79b76d99e2
0xf33e5fb3c4
0xae2eabe2a8
0x1e4f43e470
```

Prefix expansion:

```
for each char in lowercase(prefix):
    charCode & 31
append 0
```

For a 20-byte HASH160:

```
version = (type << 3) | size_code
size_code = 0 for 20-byte hashes
```

P2PKH CashAddr:

```
type = 0
version = 0
payload = convertBits([version] || pkh, 8, 5, true)
checksumValue = cashaddrPolymod(prefix_expand || payload || eight zeros)
checksum = eight 5-bit values, most significant first
address = "bitcoincash:" || payload_chars || checksum_chars
```

P2SH CashAddr:

```
type = 1
version = 8
payload = convertBits([version] || script_hash, 8, 5, true)
```

In v1.6.0, Bitcoin Cash network configuration exposes only p2pkh as supported, so the default UI path emits P2PKH CashAddr. The generic pubToAddress function also contains the P2SH CashAddr branch for p2wpkh-p2sh if a network ever enables that script slot.

If the legacy toggle is enabled, Bitcoin Cash P2PKH uses:

```
Base58Check(0x00 || HASH160(pubkey))
```

UTXO Address Construction

For Bitcoin, Bitcoin Cash, Litecoin, and Dogecoin:

```
child_pub = serP(child.k * G)
pkh = HASH160(child_pub)
```

P2PKH:

```
if Bitcoin Cash and legacy toggle is off:
    address = CashAddr("bitcoincash", type=0, pkh)
else:
    address = Base58Check(p2pkh_prefix || pkh)
```

P2WPKH-P2SH:

```
redeem = 0x00 || 0x14 || pkh
script_hash = HASH160(redeem)
if CashAddr prefix exists and legacy toggle is off:
    address = CashAddr(prefix, type=1, script_hash)
else:
    address = Base58Check(p2sh_prefix || script_hash)
```

P2WPKH:

```
address = Bech32(hrp, [0] || convertBits(pkh, 8, 5, true))
```

P2TR:

```
handled through Taproot key material below.
```

Every UTXO address row contains:

```
path
address
public_key_hex
private_key_hex
private_key_wif
```

Taproot Key Material

Taproot is applied when script type is p2tr and the selected network has p2tr extended-key version parameters.

Input:

```
child private scalar d
```

Step 1, internal key parity normalization:

```
P = d * G
if y(P) is odd:
    internalPrivateKey = N - d mod N
    internalPublicKeyPoint = -P
else:
    internalPrivateKey = d
    internalPublicKeyPoint = P

internalPublicKey = uint256_be(x(internalPublicKeyPoint))
```

Step 2, BIP340/BIP341 tagged hash:

```
tagHash = SHA-256(UTF-8("TapTweak"))
tweakBytes = SHA-256(tagHash || tagHash || internalPublicKey)
tweak = int_be(tweakBytes)
```

Reject if:

```
tweak >= N
```

Step 3, output key:

```
outputPubPoint = internalPublicKeyPoint + tweak * G
outputPrivateKey = (internalPrivateKey + tweak) mod N
```

Reject if:

```
outputPubPoint is null
outputPrivateKey == 0
```

Step 4, address:

```
outputPublicKey = uint256_be(x(outputPubPoint))
outputParity = y(outputPubPoint) & 1
address = segwitAddress(hrp, 1, outputPublicKey)
```

Taproot row additions:

```
taproot_internal_public_key_hex
taproot_internal_private_key_hex
```

```
taproot_internal_private_key_wif
taproot_tweak_hex
taproot_output_public_key_hex
taproot_output_private_key_hex
taproot_output_private_key_wif
taproot_output_key_parity
```

Both internal and output private keys are funds-controlling secrets.

Extended Key Serialization

Extended private key payload:

```
offset  length  field
------  ------  -----
0       4       uint32_be(version)
4       1       depth
5       4       parent fingerprint
9       4       child number, ser32
13      32      chain code
45      1       0x00
46      32      private scalar ser256
```

Extended public key payload:

```
offset  length  field
------  ------  -----
0       4       uint32_be(version)
4       1       depth
5       4       parent fingerprint
9       4       child number, ser32
13      32      chain code
45      33      compressed public key
```

Both payloads are 78 bytes before Base58Check checksum.

The HTML emits account/root key fields by script family:

```
always:
    root_xprv, root_xpub, account_xprv, account_xpub

p2wpkh-p2sh:
    root_yprv, root_ypub, account_yprv, account_ypub

p2wpkh:
    root_zprv, root_zpub, account_zprv, account_zpub

p2tr:
    root_trprv, root_trpub, account_trprv, account_trpub
```

Ethereum and XRP use standard xprv/xpub version bytes in the HTML network object for every script slot, but their script type is forced to their address family.

Ethereum

Ethereum uses secp256k1 BIP32 root/account/branch/index derivation.

For each child:

```
pub = child.k * G
uncompressed_no_prefix = uint256_be(pub.x) || uint256_be(pub.y)
hash = Keccak-256(uncompressed_no_prefix)
address_bytes = hash[12:32]
address_hex = lowercase hex(address_bytes)
```

EIP-55 checksum:

```
hash_hex = hex(Keccak-256(UTF-8(address_hex)))
checked = ""
for i in 0..39:
    if int(hash_hex[i], 16) >= 8:
        checked += uppercase(address_hex[i])
    else:
        checked += address_hex[i]

address = "0x" || checked
```

Ethereum row additions:

```
  ethereum_public_key_hex = uint256_be(x) || uint256_be(y)
  erc20_address = address
  erc20_note = "ERC-20 tokens on Ethereum use this same account address."
```

private_key_hex is the 32-byte secp256k1 private scalar. No WIF is emitted.

XRP

XRP uses secp256k1 BIP32 root/account/branch/index derivation.

For each child:

```
  pub = compressed public key, 33 bytes
  accountId = HASH160(pub)
  payload = 0x00 || accountId
  address = Base58Check(payload, XRP_B58)
```

XRP row additions:

```
  xrp_classic_address = address
  xrp_note = "XRP Ledger classic address. Destination tags, when required by an exchange or
custodian, are not derived from the seed."
```

Destination tags are never derived. They are off-chain or custodian-assigned routing metadata.

Ed25519 Curve and Encoding

Solana, Stellar, and Monero use Ed25519 public-key arithmetic. The HTML engine implements the twisted Edwards curve:

```
  p = 2^255 - 19
  d = -121665 / 121666 mod p
```

Base point:

```
  x = 15112221349535400772501151409588531511454012693041857206046113283949847762202
  y = 46316835694926478169428394003475163141307993866256225615783033603165251855960
```

Note: the implementation stores these as decimal BigInt constants.

The group order used for scalar reduction is:

```
  L = 2^252 + 27742317777372353535851937790883648493
```

Point addition uses the Edwards formulas:

```
  xy = d * x1 * x2 * y1 * y2 mod p
  x3 = (x1*y2 + y1*x2) * inverse_mod(1 + xy, p) mod p
  y3 = (y1*y2 + x1*x2) * inverse_mod(1 - xy, p) mod p
```

Ed25519 public key from a 32-byte seed:

```
  h = SHA-512(seed)
  scalarBytes = h[0:32]
  scalarBytes[0] &= 248
  scalarBytes[31] &= 127
  scalarBytes[31] |= 64
  point = scalarBytes_le * G
  encoded = little_endian_y(point), with x parity in top bit of byte 31
```

Ed25519 public key from scalar bytes:

```
  point = little_endian_scalar(scalarBytes) * G
  encoded = little_endian_y(point), with x parity in top bit of byte 31
```

The distinction matters:

```
  - Solana and Stellar public keys use ed25519PublicKeyFromSeed on SLIP-0010 k.
  - Monero spend/view public keys use ed25519PublicKeyFromScalarBytes after
    scalar reduction.
  - Cardano public keys use ed25519PublicKeyFromScalarBytes(k[0:32]).
```

SLIP-0010 Ed25519 Master and Child Derivation

Solana, Stellar, and Monero use the SLIP-0010 Ed25519 tree.

Master:

```
I = HMAC-SHA512(key = UTF-8("ed25519 seed"), data = BIP39 seed)
k = I[0:32]
c = I[32:64]
```

Child:

```
require index has HARDENED bit set
data = 0x00 || parent.k || ser32(index)
I = HMAC-SHA512(parent.c, data)
child.k = I[0:32]
child.c = I[32:64]
```

Unhardened Ed25519 derivation throws:

```
"Ed25519 derivation requires hardened path indexes."
```

Solana

Default path:

```
m/44'/501'/0'
```

Path-per-index rule:

```
ints = parsePath(basePath)
if ints.length >= 4
   and purpose == 44
   and coin type == 501
   and last component masked == 0:
       replace component 2 with index'
       path = pathFromIndexes(next)
else if index == 0:
       path = normalizePath(basePath)
else:
       path = normalizePath(basePath) + "/" + index + "'"
```

This rule is intentionally implementation-specific. For default m/44'/501'/0', index 0 resolves to m/44'/501'/0'. Index 1 resolves to m/44'/501'/1'.

Account object fields:

```
derivation
account_script_type_used = solana
root_private_key_hex
root_chain_code_hex
root_public_key_hex
account_private_key_hex
account_chain_code_hex
account_public_key_hex
receiving
change = []
```

Row fields:

```
branch = account
path
address = Base58(public_key)
public_key_hex
private_key_hex
solana_note
```

Stellar

Default path:

```
m/44'/148'/0'
```

Path-per-index rule:

```
if base path has purpose 44' and coin type 148':
    replace component 2 with index'
else if index == 0:
    use normalized base path
else:
    append "/" + index + "'"
```

StrKey construction:

```
body = versionByte || payload
checksum = CRC16-XModem(body), emitted little-endian
strkey = Base32(body || checksum), no padding
```

Version bytes:

```
public address: 6 << 3  = 0x30
secret seed:    18 << 3 = 0x90
```

CRC16-XModem:

```
initial crc = 0
polynomial = 0x1021
for each byte:
    crc ^= byte << 8
    repeat 8:
        if crc & 0x8000:
            crc = (crc << 1) ^ 0x1021
        else:
            crc = crc << 1
        crc &= 0xffff
```

Row fields:

```
branch = account
path
address = stellarPublicAddress(public_key)
public_key_hex
private_key_hex
stellar_secret_seed
stellar_note
```

Monero

Default path:

```
m/44'/128'/0'
```

Path-per-index rule matches Stellar, with coin type 128.

Private spend key:

```
privateSpendKey = reduceEd25519Scalar(SLIP-0010 child.k)
```

Scalar reduction:

```
scalar = little_endian_int(bytes) mod L
if scalar == 0:
    scalar = 1
output = uint256_le(scalar)
```

Private view key:

```
privateViewKey = reduceEd25519Scalar(Keccak-256(privateSpendKey))
```

Public keys:

```
publicSpendKey = ed25519PublicKeyFromScalarBytes(privateSpendKey)
publicViewKey  = ed25519PublicKeyFromScalarBytes(privateViewKey)
```

Standard mainnet address payload:

```
body = 0x12 || publicSpendKey || publicViewKey
checksum = Keccak-256(body)[0:4]
address = MoneroBase58(body || checksum)
```

Monero Base58 chunk encoding:

```
process bytes in chunks of up to 8 bytes
interpret each chunk as a big-endian integer
encode with Bitcoin Base58 alphabet
left-pad each encoded chunk to fixed size:

  raw bytes  encoded chars
  ---------  -------------
  1          2
  2          3
  3          5
```

```
4          6
5          7
6          9
7          10
8          11
```

Account object includes both root and first-account Monero material:

```
root_private_spend_key_hex
root_private_view_key_hex
root_public_spend_key_hex
root_public_view_key_hex
account_private_spend_key_hex
account_private_view_key_hex
account_public_spend_key_hex
account_public_view_key_hex
```

Row fields:

```
branch = account
path
address
public_spend_key_hex
public_view_key_hex
private_spend_key_hex
private_view_key_hex
monero_note
```

Cardano Icarus Ed25519-BIP32

Cardano in v1.6.0 does not use the 64-byte BIP39 seed. It reconstructs the BIP39 entropy bytes from the mnemonic word indexes:

```
entropy = mnemonicEntropyBytes(mnemonic)
```

Icarus master:

```
keyBytes = PBKDF2-HMAC-SHA512(
    password   = empty byte string,
    salt       = entropy,
    iterations = 4096,
    dkLen      = 96
)
```

Clamp:

```
keyBytes[0]  &= 0xf8
keyBytes[31] &= 0x1f
keyBytes[31] |= 0x40
```

Root node:

```
k = keyBytes[0:64]
c = keyBytes[64:96]
```

Cardano private key k is 64 bytes:

```
left  half = k[0:32]
right half = k[32:64]
```

Cardano public key:

```
ed25519PublicKeyFromScalarBytes(k[0:32])
```

Cardano child derivation uses little-endian child indexes:

```
idxBytes = uint32_le(index)
pub = cardanoPublicKeyFromPrivate(parent.k)
```

If index hardened:

```
z       = HMAC-SHA512(parent.c, 0x00 || parent.k || idxBytes)
cDigest = HMAC-SHA512(parent.c, 0x01 || parent.k || idxBytes)
```

If index unhardened:

```
z       = HMAC-SHA512(parent.c, 0x02 || pub || idxBytes)
cDigest = HMAC-SHA512(parent.c, 0x03 || pub || idxBytes)
```

Then:

```
kl = little_endian_int(parent.k[0:32])
kr = little_endian_int(parent.k[32:64])
zl = little_endian_int(z[0:28])
zr = little_endian_int(z[32:64])

childLeft = kl + 8 * zl
reject if childLeft mod ED25519_L == 0

childRight = (kr + zr) mod 2^256

child.k = uint256_le(childLeft) || uint256_le(childRight)
child.c = cDigest[32:64]
```

Default path:

```
m/1852'/1815'/0'/0/0
```

Path selection per address index:

```
ints = parsePath(basePath)
account = HARDENED
if ints.length >= 3 and purpose == 1852 and coin type == 1815:
    account = ints[2]

paymentPath = m/1852'/1815'/account/0/index
stakingPath = m/1852'/1815'/account/2/0
```

If the default base path is used, account is 0'. The final /0/0 in the base path is not used directly as an index template; v1.6.0 reconstructs payment and staking paths from purpose, coin type, and account.

BLAKE2b-224:

```
output length = 28 bytes
IV = standard BLAKE2b IV
parameter block sets digest length 28
compression rounds = 12
little-endian 64-bit words
```

Shelley base address:

```
paymentHash = blake2b224(paymentPub)
stakeHash = blake2b224(stakePub)
payload = 0x01 || paymentHash || stakeHash
address = Bech32("addr", convertBits(payload, 8, 5, true))
```

Cardano row fields:

```
branch = receiving
path = paymentPath
staking_path
address
public_key_hex
stake_public_key_hex
private_key_hex
stake_private_key_hex
cardano_note
```

The 0x01 header indicates Shelley base address, mainnet network id 1, payment credential by key hash, and stake credential by key hash.

Output Object: secp256k1 Families

Top-level result:

```
{
  coin: internal coin id,
  network: "mainnet",
  word_count: 12 or 24,
  accounts: [ account ]
}
```

secp256k1 account object:

```
{
  derivation,
  account_script_type_used,
  root_xprv,
```

```
  root_xpub,
  optional root_yprv/root_ypub,
  optional root_zprv/root_zpub,
  optional root_trprv/root_trpub,
  account_xprv,
  account_xpub,
  optional account_yprv/account_ypub,
  optional account_zprv/account_zpub,
  optional account_trprv/account_trpub,
  receiving: [],
  change: []
}
```

Common secp256k1 row:

```
{
  path,
  address,
  public_key_hex,
  private_key_hex
}
```

UTXO rows add:

```
private_key_wif
```

Ethereum rows add:

```
ethereum_public_key_hex
erc20_address
erc20_note
```

XRP rows add:

```
xrp_classic_address
xrp_note
```

Taproot rows add the Taproot fields listed earlier.

Output Object: Ed25519 and Cardano Families

Solana and Stellar account objects:

```
derivation
account_script_type_used
root_private_key_hex
root_chain_code_hex
root_public_key_hex
account_private_key_hex
account_chain_code_hex
account_public_key_hex
receiving
change = []
```

Monero account object:

```
derivation
account_script_type_used
root_private_key_hex
root_chain_code_hex
account_private_key_hex
account_chain_code_hex
account_private_spend_key_hex
account_private_view_key_hex
account_public_spend_key_hex
account_public_view_key_hex
root_private_spend_key_hex
root_private_view_key_hex
root_public_spend_key_hex
root_public_view_key_hex
receiving
change = []
```

Cardano account object:

```
derivation
account_script_type_used
root_private_key_hex
root_chain_code_hex
```

```
root_public_key_hex
account_private_key_hex
account_chain_code_hex
account_public_key_hex
receiving
change = []
```

These families do not emit BIP32 xprv/xpub strings in v1.6.0.

Flattened Table and CSV Row Model

flattenDerivedRows(data) iterates every account, extracts account-level fields, then overlays each row from account.receiving and account.change.

Before copying account-level fields, the table flattener excludes:

```
all keys in AK_KEY_ORDER
network
word_count
derivation
account_script_type_used
coin
receiving
change
```

For each branch:

```
for branchName in ["receiving", "change"]:
    for item in account[branchName]:
        row = { branch: branchName, ...accountFields, ...item }
```

Important consequence: if an item has its own branch field, as Solana, Stellar, Monero, and Cardano rows do, the item branch overwrites the loop branch. Thus Solana/Stellar/Monero rows show branch "account", and Cardano rows show branch "receiving".

CSV header order is insertion order of first appearance across rows. CSV fields are escaped by:

```
text = value == null ? "" : String(value)
if text contains comma, quote, CR, or LF:
    output quote + text.replace(/"/g, '""') + quote
else:
    output text
```

CSV rows are joined with LF and the file ends with LF.

TXT Export Field Model

TXT export emits:

```
Arculus Derived Keys and Addresses

coin: <coin>
network: <network>
word count: <word_count>
```

Then for each account:

```
Account N
account scalar fields, excluding receiving and change
```

Then for each branch:

```
Receiving Addresses
Change Addresses
```

Row labels are lower-case field keys with underscores replaced by spaces. Null and undefined values are skipped.

PDF Export Field Model

PDF export first flattens derived rows. It uses four visible address-table columns:

```
Branch
Path
Address
Private key column
```

The private key column is selected by coin:

```
Monero:
    private_spend_key_hex, label "Private Spend Key Hex"

Ethereum, Solana, Stellar, Cardano, XRP:
    private_key_hex, label "Private Key Hex"

All other coins:
    private_key_wif, label "Private Key WIF"
```

Extended keys are printed above the table when present in the account object. Ed25519/Cardano families do not have xprv/xpub fields, so their PDF exports skip that extended-key section unless future fields are added.

Failure Modes

The derivation engine produces no partial successful result when a hard failure is encountered.

Input failures:

- word count not 12 or 24
- unknown BIP39 word
- invalid BIP39 checksum
- unsupported coin id
- malformed derivation path component
- derivation index >= 0x80000000 before hardening

secp256k1 failures:

- invalid BIP32 master key
- invalid BIP32 child key
- unsupported script type for selected network
- WIF requested for a coin without WIF
- Taproot tweak >= N
- Taproot output public key at infinity
- Taproot output private key == 0

SLIP-0010 failures:

- unhardened Ed25519 derivation component

Cardano failures:

- childLeft mod ED25519_L == 0

QR, export, and .arc errors are specified in their own documents.

Practical Recovery Procedure

When searching for a wallet:

- Validate the mnemonic checksum.
- Enter the exact BIP39 passphrase if one was used.
- Confirm the root fingerprint where available.
- Select the original coin.
- Match the original wallet's account path.
- Match the original wallet's script type.
- Derive index 0 and compare the first known receive address.

- If no match, check common purpose paths and both receiving/change branches.
- Increase count or use Range for large address gaps.
- Export only after path and passphrase have been confirmed.

For Bitcoin/Bitcoin Cash/Litecoin/Dogecoin:

```
check m/44', m/49', m/84', and m/86' where supported
```

For Ethereum:

```
start with m/44'/60'/0'/0/0 and branch 0
```

For XRP:

```
destination tags are not derivation data; obtain them from the custodian
```

For Solana/Stellar/Monero:

```
account index changes are hardened account path changes in v1.6.0
```

For Cardano:

```
payment and staking paths are reconstructed from the account component
```

Compatibility Reference

Canonical HTML v1.6.0:

```
Full behavior described by this document.
```

Python CLI in the current repository:

```
Reports APP_VERSION "1.6.0".
Supports the v1.6.0 derivation coin set:

  bitcoin
  bitcoincash
  litecoin
  dogecoin
  ethereum
  solana
  stellar
  monero
  cardano
  xrp

Supports JSON/CSV/TXT scripted output through --output-format.
Uses --script-type auto by default so Bitcoin Cash selects P2PKH/CashAddr
instead of an unsupported SegWit script family.
Emits Bitcoin Cash CashAddr strings by default.
Emits Solana raw Ed25519 public keys encoded with Bitcoin Base58.
Emits Stellar StrKey public addresses and stellar_secret_seed values.
Emits Monero standard mainnet addresses plus private/public spend and view
key fields.
Emits Cardano Shelley base addresses from payment and staking keys.
Provides Python helper code for derived PDF and QR PNG generation, but the
browser UI remains the canonical interactive export workflow.
```

PySide6 GUI:

```
Renders packaged HTML. Its behavior follows the packaged asset that was copied
into src/arculus_recovery/assets.
```

Tauri wrapper:

```
Renders packaged HTML and provides native save bridge. Derivation behavior is
still the HTML behavior.
```

Implementation Function Map

Canonical HTML functions:

```
generateRandomMnemonic(wordCount)
analyzeMnemonic(mnemonic, passphrase)
checkMnemonic(mnemonic, passphrase)
bip39Seed(mnemonic, passphrase)
mnemonicEntropyBytes(mnemonic)
```

```
masterFromSeed(seed)
ckdPriv(node, index)
derive(node, path)
masterFromSeedEd25519(seed)
ckdPrivEd25519(node, index)
deriveEd25519(node, path)
cardanoIcarusMasterFromEntropy(entropyBytes)
ckdPrivCardano(node, index)
deriveCardano(node, path)
deriveSolanaOutput(seed, derivation, count, startIdx, wordCount, networkName)
deriveStellarOutput(seed, derivation, count, startIdx, wordCount, networkName)
deriveMoneroOutput(seed, derivation, count, startIdx, wordCount, networkName)
deriveCardanoOutput(mnemonic, derivation, count, startIdx, wordCount, networkName)
deriveOutput()
pubToAddress(pub, scriptType, net, useLegacy)
taprootKeyData(privKey)
xprv(node, version)
xpub(node, version)
toWif(k, net)
flattenDerivedRows(data)
derivedToCsv(data)
derivedToText(data)
```

Python compatibility functions are now useful for cross-checking both the secp256k1 subset and the expanded v1.6.0 Ed25519-family coin set. Conformance for either implementation must still be measured against the byte-level rules in this document and the vectors in TestVectors.txt; if Python and HTML disagree, the canonical HTML behavior and documented test vectors control until the Python code is corrected.

Security Classification of Derived Fields

Critical secret fields:

```
root_xprv
account_xprv
root_yprv
account_yprv
root_zprv
account_zprv
root_trprv
account_trprv
private_key_hex
private_key_wif
taproot_internal_private_key_hex
taproot_internal_private_key_wif
taproot_output_private_key_hex
taproot_output_private_key_wif
stellar_secret_seed
private_spend_key_hex
private_view_key_hex
account_private_spend_key_hex
account_private_view_key_hex
root_private_spend_key_hex
root_private_view_key_hex
stake_private_key_hex
```

Public but linkable fields:

```
addresses
xpub/ypub/zpub/trpub
public keys
root fingerprint
derivation paths
```

Operationally, any export containing critical secret fields must be handled as funds-controlling key material.

Arculus Recovery v1.6.0 - QR Payload and Encoder Specification

Scope

This document specifies the QR Export subsystem implemented inside Arculus_Recovery.html v1.6.0. It covers the modal workflow, address normalization, URI payload construction, XRP destination-tag handling, QR byte encoding, version/capacity selection, Reed-Solomon generation, matrix construction, masking, rendering, PNG export, clipboard behavior, and security boundary.

The QR subsystem is self-contained. It does not call a CDN, external QR library, wallet service, blockchain node, or network API.

Public Entry Points

The embedded QR module is:

```
const QRGen = (() => { ... return { generate, render }; })();
```

Public methods:

```
QRGen.generate(text)
    Converts a JavaScript string into a QR matrix.

QRGen.render(matrix, canvas, opts)
    Draws a QR matrix onto an HTML canvas.
```

UI functions:

```
openQrModal()
bindQrUI()
isXrpAddress(a)
toBip21Uri(address, memo, coinContext)
renderXrpQr(address, memo)
renderQrFromAddress(address, memo, coinContext)
resetQrOutput()
```

File export:

```
Save PNG uses canvas.toBlob(..., "image/png") and saveBlob.
```

QR Modal Workflow

Open:

```
QR Export button calls openQrModal().
```

openQrModal behavior:

- resetQrOutput()
- qrAddressInput.value = ""
- qrAddressInput.dataset.coin = ""
- qrMemoInput.value = ""
- show modal with showModal() if available
- otherwise set open attribute
- focus address input after 50 ms

Close:

- Close button closes the dialog.
- Clicking the backdrop closes the dialog when event target is the dialog.

Generate:

- Trim address input.
- If empty, do nothing.

- resetQrOutput().
- If address matches XRP classic syntax, show destination-tag prompt.
- Otherwise render immediately.

XRP prompt:

- Confirm reads trimmed memo/tag and renders.
- Skip renders with empty memo/tag.
- Enter in memo field triggers Confirm.

After rendering:

- canvas is shown
- label is shown
- Save PNG, Copy Address, and Edit controls are shown

Edit:

- resetQrOutput()
- if XRP address, show memo prompt again
- otherwise focus address input

Input Normalization

Address input:

```
a = address.trim()
```

Memo/tag input:

```
m = (memo || "").trim()
```

Empty address:

```
Generate returns without rendering.
```

No chain validation:

```
The QR subsystem does not validate balances, accounts, destination-tag
requirements, network, token contract, or exchange deposit policy.
```

XRP Address Detection

Function:

```
isXrpAddress(a)
```

Regex:

```
/^r[1-9A-HJ-NP-Za-km-z]{24,}/
```

Input:

```
a.trim()
```

Consequences:

```
- Address must begin with "r".
- The following characters must be from the XRP Base58-like alphabet subset
  in the regex.
- Minimum regex length is "r" plus at least 24 following characters.
- The regex is syntactic and does not validate checksum.
```

When true, the UI uses the XRP-specific memo prompt and renderXrpQr path.

URI Payload Construction

Function:

```
toBip21Uri(address, memo, coinContext)
```

Inputs:

```
address:
    User-entered address string.

memo:
    Optional memo/destination tag string.

coinContext:
    Optional internal coin id.
```

Processing:

```
a = address.trim()
m = (memo || "").trim()
```

Rule order is normative. First match wins:

- Existing scheme:

  ```
  if /^[a-z]+:/i.test(a):
      return a
  ```

- Explicit Solana context:

  ```
  if coinContext == "solana":
      return "solana:" + a
  ```

- Explicit Stellar context:

  ```
  if coinContext == "stellar":
      return "web+stellar:pay?destination=" + encodeURIComponent(a)
  ```

- Explicit Monero context:

  ```
  if coinContext == "monero":
      return "monero:" + a
  ```

- Explicit Cardano context:

  ```
  if coinContext == "cardano":
      return a
  ```

- Bitcoin syntax:

  ```
  if /^(bc1|[13])[a-zA-HJ-NP-Z0-9]{25,}/.test(a):
      return "bitcoin:" + a
  ```

- Bitcoin Cash prefixed CashAddr:

  ```
  if /^bitcoincash:[qp][a-z0-9]{20,}/i.test(a):
      return a
  ```

- Bitcoin Cash prefixless CashAddr:

  ```
  if /^[qp][a-z0-9]{20,}/i.test(a):
      return "bitcoincash:" + a
  ```

- Litecoin syntax:

  ```
  if /^(ltc1|[LM])[a-zA-HJ-NP-Z0-9]{25,}/.test(a):
      return "litecoin:" + a
  ```

- Dogecoin syntax:

  ```
  if /^[DA9][a-zA-HJ-NP-Z0-9]{25,}/.test(a):
      return "dogecoin:" + a
  ```

- Ethereum syntax:

  ```
  if /^0x[0-9a-fA-F]{40}$/.test(a):
      return "ethereum:" + a
  ```

- XRP syntax:

  ```
  if isXrpAddress(a):
      return a + "?dt=" + (m ? encodeURIComponent(m) : "00000")
  ```

- Fallback:

  ```
  return a
  ```

Cardano note:

```
Cardano is returned raw only when coinContext is "cardano". Without explicit
context, a Cardano address falls through to raw payload.
```

XRP Payload Rule

XRP uses renderXrpQr rather than the generic non-XRP render path.

Payload:

```
uri = address.trim() + "?dt=" + tag_component
```

tag_component:

```
if trimmed memo/tag is non-empty:
    encodeURIComponent(m)
else:
    "00000"
```

XRP payload intentionally does not include an "xrp:" URI scheme in v1.6.0.

Label shown under QR:

```
The full XRP payload including "?dt=...".
```

Copy Address button:

```
Copies only qrAddressInput.value.trim(), not the label payload. For XRP this
means the destination-tag component is not copied by Copy Address.
```

Text Encoding and QR Mode

QRGen.generate(text):

```
enc = new TextEncoder().encode(text)
byteLen = enc.length
```

Mode:

```
Byte mode only.
```

Mode indicator:

```
0100
```

Character count length:

```
8 bits for versions 1 through 9
16 bits for version 10
```

Bitstream before padding:

```
mode_4_bits ||
byte_length_count_indicator ||
UTF-8 payload bytes as 8-bit values
```

Terminator:

```
Four zero bits are pushed by buildCodewords.
```

Byte alignment:

```
Zero bits are appended until bit length is divisible by 8.
```

Pad bytes:

```
0xEC, 0x11 alternating until data capacity is filled.
```

Capacity Selection

Supported versions:

```
1 through 10
```

Supported error-correction levels:

```
ecLevel 0 = M
ecLevel 1 = Q
```

Important v1.6.0 behavior:

```
The implementation tests level M first, then level Q, for each version.
```

Algorithm:

```
ver = 1
ecLevel = 0

for v in 1..10:
    lenBitsV = v < 10 ? 8 : 16
    totalBits = 4 + lenBitsV + byteLen * 8 + 4
    reqBytes = ceil(totalBits / 8)

    if CAP[v][0].db >= reqBytes:
        ver = v
        ecLevel = 0
        break

    if CAP[v][1].db >= reqBytes:
        ver = v
        ecLevel = 1
        break
```

Consequence:

```
The selected QR is the first version where M fits; if M does not fit but Q
does fit at that same version, Q is selected. The code does not continue
searching later versions for a different level after finding Q.
```

Capacity table:

```
Version   M data bytes   Q data bytes
-------   ------------   ------------
      1             16             13
      2             28             22
      3             44             32
      4             64             48
      5             86             64
      6            108             84
      7            124             93
      8            154            122
      9            182            154
     10            216            180
```

Overflow behavior:

```
The code initializes ver=1/ecLevel=0 before the loop. In normal UI use,
address payloads fit in versions 1-10. Very large payloads should be treated
as unsupported; downstream rendering is not intended as an arbitrary large
text QR generator.
```

Reed-Solomon Parameters

Capacity entries include:

```
db:
    data bytes

ec:
    ECC bytes per block

b:
    number of blocks
```

Table:

```
Version   M ec/block blocks   Q ec/block blocks
-------   -----------------   -----------------
      1          10      1          13      1
      2          16      1          22      1
      3          26      2          18      2
      4          18      2          26      2
      5          24      2          18      4
      6          16      4          24      4
      7          18      4          18      6
      8          22      4          22      6
      9          22      5          20      7
     10          26      5          24      8
```

GF(256) Arithmetic

The encoder builds EXP and LOG tables:

```
EXP: Uint8Array(512)
LOG: Uint8Array(256)
```

Primitive polynomial:

```
0x11d
```

Generator element:

```
2
```

Table construction:

```
x = 1
for i in 0..254:
    EXP[i] = x
    LOG[x] = i
    x = x << 1
    if x > 255:
        x ^= 0x11d

for i in 255..511:
    EXP[i] = EXP[i - 255]
```

Multiplication:

```
gfMul(a, b):
    if a != 0 and b != 0:
        return EXP[LOG[a] + LOG[b]]
    else:
        return 0
```

Generator Polynomial and ECC

Generator polynomial:

```
gfPoly(ec):
    p = [1]
    for i in 0..ec-1:
        q = [1, EXP[i]]
        r = zero array length p.length + 1
        for each p coefficient a:
            for each q coefficient b:
                r[a+b] ^= gfMul(p[a], q[b])
        p = Array.from(r)
    return p
```

ECC encode:

```
rsEncode(data, ec):
    gen = gfPoly(ec)
    out = data || zero(ec)

    for i in 0..data.length-1:
        c = out[i]
        if c != 0:
            for j in 0..gen.length-1:
                out[i+j] ^= gfMul(gen[j], c)

    return out.slice(data.length)
```

Block Interleaving

buildCodewords(bits, totalData, ecPerBlock, numBlocks):

- Append terminator and padding bits.
- Convert data bits into totalData bytes.
- Split data into numBlocks blocks.
- Some final blocks may be one byte longer:

  ```
  blockSize = floor(totalData / numBlocks)
  ```

```
longBlocks = totalData % numBlocks

for block index b:
    len = b < numBlocks - longBlocks ? blockSize : blockSize + 1
```

- Generate ECC for each block.
- Interleave data bytes by position across blocks.
- Interleave ECC bytes by position across blocks.

The returned codeword stream is placed into the QR matrix.

Matrix Construction

Matrix size:

```
size = version * 4 + 17
```

Cell representation:

```
null:
    unfilled module

1:
    dark function module

0:
    light function module

true/false:
    data or ECC module before/after masking
```

The distinction between numeric function modules and boolean data modules is important. applyMask only flips cells whose type is boolean.

Function patterns:

- three finder patterns
- one-module light separator around finders
- timing patterns on row 6 and column 6
- alignment patterns for versions 2-10
- dark module at (size - 8, 8)
- format-information reservation cells

Finder locations:

```
top-left
top-right
bottom-left
```

Alignment coordinates:

```
v1  []
v2  [6,18]
v3  [6,22]
v4  [6,26]
v5  [6,30]
v6  [6,34]
v7  [6,22,38]
v8  [6,24,42]
v9  [6,26,46]
v10 [6,28,50]
```

Data Placement

placeData(m, codewords):

- Start at the lower-right module.
- Move in two-column vertical stripes.

- Skip column 6.
- Alternate upward and downward direction per stripe.
- Fill only null cells.
- Write codeword bits most-significant-bit first.
- If codeword bits run out, write false for remaining data slots.

Mask Functions

All eight masks are evaluated:

```
0: (r + c) % 2 == 0
1: r % 2 == 0
2: c % 3 == 0
3: (r + c) % 3 == 0
4: (floor(r/2) + floor(c/3)) % 2 == 0
5: (r*c) % 2 + (r*c) % 3 == 0
6: ((r*c) % 2 + (r*c) % 3) % 2 == 0
7: ((r+c) % 2 + (r*c) % 3) % 2 == 0
```

applyMask:

```
out = copy of matrix
for each cell:
    if typeof out[r][c] == "boolean":
        out[r][c] = out[r][c] !== maskFunction(r,c)
```

Function modules are not masked.

Format Bits

Precomputed format tables:

```
FMT_M:
  101010000010010
  101000100100101
  101111001111100
  101101101001011
  100010111111001
  100000011001110
  100111110010111
  100101010100000

FMT_Q:
  011010101011111
  011000001101000
  011111100110001
  011101000000110
  010010010110100
  010000110000011
  010111011011010
  010101111101101
```

writeFormat(m, ecLevel, mask):

- Chooses FMT_Q when ecLevel == 1, otherwise FMT_M.
- Writes bits to top-left format track.
- Writes bits to top-right and bottom-left copy tracks.

Penalty Scoring

v1.6.0 penaltyScore implements two penalty families:

- Runs of five or more equal modules in rows and columns:

  ```
  if run >= 5:
      score += run - 2
  ```

- 2x2 blocks of equal modules:

  ```
  score += 3
  ```

It does not implement finder-like pattern penalty or dark-ratio penalty in the current HTML build. Any compatibility implementation that wants byte-identical mask choice must use the same two-family scorer.

Mask choice:

```
bestScore = Infinity
for mask in 0..7:
    candidate = applyMask(base, mask)
    writeFormat(candidate, ecLevel, mask)
    s = penaltyScore(candidate)
    if s < bestScore:
        bestScore = s
        bestMask = mask
```

The first mask with the lowest score wins because the update condition is strictly less-than.

Rendering

Function:

```
QRGen.render(matrix, canvas, opts = {})
```

Defaults:

```
quiet = opts.quiet ?? 3
dim = opts.size ?? 256
dark = opts.dark ?? "#000000"
light = opts.light ?? "#ffffff"
```

v1.6.0 modal rendering:

```
Non-XRP:
    size: 256
    quiet: 4
    dark: "#000000"
    light: "#ffffff"

XRP:
    size: 320
    quiet: 4
    dark: "#000000"
    light: "#ffffff"
```

Actual canvas dimensions:

```
totalModules = matrix_size + quiet * 2
module_px = floor(dim / totalModules)
actual = module_px * totalModules
canvas.width = actual
canvas.height = actual
```

Draw:

- Fill entire canvas with light color.
- For each truthy matrix cell, draw a dark rectangle at:

```
x = (column + quiet) * module_px
y = (row + quiet) * module_px
width = module_px
height = module_px
```

All modal QR codes are black on white, independent of UI theme.

PNG Export

Save button:

```
canvas.toBlob(resolve, "image/png")
```

Filename:

```
safeName = address.replace(/[^a-zA-Z0-9]/g, "_").slice(0, 40) || "address"
filename = "qr_" + safeName + ".png"
```

Save route:

```
saveBlob(blob, filename)
```

Browser:

```
Downloads the PNG through an object URL.
```

Tauri:

```
Converts PNG bytes to base64 and passes them to save_export.
```

Copy Address

Copy Address button:

```
address = qrAddressInput.value.trim()
copyTextToClipboard(address)
```

It does not copy:

- QR label text
- URI scheme generated by toBip21Uri
- XRP "?dt=" destination-tag component

The button text changes to "Copied!" for 1500 ms.

Security Boundary

QR export is intended for public address payloads.

It must not be used to encode:

- mnemonic words
- BIP39 passphrases
- .arc passwords
- keyfile bytes
- private keys
- extended private keys
- encrypted seed file contents

Address QR codes can still be privacy-sensitive. Scanning a QR into an online wallet, exchange, camera app, or cloud-synced photo library can link the address to that service or device.

Operational Checks

Before using a QR code:

```
- Confirm the displayed label matches the intended address or URI.
- For XRP, confirm the destination tag with the receiving service.
- For Stellar, confirm memo requirements separately; v1.6.0 encodes only the
  destination in the Stellar URI rule.
- For Ethereum, confirm the chain/network in the receiving wallet.
- For Bitcoin Cash, confirm whether the receiving wallet expects CashAddr.
- For Cardano, confirm the raw address form is accepted by the scanner.
```

Do not assume a wallet scanner validates chain, exchange memo, or custodial requirements just because it accepts the QR code.

Compatibility Checklist

A compatible v1.6.0 QR implementation should:

- Trim address and memo/tag inputs.
- Detect XRP with /^r[1-9A-HJ-NP-Za-km-z]{24,}/.
- Use the exact toBip21Uri rule order.
- Use raw Cardano payload when coinContext is "cardano".

- Use XRP payload address + "?dt=" + tag-or-00000.
- Encode QR data in byte mode using UTF-8.
- Support versions 1 through 10.
- Test M before Q for each version.
- Use the v1.6.0 CAP table.
- Use GF(256) polynomial 0x11d.
- Interleave data and ECC blocks as implemented.
- Mask only boolean data/ECC modules.
- Score masks with run and 2x2 penalties only.
- Render black-on-white with quiet zone 4 in the modal.
- Save PNG through canvas.toBlob.

Capacity Worked Examples

Example: Ethereum address

```
Address:
    0x1111111111111111111111111111111111111111

URI:
    ethereum:0x1111111111111111111111111111111111111111

Byte length:
    9 bytes for "ethereum:"
    42 bytes for address
    total 51 bytes

Version selection:
    v1 M req > 16, no
    v2 M req > 28, no
    v3 M req > 44, no
    v4 M data bytes 64, yes

Result:
    version 4, level M
```

Example: prefixless Bitcoin Cash address

```
Input:
    q...

URI:
    bitcoincash:q...

The prefix is part of the encoded payload unless the input already begins
with "bitcoincash:".
```

Example: XRP address without tag

```
Input:
    r...

User skips destination-tag prompt.

Payload:
    r...?dt=00000

Copy Address:
    copies only r...
```

Example: Stellar address

```
Explicit coin context:
    stellar

URI:
    web+stellar:pay?destination=<encoded-address>

Memo:
    v1.6.0 does not include memo in the Stellar URI rule.
```

Known Limitations

The QR subsystem intentionally does not:

- implement numeric mode
- implement alphanumeric mode selection
- implement Kanji mode
- support versions above 10
- expose custom ECC level selection in the UI
- validate address checksums
- validate XRP destination tag range
- validate Stellar memo format
- validate Ethereum chain ID
- validate Cardano network ID
- decode QR images
- import QR payloads

The generated QR code is a transport representation of the payload string. It does not prove the payload is a correct payment request for a receiving service.

Scanner Compatibility Guidance

Most wallet scanners are strict about:

- quiet zone
- black-on-white contrast
- absence of blur
- correct URI scheme
- expected memo/tag syntax

v1.6.0 uses black modules on a white background and a four-module quiet zone in the modal render paths. Do not crop the PNG to the QR edges; cropping can remove the quiet zone and break scanning.

If a scanner rejects a code:

- compare the label under the QR with the intended payload
- try copying the address manually
- verify whether the scanner expects a bare address or URI scheme
- for Bitcoin Cash, verify CashAddr prefix expectations
- for XRP/Stellar custodial deposits, verify tag/memo requirements

File Handling

QR PNG export writes address payload images, not private key files. They are less sensitive than derived private-key exports, but they can still reveal:

- wallet address ownership
- coin family
- exchange/custodian destination tag in XRP payload label if shared visually
- operational recovery activity

Do not allow QR PNGs to be automatically uploaded to:

- cloud photo libraries
- chat applications
- issue trackers
- email drafts
- shared desktop folders

Byte-Level Implementation Map

QRGen.generate:

- TextEncoder input string.
- Select version and ECC level from CAP.
- Push byte-mode indicator.
- Push byte-length count.
- Push payload bytes.
- Pad to data capacity.
- Generate Reed-Solomon ECC.
- Build matrix.
- Place data.
- Apply all masks.
- Write format bits.
- Choose lowest two-family penalty score.
- Return final matrix.

QRGen.render:

- Compute actual canvas size.
- Set canvas width and height.
- Fill background.
- Draw dark modules.

renderQrFromAddress:

- Detect XRP and delegate to renderXrpQr.
- Otherwise build URI with toBip21Uri.
- Generate matrix.
- Render 256 px requested QR.
- Show label and save controls.

renderXrpQr:

- Build raw r-address plus ?dt payload.
- Generate matrix.
- Render 320 px requested QR.
- Add xrp-mode class.
- Show label and save controls.

Regression Checks

After changing QR code:

- Verify a Bitcoin bc1 address encodes with bitcoin: prefix.
- Verify a prefixless Bitcoin Cash q address encodes with bitcoincash: prefix.
- Verify an already-prefixed bitcoincash: address is not double-prefixed.
- Verify Ethereum 0x address encodes with ethereum: prefix.
- Verify explicit Cardano context encodes raw address.
- Verify XRP without tag encodes ?dt=00000.
- Verify XRP with tag encodes the URL-encoded tag.
- Verify Copy Address copies the address input, not the generated URI.
- Verify PNG save filename sanitizes non-alphanumeric characters.
- Verify generated modules are nonblank on light and dark UI themes.

Arculus Recovery v1.6.0 - Canonical Test Vector Specification

Scope

This document is the byte-level conformance suite for Arculus_Recovery.html v1.6.0. It records deterministic input/output pairs for mnemonic validation, PBKDF2 seed stretching, BIP32 and SLIP-0010 root construction, account derivation, address encoding, ARC V2 encryption, and negative rejection paths.

An implementation is specification-equivalent to v1.6.0 only when it produces the exact bytes and exact encoded strings in this document for the stated inputs. Matching only an address while producing a different private key, public key, chain code, extended key, or encryption transcript is not compliant.

Randomized operations are not byte-stable unless their random inputs are fixed. For ARC V2 encryption this document fixes salt, nonce, created_at, and plaintext so the whole armored file can be compared byte-for-byte. For generated mnemonics and normal ARC exports, implementations must validate structure and round-trip behavior instead of comparing random bytes.

Global Encoding Rules

All hexadecimal strings are lowercase, contain no "0x" prefix, and encode raw bytes in big-endian display order. Spaces and line breaks shown in this document are for readability only unless a field explicitly says ASCII line bytes are part of the encoded value.

Base64 strings use RFC 4648 standard base64 with "+" and "/" and "=" padding. Base58Check strings use the alphabet appropriate to the target protocol: Bitcoin-family Base58 for Bitcoin, Bitcoin Cash legacy, Litecoin, and Dogecoin; XRPL Base58 for XRP. Bech32 strings are lowercase. Bech32m strings are lowercase. Ethereum addresses are EIP-55 checksum addresses.

Mnemonic text is normalized exactly as v1.6.0 does before seed derivation:

- Trim leading and trailing JavaScript whitespace.
- Split on one or more whitespace code points.
- Drop empty tokens.
- Join tokens with a single ASCII space, 0x20.
- Apply Unicode NFKD normalization.
- UTF-8 encode the normalized mnemonic.

The BIP39 passphrase is normalized with Unicode NFKD and UTF-8 encoded. The PBKDF2 salt is ASCII("mnemonic") || UTF8(NFKD(passphrase)).

All derivation paths are absolute and start with "m". Hardened components may be written with "'", "h", or "H" at input, but vectors in this document use apostrophe notation. A hardened index is encoded as uint32(index + 0x80000000). Non-hardened index i is encoded as uint32(i). Child-number bytes in BIP32 HMAC messages are 4-byte big-endian unsigned integers.

Canonical Inputs

TV1 mnemonic:

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon about
```

TV1 passphrase:

```
empty string
```

TV1 normalized mnemonic bytes, hex:

```
6162616e646f6e206162616e646f6e206162616e646f6e206162616e646f6e20
6162616e646f6e206162616e646f6e206162616e646f6e206162616e646f6e20
6162616e646f6e206162616e646f6e206162616e646f6e2061626f7574
```

TV1 normalized mnemonic byte length:

```
93
```

TV1 entropy bytes:

```
00000000000000000000000000000000
```

TV1 entropy byte length:

```
16
```

TV1 entropy bits:

```
128
```

TV1 checksum bit length:

```
4
```

TV1 checksum bits:

```
0011
```

TV1 final word index:

```
3
```

TV1 final word:

```
about
```

TV1 BIP39 seed, 64 bytes:

```
5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc1
9a5ac40b389cd370d086206dec8aa6c43daea6690f20ad3d8d48b2d2ce9e38e4
```

Seed derivation parameters:

```
algorithm: PBKDF2-HMAC-SHA512
password bytes: UTF8(NFKD(TV1 mnemonic))
salt bytes: ASCII("mnemonic")
iterations: 2048
dkLen: 64
```

BIP32 secp256k1 Root Node, TV1

Master node derivation:

```
algorithm: HMAC-SHA512
key bytes: ASCII("Bitcoin seed")
data bytes: TV1 BIP39 seed
output length: 64 bytes
IL: bytes 0..31
IR: bytes 32..63
```

Master private key, IL, 32 bytes:

```
1837c1be8e2995ec11cda2b066151be2cfb48adf9e47b151d46adab3a21cdf67
```

Master chain code, IR, 32 bytes:

```
7923408dadd3c7b56eed15567707ae5e5dca089de972e07f3b860450e2a3b70e
```

Root fingerprint:

```
73c5da0a
```

Root xprv:

```
xprv9s21ZrQH143K3GJpoapnV8SFfukcVBSfeCficPSGfubmSFDxo1kuHnLisriDvSnRRuL2Qrg5ggqHKNVpxR86QEC8
w35uxmGoggxtQTPvfUu
```

Root xpub:

```
xpub661MyMwAqRbcFkPHucMnrGNzDwb6teAX1RbKQmqtEF8kK3Z7LZ59qafCjB9eCRLiTVG3uxBxgKvRgbubRhqSKXnG
Gb1aoaqLrpMBDrVxga8
```

BIP32 serialized private extended key payload before checksum is 78 bytes:

```
version: 0488ade4
depth: 00
parent_fingerprint: 00000000
child_number: 00000000
chain_code: 7923408dadd3c7b56eed15567707ae5e5dca089de972e07f3b860450e2a3b70e
```

```
key_data: 00 || 1837c1be8e2995ec11cda2b066151be2cfb48adf9e47b151d46adab3a21cdf67
```

BIP32 serialized public extended key payload before checksum is 78 bytes:

```
version: 0488b21e
depth: 00
parent_fingerprint: 00000000
child_number: 00000000
chain_code: 7923408dadd3c7b56eed15567707ae5e5dca089de972e07f3b860450e2a3b70e
key_data: 0339a36013301597daef41fdf21a57419e8c75857fbbfddf8b76b3a8dc9cd0d4d8
```

Default v1.6.0 Account Paths

The v1.6.0 UI preselects these account-level derivation paths for the supported coin selector values:

```
Bitcoin: m/0'
Bitcoin Cash: m/0'
Litecoin: m/84'/2'/0'
Dogecoin: m/44'/3'/0'
Ethereum / ERC-20: m/44'/60'/0'
Solana: m/44'/501'/0'
Stellar: m/44'/148'/0'
Monero: m/44'/128'/0'
Cardano: m/1852'/1815'/0'/0/0
XRP: m/44'/144'/0'
```

For secp256k1 coins, the displayed receiving rows append /0/i and displayed change rows append /1/i to the account path. For Solana and Stellar, v1.6.0 uses hardened account-style rows under the selected account path. For Monero, the account path itself is the displayed account address. For Cardano, the selected path is the payment key path and the staking key path is fixed at m/1852'/1815'/0'/2/0 for the displayed Shelley base address.

Bitcoin Mainnet, TV1, Auto Script, m/0'

Inputs:

```
mnemonic: TV1
passphrase: empty string
coin selector: Bitcoin
internal coin id: bitcoin
network: mainnet
derivation: m/0'
script selector: Auto
count: 2
range: empty
```

Account output:

```
account_script_type_used: p2pkh
account_xprv:
xprv9ukW2Usuz4v7Yd2EC4vNXaMckdsEdgBA9n7MQbqMJbW9FuHDWWjDwzEM2h6XmFnrzX7JVmfcNWMEVoRauU6hQpbo
kqPPNTbdycW9fHSPYyF
account_xpub:
xpub68jrRzQopSUQm76hJ6TNtiJMJfhj38u1X12xCzExrw388hcN443UVnYpswdUkV7vPJ3KayiCdp3Q5E23s4wvkuco
hVTh7eSstJdBFyn2DMx
```

Receiving row 0:

```
path: m/0'/0/0
address: 17871ErDqdevLTLWBH6WzjUc1EKGDQzCMA
public_key_hex: 026666422d00f1b308fc7527198749f06fedb028b979c09f60d0348ef79c985e41
private_key_hex: e81fa7bb95cc6bee975bf675bfebbab4044c1390e6a00ae246e55c07e3cf835e
private_key_wif: L4zvqB8Cme7xRmTWxQ6TVmQs7smCNKD1rfAKmYZx6DAqyLjm4sp4
```

Receiving row 1:

```
path: m/0'/0/1
address: 1B4ynFJzDPqvuttzgF16jccWGxvdJwHSWr
public_key_hex: 0384257cf895f1ca492bbee5d7485ae0ef479036fdf59e15b92e37970a98d6fe75
private_key_hex: 660e9c5602379d842bb74883f34e888e4ffcb45ea38a8b57f20dc6099eef5a13
private_key_wif: Kze6YryiB5bfUTddt8fQi2ZPY2f944W1PWuJPFf5CSTy1owCpQGZ
```

Change row 0:

```
path: m/0'/1/0
address: 1MseVFBWLkbPeGMpkAsahBujinBq3QjGo4
public_key_hex: 030247a355c3263a69dce173ac12fcc49408260b76b089ab02dc68f3389fc57cb8
private_key_hex: 2c8de4729a7e4dc1ab55e4638bb8540d70cd01a3e31e776631a02b7b72c53b30
```

```
  private_key_wif: KxiKRsSXMTSzAeBMyriBwDbu482pdWHAE5cGYNiFQNMaYwRJKkTB
```

Change row 1:

```
  path: m/0'/1/1
  address: 1AopQaJaiEt9sw2cDyf3CqSHZfiNtrWzxd
  public_key_hex: 0394e33c4c819a55235cde10baed2df62f2787be0607d920dc3947069b23633022
  private_key_hex: e4e54ad17c56f88f18e12d83dba09956be07dbdc15467d4f22a085aadea4d4ed
  private_key_wif: L4tetqr49fjQreYr6nMJyPsBERh9XvWoWWyNz9NHeEUs9CsFV6xK
```

Bitcoin Mainnet, TV1, P2WPKH Script, m/0'

Inputs are the same as the Bitcoin Auto vector except script selector is P2WPKH.

Account output:

```
  account_script_type_used: p2wpkh
  account_zprv:
zprvAZR2dpDkHS15FDQTrnVcwkYd6aA8WvA9z19nyPd84cFuN6ug1q4MC7Yd571hm56hooLuzirjHq4LGNeiLrvj1Hy1
VWnEYHEcX4dSSTGpeBe
  account_zpub:
zpub6nQP3Kke7oZNThUvxp2dJtVMebzcvNt1ME5Pmn2jcwntEuEpZNNbjus6vMYekJRmCaGw5vuKZ8kVqoFBJTmxMNz1
SArYHU5rRkkU38oTcty
```

Receiving row 0:

```
  path: m/0'/0/0
  address: bc1qgv52mt89gpev6p56huggl970sppqkgftxakv7f
  public_key_hex: 026666422d00f1b308fc7527198749f06fedb028b979c09f60d0348ef79c985e41
  private_key_hex: e81fa7bb95cc6bee975bf675bfebbab4044c1390e6a00ae246e55c07e3cf835e
  private_key_wif: L4zvqB8Cme7xRmTWxQ6TVmQs7smCNKD1rfAKmYZx6DAqyLjm4sp4
```

Receiving row 1:

```
  path: m/0'/0/1
  address: bc1qdec7wsahwjs4tgg7a66gk9g7vul0ufjhemdpqz
  public_key_hex: 0384257cf895f1ca492bbee5d7485ae0ef479036fdf59e15b92e37970a98d6fe75
  private_key_hex: 660e9c5602379d842bb74883f34e888e4ffcb45ea38a8b57f20dc6099eef5a13
  private_key_wif: Kze6YryiB5bfUTddt8fQi2ZPY2f944W1PWuJPFf5CSTy1owCpQGZ
```

Change row 0:

```
  path: m/0'/1/0
  address: bc1qunmfkdmckn76c8nmf3g22du699gne5q8c3xqhl
  public_key_hex: 030247a355c3263a69dce173ac12fcc49408260b76b089ab02dc68f3389fc57cb8
  private_key_hex: 2c8de4729a7e4dc1ab55e4638bb8540d70cd01a3e31e776631a02b7b72c53b30
  private_key_wif: KxiKRsSXMTSzAeBMyriBwDbu482pdWHAE5cGYNiFQNMaYwRJKkTB
```

Change row 1:

```
  path: m/0'/1/1
  address: bc1qdwfad4n29p6kwh7dh6ecz0c8uv4z0u0f0njxhf
  public_key_hex: 0394e33c4c819a55235cde10baed2df62f2787be0607d920dc3947069b23633022
  private_key_hex: e4e54ad17c56f88f18e12d83dba09956be07dbdc15467d4f22a085aadea4d4ed
  private_key_wif: L4tetqr49fjQreYr6nMJyPsBERh9XvWoWWyNz9NHeEUs9CsFV6xK
```

P2WPKH address construction for row 0:

```
  witness_version: 0
  witness_program: HASH160(compressed_public_key), 20 bytes
  witness_program_hex: 4328adb4e54072cd068abd108f97cf80420b212b
  human_readable_part: bc
  address: bc1qgv52mt89gpev6p56huggl970sppqkgftxakv7f
```

Bitcoin Cash Mainnet, TV1, Auto Script, m/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Bitcoin Cash
  internal coin id: bitcoincash
  network: mainnet
  derivation: m/0'
  script selector: Auto
  count: 2
```

Account script type:

```
p2pkh
```

Receiving row 0:

```
path: m/0'/0/0
address: bitcoincash:qppj3tdvu4q89ngxn2l3pruhe7qyyzep9v3l5ke649
legacy_payload_hash160: 4328adb4e54072cd068abd108f97cf80420b212b
public_key_hex: 026666422d00f1b308fc7527198749f06fedb028b979c09f60d0348ef79c985e41
private_key_hex: e81fa7bb95cc6bee975bf675bfebbab4044c1390e6a00ae246e55c07e3cf835e
private_key_wif: L4zvqB8Cme7xRmTWxQ6TVmQs7smCNKD1rfAKmYZx6DAqyLjm4sp4
```

Receiving row 1:

```
path: m/0'/0/1
address: bitcoincash:qph8re6rka62z4dprmhtfzc4rennal3x2uuu3nua4w
public_key_hex: 0384257cf895f1ca492bbee5d7485ae0ef479036fdf59e15b92e37970a98d6fe75
private_key_hex: 660e9c5602379d842bb74883f34e888e4ffcb45ea38a8b57f20dc6099eef5a13
private_key_wif: Kze6YryiB5bfUTddt8fQi2ZPY2f944W1PWuJPFf5CSTy1owCpQGZ
```

Change row 0:

```
path: m/0'/1/0
address: bitcoincash:qrj0dxeh0z60mtq70dx9pffhng54z0xsquelhdnxpz
public_key_hex: 030247a355c3263a69dce173ac12fcc49408260b76b089ab02dc68f3389fc57cb8
private_key_hex: 2c8de4729a7e4dc1ab55e4638bb8540d70cd01a3e31e776631a02b7b72c53b30
private_key_wif: KxiKRsSXMTSzAeBMyriBwDbu482pdWHAE5cGYNiFQNMaYwRJKkTB
```

Change row 1:

```
path: m/0'/1/1
address: bitcoincash:qp4e84kkdg582e6leklt8qflql3j5fl3ayy20l89y3
public_key_hex: 0394e33c4c819a55235cde10baed2df62f2787be0607d920dc3947069b23633022
private_key_hex: e4e54ad17c56f88f18e12d83dba09956be07dbdc15467d4f22a085aadea4d4ed
private_key_wif: L4tetqr49fjQreYr6nMJyPsBERh9XvWoWWyNz9NHeEUs9CsFV6xK
```

Litecoin Mainnet, TV1, Auto Script, m/84'/2'/0'

Inputs:

```
mnemonic: TV1
passphrase: empty string
coin selector: Litecoin
internal coin id: litecoin
network: mainnet
derivation: m/84'/2'/0'
script selector: Auto
count: 2
```

Account output:

```
account_script_type_used: p2wpkh
account_xprv:
Ltpv77z3LHgiTumhjPwxY5THUn11RJvRPQU6yK8iFN9TCUnfvox2ixQQGHmuNx5Bo1yGQWrSbo87JgF1CKVQ28zddVZ1
5xgNeL4QnHXSKrXfbih
account_xpub:
Ltub2ZANw57Fi4JfMGenngwfZMqDrsccTf2m3qEjvLuk8faGXB973sSqGttAdKBvGSCLG9DXh2yxaUB9qpwQUe2zCCtr
ZPhjxGuMAWqL6F7Xrpd
account_zprv:
zprvAdQSgFiAHcXKb1gcktyukXyGZykTBemmPZ9brfYnxqxM2CocMdxy9aPXTbTLv7dvJgWn2Efi4vFSyPbT4QqgarYr
s583WCeMXM2q3TUU8FS
account_zpub:
zpub6rPo5mF47z5coVm5rvWv7fv181awb7Vckn5Cf3xQXBVKu18kuBHDhNi1Jrb4br6vVD3ZbrnXemEsWJoR18mZwkUd
zwD8TQnHDUCGxqZ6swA
```

Receiving row 0:

```
path: m/84'/2'/0'/0/0
address: ltc1qjmxnz78nmc8nq77wuxh25n2es7rzm5c2rkk4wh
public_key_hex: 02e49c9b9b5d0f127235dc26a0c252814c52fb333d651a946773f59d72c2da9904
private_key_hex: 4ab54480cbbaa53d5d3ba43a3e85d221df085490e88346ad11579e17000db7bb
private_key_wif: T5ZCYhLqXu6EJKk2nhjvwsaLH357CisixhLGWpKXEiqWTUtzte6o
```

Receiving row 1:

```
path: m/84'/2'/0'/0/1
address: ltc1qwlezpr3890hcp6vva9twqh27mr6edadreqvhnn
public_key_hex: 021c1750d4a5ad543967b30e9447e50da7a5873e8be133eb25f2ce0ea5638b9d17
private_key_hex: 61fa14b9b3d872aaa327855cc94b5a220094395d55d01319cfee32e236a26d1e
private_key_wif: T6LRyVtoN2JxywNyYXwk9KoMpWa34Fjino4upyy9RKs5QQmC8RSr
```

Change row 0:

```
  path: m/84'/2'/0'/1/0
  address: ltc1qyeljcy9v88jg8sqvnqh0m5q390xruc5r98q9yy
  public_key_hex: 029857513f0fe1dc125f219ffef22098e14c653c4bcb0a2aaeff47b3a252569f1a
  private_key_hex: dbf6dd9d691b00d0a7f6f93605a67ddb42ba4baafcf9b0d98979b66f8a21184e
  private_key_wif: TARZNteayzJqRiXnqKJX3h5zr6H4tx4sXvoZ21pHjAEgNc1g2MWm
```

Change row 1:

```
  path: m/84'/2'/0'/1/1
  address: ltc1qzenvkhkqqazfanrjs6htluwlm2sdwt0sgc3jh8
  public_key_hex: 02df3af22530218813eab494b0bec331ee13e4662a566e24bed32a35a4cc2a248f
  private_key_hex: acfa7e72eeb4254be5f87a09685089039f6e4736cb7cb998e48ff51202c5fdc8
  private_key_wif: T8rDzH8bF1cgAG2oZKctJX9qmxfVK7HmgQqww4wom15ndzPzF2uu
```

Dogecoin Mainnet, TV1, Auto Script, m/44'/3'/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Dogecoin
  internal coin id: dogecoin
  network: mainnet
  derivation: m/44'/3'/0'
  script selector: Auto
  count: 2
```

Account output:

```
  account_script_type_used: p2pkh
  account_xprv:
dgpv57bftCH9z6cEAdAY9SCDV9NfVsygaQWdi5LuCXdumz5qUPWnw1S3YBM7PdHXMvA8oSGS6Pbes1xEHMd5Zi2qHVK4
5y5FKKXzBXsZcTtYmX5
  account_xpub:
dgub8rUhDtD3YFGZTUphBfpBbzvFxSMKQXYLzg87Me2ta78r2SdVLmypBUkkxrrn9RTnchsyiJSkHZyLWxD13ibBiXtu
FWktBoDaGaZjQUBLNLs
```

Receiving row 0:

```
  path: m/44'/3'/0'/0/0
  address: DBus3bamQjgJULBJtYXpEzDWQRwF5iwxgC
  public_key_hex: 02cc6b0dc33aabcf3a23643e5e2919a80c50fb3dd2129ce409bbc5f0d4643d05e0
  private_key_hex: 21f5e16d57b9b70a1625020b59a85fa9342de9c103af3dd9f7b94393a4ac2f46
  private_key_wif: QPkeC1ZfHx3c9g7WTj9cQ8gnvk2iSAfAcbq1aVAWjNTwDAKfZUzx
```

Receiving row 1:

```
  path: m/44'/3'/0'/0/1
  address: DAcDAtJRztxBHyA6D6h8du1HguyTR43Mas
  public_key_hex: 025a8ad8881f6facdc949c4a4d03257414153faea67e96acf57344660080610788
  private_key_hex: 0ce96fd14dd5bd468a0927b5016d138e06d5dfb1db64c69a1b22e79c65120b63
  private_key_wif: QP3j5SpiwZk9k8z9kaV2S6gbrVkoPaWyfW1Wy4x1WUQoyTab9VfH
```

Change row 0:

```
  path: m/44'/3'/0'/1/0
  address: D7ReBLrRv12mi9pYh5HtfFLTt1PSoeAa7e
  public_key_hex: 020745c065a8b7cb03a843a57339cf9164d675a2e5010c6fbd61c09239bbb5f4df
  private_key_hex: da27a8fcc03519980a5db5dfdceaeab7b61985e9463f3fb750a1d86b2cefd65c
  private_key_wif: QVvh62CvvLSWYTTJXS62RdyuXnBn9adqgzJ6xrZRjpwYRvuHLCtw
```

Change row 1:

```
  path: m/44'/3'/0'/1/1
  address: D92vUBQJjMhmczzGf7ReUswoDEZuqaNout
  public_key_hex: 02475f40b385bb486ef27171fe119a89ce9b46fa1ab0f5bc542ee0e1e1fcd78623
  private_key_hex: 6799f60741b5023781d69ef9b865714d12d99ffa2ee1c66d1edd091cab07182d
  private_key_wif: QS61pQ4VhzRFbfLyxTDBSeztbuk2pu15bRw1Z6LAgpFzp8fHdF25
```

Ethereum Mainnet, TV1, m/44'/60'/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Ethereum / ERC-20
```

```
  internal coin id: ethereum
  network: mainnet
  derivation: m/44'/60'/0'
  script selector: Auto
  count: 2
```

Account output:

```
  account_script_type_used: ethereum
  account_xprv:
xprv9zDSoJv1aBcjX6sNgEpE2J9K6MV2MUnXuqXsFgzVn3zY2aHyupaFQdYCtdCbNMkvcTdx9FeN49sgXw6mjrhrFLRS
zJVnRYPfSCCgjeg4GxY
  account_xpub:
xpub6DCoCpSuQZB2jawqnGMEPS63ePKWkwWPH4TU45Q7LPXWuNd8TMtVxRrgjtEshuqpK3mdhaWHPFsBngh5GFZaM6si
3yZdUsT8ddYM3PwnATt
```

Receiving row 0:

```
  path: m/44'/60'/0'/0/0
  address: 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
  public_key_hex: 0237b0bb7a8288d38ed49a524b5dc98cff3eb5ca824c9f9dc0dfdb3d9cd600f299
  ethereum_public_key_hex:
37b0bb7a8288d38ed49a524b5dc98cff3eb5ca824c9f9dc0dfdb3d9cd600f299a6179912b7451c09896c4098eca7
ce6b2e58330672795e847c4d6af44e024230
  private_key_hex: 1ab42cc412b618bdea3a599e3c9bae199ebf030895b039e9db1e30dafb12b727
  erc20_address: 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
```

Receiving row 1:

```
  path: m/44'/60'/0'/0/1
  address: 0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0
  public_key_hex: 039fd0991d0222b4e1339c1a1a5b5f6d9f6a96672a3247b638ee6156d9ea877a2f
  ethereum_public_key_hex:
9fd0991d0222b4e1339c1a1a5b5f6d9f6a96672a3247b638ee6156d9ea877a2f1735e3a9260940e4c2225c344a8c
ea6c7b6a6057d0eb90a9a875f446c131031d
  private_key_hex: 9a983cb3d832fbde5ab49d692b7a8bf5b5d232479c99333d0fc8e1d21f1b55b6
  erc20_address: 0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0
```

Change row 0:

```
  path: m/44'/60'/0'/1/0
  address: 0x399Db6Ed32539fbDF44c3e7678b5b428e378F666
  public_key_hex: 03abdc0424e5a951abb5e12df771738e269566fc170dfe8a5f0dae27052a168559
  ethereum_public_key_hex:
abdc0424e5a951abb5e12df771738e269566fc170dfe8a5f0dae27052a1685590ed3baf090b916db5f923ae4c5d3
4aba23466c172027e17749f0c8f579f0935b
  private_key_hex: dcea9371bd9641a066ad61b6542ffa3eb2835cababbf202ce2250c726ce736b3
  erc20_address: 0x399Db6Ed32539fbDF44c3e7678b5b428e378F666
```

Change row 1:

```
  path: m/44'/60'/0'/1/1
  address: 0x26db4d065800Bd118928848E69A1cBF956Cff1D0
  public_key_hex: 02dd553226bc6d4d2efafabfe7600a10d4d069de1371dd3207a293fb96766b3073
  ethereum_public_key_hex:
dd553226bc6d4d2efafabfe7600a10d4d069de1371dd3207a293fb96766b307376605b0580017eb8e3ee5202d613
fca46d7befcbbe3303ccc1da9d32f8fd4e96
  private_key_hex: 011fe87587edf9d5bad64e253778ed929ecf319302bd41f4ce051e2678052cc1
  erc20_address: 0x26db4d065800Bd118928848E69A1cBF956Cff1D0
```

Ethereum address construction for receiving row 0:

```
  private_key: 32 bytes, secp256k1 scalar
  public_key: uncompressed secp256k1 point, 65 bytes, prefix 0x04 removed
  hash: Keccak-256(64-byte x||y)
  address_body: last 20 bytes of hash
  display: EIP-55 checksum over lowercase hex address_body
```

Solana Mainnet, TV1, SLIP-0010 Ed25519, m/44'/501'/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Solana
  internal coin id: solana
  network: mainnet
  derivation: m/44'/501'/0'
  script selector: Auto
```

```
count: 2
```

Root Ed25519 node:

```
root_private_key_hex: 560f9f3c94558b6551928bb781cf6092c6b8800b4fc544af2c9444ed126d51aa
root_chain_code_hex: ddfa71109701bbf7c126c8c7ab5880b0dec3d167a8fe6afa7a9597df0bbee72b
root_public_key_hex: e96b1c6b8769fdb0b34fbecfdf85c33b053cecad9517e1ab88cba614335775c1
```

Account Ed25519 node:

```
account_private_key_hex: ec252c5d95bcf80a4b22df119cedd4ae1aed07364578e8d43bdfa0435625dbd1
account_chain_code_hex: 30f34004e6e8a90ec46c36ec8fef945a52b269adb55816a98ec9c063434a6b3d
account_public_key_hex: e9b6062841bb977ad21de71ec961900633c26f21384e015b014a637a61499547
```

Receiving row 0:

```
branch: account
path: m/44'/501'/0'
address: GjJyeC1r2RgkuoCWMyPYkCWSGSGLcz266EaAkLA27AhL
public_key_hex: e9b6062841bb977ad21de71ec961900633c26f21384e015b014a637a61499547
private_key_hex: ec252c5d95bcf80a4b22df119cedd4ae1aed07364578e8d43bdfa0435625dbd1
```

Receiving row 1:

```
branch: account
path: m/44'/501'/0'/1'
address: GKreMsHvt8A79VApjboYDq3J4ZCXSJRYYQk9BscMbi1H
public_key_hex: e3b3e4299dc3a9380b9ac55acd5c18e87666bbfdf010c3e79928d784670919c4
private_key_hex: 22b1890a6c748d580a6fe6fb99334d76bd2496126be46ecf9e6fc412b53368e1
```

Solana address construction:

```
address bytes: 32-byte Ed25519 public key
address encoding: raw public key bytes encoded with Bitcoin Base58 alphabet
checksum: none
```

Stellar Mainnet, TV1, SLIP-0010 Ed25519, m/44'/148'/0'

Inputs:

```
mnemonic: TV1
passphrase: empty string
coin selector: Stellar
internal coin id: stellar
network: mainnet
derivation: m/44'/148'/0'
script selector: Auto
count: 2
```

Root Ed25519 node:

```
root_private_key_hex: 560f9f3c94558b6551928bb781cf6092c6b8800b4fc544af2c9444ed126d51aa
root_chain_code_hex: ddfa71109701bbf7c126c8c7ab5880b0dec3d167a8fe6afa7a9597df0bbee72b
root_public_key_hex: e96b1c6b8769fdb0b34fbecfdf85c33b053cecad9517e1ab88cba614335775c1
```

Account Ed25519 node:

```
account_private_key_hex: 695db2365365e06159f13cc752a5d3b0c7376c45012b2f695e3a4da2434948bc
account_chain_code_hex: fe47279273939c24fc0b8bdda584b5b0ad01f0c51208661d2aae5ffa687ce6f8
account_public_key_hex: 7691d85048acc4ed085d9061ce0948bbdf7de6a92b790aaf241d31b7dcaa4238
```

Receiving row 0:

```
branch: account
path: m/44'/148'/0'
address: GB3JDWCQJCWMJ3IILWIGDTQJJC5567PGVEVXSCVPEQOTDN64VJBDQBYX
public_key_hex: 7691d85048acc4ed085d9061ce0948bbdf7de6a92b790aaf241d31b7dcaa4238
private_key_hex: 695db2365365e06159f13cc752a5d3b0c7376c45012b2f695e3a4da2434948bc
stellar_secret_seed: SBUV3MRWKNS6AYKZ6E6MOUVF2OYMON3MIUASWL3JLY5E3ISDJFELYBRZ
```

Receiving row 1:

```
branch: account
path: m/44'/148'/1'
address: GDVSYYTUAJ3ACHTPQNSTQBDQ4LDHQCMNY4FCEQH5TJUMSSLWQSTG42MV
public_key_hex: eb2c62740276011e6f8365380470e2c678098dc70a2240fd9a68c9497684a66e
private_key_hex: 8e315456301464326e8d7cae8f18aec38e58db8838cf304132567a0ad4d305e1
stellar_secret_seed: SCHDCVCWGAKGIMTORV6K5DYYV3BY4WG3RA4M6MCBGJLHUCWU2MC6DL66
```

Stellar address construction:

```
  public raw payload: version_byte 0x30 || 32-byte Ed25519 public key
  secret raw payload: version_byte 0x90 || 32-byte Ed25519 seed
  checksum: CRC16-XModem, little-endian in encoded payload
  encoding: RFC 4648 base32 without padding in display form
```

Monero Mainnet, TV1, m/44'/128'/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Monero
  internal coin id: monero
  network: mainnet
  derivation: m/44'/128'/0'
  script selector: Auto
  count: 1
```

Root Ed25519 node:

```
  root_private_key_hex: 560f9f3c94558b6551928bb781cf6092c6b8800b4fc544af2c9444ed126d51aa
  root_chain_code_hex: ddfa71109701bbf7c126c8c7ab5880b0dec3d167a8fe6afa7a9597df0bbee72b
  root_private_spend_key_hex:
14c8049b8c76d3f4f171df59cf0cabc1c5b8800b4fc544af2c9444ed126d510a
  root_private_view_key_hex:
e11a19cef93c1c40022ca506220c0f3cd0430bee4c9598d3a3b668bde1611b07
  root_public_spend_key_hex:
1652f408a68b70b1f451a3fe6ee02492ca40cb5723b6f55b9b518e0d262a2fca
  root_public_view_key_hex: 76d07541ef20a993c4ed574decafafcd45966a48efdb2b668c6f664d14c9c4a1
```

Account Monero keys:

```
  account_private_key_hex: 98f0cdd5993ce34eb46b57c1fa6399f6643516416fae089d74ecfd0c389a6bd8
  account_chain_code_hex: aded23c191b516375a114a093db93f2690cdb5d6cdb7257e653243ab432eaeca
  account_private_spend_key_hex:
8f2d521d4334f4d5d174c47aacb346e7633516416fae089d74ecfd0c389a6b08
  account_private_view_key_hex:
84b0087a63854856686f80596a5a1a6090795b4d1311139173262170d7e7180f
  account_public_spend_key_hex:
51fd81faa5e2641bca8a5c43d764ab276033c12b459fd9f5c9c23b7733af67e6
  account_public_view_key_hex:
eb8855fd7d21670f9bdfc0cdcfa54172e80322c4eaee5ce30d85cb9ec90d21d1
```

Receiving row 0:

```
  branch: account
  path: m/44'/128'/0'
  address:
44jKQv6ZKMd5ecLLmkNJGi7azgSptEq8ki7TFiat1TfLfdDQ1tQ7ZYa3cRh7X2uRwvLDjddWh97ajeyhR2seKSECQeDx
1WR
  public_spend_key_hex: 51fd81faa5e2641bca8a5c43d764ab276033c12b459fd9f5c9c23b7733af67e6
  public_view_key_hex: eb8855fd7d21670f9bdfc0cdcfa54172e80322c4eaee5ce30d85cb9ec90d21d1
  private_spend_key_hex: 8f2d521d4334f4d5d174c47aacb346e7633516416fae089d74ecfd0c389a6b08
  private_view_key_hex: 84b0087a63854856686f80596a5a1a6090795b4d1311139173262170d7e7180f
```

Monero address construction:

```
  network byte: 0x12 for standard mainnet address
  spend key: 32-byte public spend key
  view key: 32-byte public view key
  checksum: first 4 bytes of Keccak-256(network || spend || view)
  encoding: Monero base58 in 8-byte blocks
```

Cardano Mainnet, TV1, Shelley Base Address, m/1852'/1815'/0'/0/0

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: Cardano
  internal coin id: cardano
  network: mainnet
  derivation: m/1852'/1815'/0'/0/0
  staking path: m/1852'/1815'/0'/2/0
  script selector: Auto
  count: 1
```

Root Icarus Ed25519-BIP32 node:

```
  root_private_key_hex:
60ce7dbec3616e9fc17e0c32578b3f380337b1b61a1f3cb9651aee30670e6f53970419a23a2e4e4082d12bf78faa
8645dfc882cee2ae7179e2b07fe88098abb2
  root_chain_code_hex: 072310084784c7308182dbbdb1449b2706586f1ff5cbf13d15e9b6e78c15f067
  root_public_key_hex: 37fdfdbe9ac856469f8d83c66c57880246cd8bf7f852bf5b94336fe535c0efc8
```

Payment key node:

```
  account_private_key_hex:
105d2ef2192150655a926bca9cccf5e2f6e496efa9580508192e1f4a790e6f53de06529129511d1cacb0664bcf04
853fdc0055a47cc6d2c6d205127020760652
  account_chain_code_hex: 88848e8af62a27a57e982215741c9eac17e6e45cbfd6ea65a0e0dcc03bb777b2
  account_public_key_hex: 7ea09a34aebb13c9841c71397b1cabfec5ddf950405293dee496cac2f437480a
```

Receiving row 0:

```
  branch: receiving
  path: m/1852'/1815'/0'/0/0
  staking_path: m/1852'/1815'/0'/2/0
  address:
addr1qy8ac7qqy0vtulyl7wntmsxc6wex80gvcyjy33qffrhm7sh927ysx5sftuw0dlft05dz3c7revpf7jx0xnlcjz3
g69mq4afdhv
  public_key_hex: 7ea09a34aebb13c9841c71397b1cabfec5ddf950405293dee496cac2f437480a
  stake_public_key_hex: 012f5dc3115b8a07981e6e50f5a671e2c6fbb26c3ffde1cd1dcaf40a7fe8f160
  private_key_hex:
105d2ef2192150655a926bca9cccf5e2f6e496efa9580508192e1f4a790e6f53de06529129511d1cacb0664bcf04
853fdc0055a47cc6d2c6d205127020760652
  stake_private_key_hex:
2047891e88934a422e3ff6eea4caaaecda795724cd2bfacf91569896750e6f534a704748705e28a7bd69d009638a
dd767f79c63eb9c839c8ffe0eff0b6e16772
```

Cardano address construction:

```
  address type: Shelley base address, payment key hash plus stake key hash
  network id: 1 for mainnet
  payment key hash: Blake2b-224(payment_public_key)
  stake key hash: Blake2b-224(stake_public_key)
  raw address bytes: 1-byte header || 28-byte payment hash || 28-byte stake hash
  display encoding: Bech32 HRP "addr"
```

XRP Mainnet, TV1, m/44'/144'/0'

Inputs:

```
  mnemonic: TV1
  passphrase: empty string
  coin selector: XRP
  internal coin id: xrp
  network: mainnet
  derivation: m/44'/144'/0'
  script selector: Auto
  count: 2
```

Account output:

```
  account_script_type_used: xrp
  account_xprv:
xprv9yFya3vnAMVKmALsNBnw3G98Trzy4AmqciPkhNSZUNUuhTvTA3gWQRg2ecJFS3PDLsfYFgwDW1UukaapjTDUENfi
wg22ryd4mWiph8Faw3p
  account_xpub:
xpub6CFKyZTfzj3cyeRLUDKwQQ5s1tqTTdVgywKMVkrB2i1taGFbhazkxDzWVsfBHZpv7rg6qpDBGYR5oA8iazEfa44C
dQkkknPFHJ7YCzncCS9
```

Receiving row 0:

```
  path: m/44'/144'/0'/0/0
  address: rHsMGQEkVNJmpGWs8XUBoTBiAAbwxZN5v3
  xrp_classic_address: rHsMGQEkVNJmpGWs8XUBoTBiAAbwxZN5v3
  public_key_hex: 031d68bc1a142e6766b2bdfb006ccfe135ef2e0e2e94abb5cf5c9ab6104776fbae
  private_key_hex: 90802a50aa84efb6cdb225f17c27616ea94048c179142fecf03f4712a07ea7a4
```

Receiving row 1:

```
  path: m/44'/144'/0'/0/1
  address: r3AgF9mMBFtaLhKcg96weMhbbEFLZ3mx17
  xrp_classic_address: r3AgF9mMBFtaLhKcg96weMhbbEFLZ3mx17
  public_key_hex: 038bf420b5271ada2d7479358ff98a29954cf18dc25155184aead05796da737e89
```

```
private_key_hex: 0974b4cfe004a2e6c4364cbf3510a36a352796728d0861f6b555ed7e54a70389
```

Change row 0:

```
path: m/44'/144'/0'/1/0
address: rE8wWANXkzNgRx6noykbeXQRcfVRNoZQv4
xrp_classic_address: rE8wWANXkzNgRx6noykbeXQRcfVRNoZQv4
public_key_hex: 02852f3ccc7e31159e3e8afdf84fa919b6313197b7fd8f454139aec2a6d9306721
private_key_hex: ec5756c9d1abc5626e434e047f4c6afb791ffec66d5396bdbe5c119d7326240b
```

Change row 1:

```
path: m/44'/144'/0'/1/1
address: rwEcK2kRntGkzULqYndeHW1Qq32zeK5fwG
xrp_classic_address: rwEcK2kRntGkzULqYndeHW1Qq32zeK5fwG
public_key_hex: 0229fbc91aa74c94fa0b0078385de4ea96bda59c5a76b8ce50a7aef86860e39321
private_key_hex: 291b4b8ad8e50a6e9da60c64007e1d6ad346f450eda289daaf8448560535d79b
```

XRP address construction:

```
account id: HASH160(compressed_public_key), 20 bytes
version byte: 0x00
checksum: first 4 bytes of double-SHA256(version || account_id)
encoding alphabet: XRPL Base58 alphabet
destination tag: not derived and not encoded in the key vector
```

ARC V2 Fixed Encryption Vector

This vector fixes every random or time-dependent value used by ARC V2 so the KDF, stream cipher, HMAC transcript, and final armored file can be tested byte-for-byte.

Inputs:

```
mnemonic: TV1
password: correct horse battery staple
created_at: 2026-06-14T00:00:00.000Z
salt bytes: 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
nonce bytes: 000102030405060708090a0b0c0d0e0f1011121314151617
iterations: 1000000
```

Salt byte length:

```
32
```

Nonce byte length:

```
24
```

Plaintext object before UTF-8 encoding:

```
{"mnemonic":"abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon about","word_count":12,"created_at":"2026-06-14T00:00:00.000Z"}
```

Plaintext JSON hex:

```
7b226d6e656d6f6e6963223a226162616e646f6e206162616e646f6e206162616e
646f6e206162616e646f6e206162616e646f6e206162616e646f6e206162616e64
6f6e206162616e646f6e206162616e646f6e206162616e646f6e206162616e646f
6e2061626f7574222c22776f72645f636f756e74223a31322c2263726561746564
5f6174223a22323032362d30362d31345430303a30303a30302e3030305a227d
```

Plaintext byte length:

```
149
```

PBKDF2 master key:

```
dfffaca375f881b8241d0cbf17b5f395db942d419ae134d9dd7b08fe26e9a246
a79d872f4e4a1caa05ec9fc5f19315e1cc28b32543a3a04da85df5257c8f5333
```

Expanded encryption key:

```
3c8d5f9435d4d935f8dbcb48285ca8d578079eded9ba046a58fb4a6c6b73e6cf
ca43a93488c5bc8757310108f3eb7b05089fd84f5bc393300affeaf79d3e5f8f
```

Expanded MAC key:

```
d5d4d1a6f5f036d040e502ab0c9759ca517463f9dbd971bc0cf3a27b5a728a93
20079532475422c1b118e53f55d33b9858342fefd7f666f570ffe5dba717b9a3
```

ciphertext_b64:

```
  rmQ+bXDPoMHcZXsulTBBKND+Q0IoX0XT3njFsJx2CyFbs70nqU/
B1yC+aUaq21aDwuKnL0OlFbwgq8BrzW7ExUcrdwYoepMcOhw7vroSALS05J/
+8on3euhQsSNa0YppkTAlNpcZz+BnTyFQUEkfZzuBJmejd01lV4n1wS9gi4Mg3Q4Vi1KYcaGbZaCEbMJ61kqZNdtx7YT
OI64N5eiYrDGlni8=
```

Ciphertext decoded byte length:

```
  149
```

mac_b64:

```
  UUZY/YB3NvCD83u9MIEjv5Nxv39pO24PsHESwbTjNWSyt+7xgGsD8nHY4IwAgpT4Cn5f4Amp5r5SUaPHL7Cr3Q==
```

MAC decoded byte length:

```
  64
```

Armored line 1:

```
  ARCULUS-ARC-V2
```

Armored line 2:

```
eyJtYWdpYyI6IkFSQ1VMVVMtQVJDIiwiZm9ybWF0IjoiYXJjdWx1cy1lbmNyeXB0ZWQtc2VlZC12MiIsInZlcnNpb24i
OjIsImNyZWF0ZWRfYXQiOiIyMDI2LTA2LTE0VDAwOjAwOjAwLjAwMFoiLCJrZGYiOnsibmFtZSI6IlBCS0RGMiIsImhh
c2giOiJTSEEtNTEyIiwiaXRlcmF0aW9ucyI6MTAwMDAwMCwic2FsdF9iNjQiOiJBQUVDQXdRRkJnY0lDUW9MREEwT0R4
QVJFaE1VRlJZWEdCa2FHeHdkSGg4PSJ9LCJjaXBoZXIiOnsibmFtZSI6IkhNQUMtU0hBNTEyLUNUUiIsIm5vbmNlX2I2
NCI6IkFBRUNBd1FGQmdjSUNRb0xEQTBPRHhBUkVoTVVGUllYIn0sImNpcGhlcnRleHRfYjY0Ijoicm1RK2JYRFBvTUhj
WlhzdWxUQkJLTkQrUTBJb1gwWFQzbmpGc0p4MkN5RmJzNzBucVUvQjF5QythVWFxMjFhRHd1S25MME9sRmJ3Z3E4QnJ6
VzdFeFVjcmR3WW9lcE1jT2h3N3Zyb1NBTFMwNUovKzhvbjNldWhRc1NOYTBZcHBrVEFsTnBjWnorQm5UeUZRVUVrZlp6
dUJKbWVqZDAxbFY0bjF3UzlnaTRNZzNRNFZpMUtZY2FHYlphQ0ViTUo2MWtxWk5kdHg3WVRPSTY0TjVlaVlyREdsbmk4
PSIsIm1hY19iNjQiOiJVVVpZL1lCM052Q0Q4M3U5TUlFanY1Tnh2MzlwTzI0UHNIRVN3YlRqTldTeXQrN3hnR3NEOG5I
WTRJd0FncFQ0Q241ZjRBbXA1cjVTVWFQSEw3Q3IzUT09In0=
```

Armored file bytes:

```
  ASCII("ARCULUS-ARC-V2") || 0a || ASCII(line2) || 0a
```

ARC V2 importer rejection rules:

```
  line 1 must be exactly ARCULUS-ARC-V2
  version must be exactly integer 2
  kdf.name must be exactly PBKDF2
  kdf.hash must be exactly SHA-512
  kdf.iterations must be >= 600000
  kdf.salt_b64 must decode to exactly 32 bytes
  cipher.name must be exactly HMAC-SHA512-CTR
  cipher.nonce_b64 must decode to exactly 24 bytes
  mac_b64 must decode to exactly 64 bytes
  HMAC must verify before plaintext is decrypted or trusted
```

Negative Test Vectors

Conforming v1.6.0 implementations must reject every input in this section before producing account rows, downloadable bundles, QR payloads, or partial derived output.

Invalid word count:

```
  mnemonic: abandon
  expected result: reject
  reason: 1 word is neither 12 nor 24
```

Invalid word count:

```
  mnemonic: abandon abandon abandon
  expected result: reject
  reason: 3 words is neither 12 nor 24
```

Invalid word count:

```
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon
  expected result: reject
  reason: 11 words is neither 12 nor 24
```

Invalid word count:

```
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon abandon
  expected result: reject
  reason: 13 words is neither 12 nor 24
```

Unknown BIP39 word:

```
  mnemonic: notaword abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon about
  expected result: reject
  reason: token "notaword" is absent from the embedded English BIP39 word list
```

Checksum failure:

```
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon
  expected result: reject
  reason: all words are valid, but checksum bits encoded by the final word do not match
SHA256(entropy)
```

Invalid 24-word checksum:

```
  mnemonic: abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
abandon abandon abandon
  expected result: reject
  reason: 24 zero-index words do not encode the 8 checksum bits required for 256-bit zero
entropy
```

Invalid path component:

```
  path: m/44'/abc'/0'
  expected result: reject
  reason: component "abc" is not a base-10 unsigned integer with optional hardened suffix
```

Invalid path component:

```
  path: m//0'
  expected result: reject
  reason: empty component between slashes
```

Invalid path root:

```
  path: n/44'/0'/0'
  expected result: reject
  reason: path root must be exactly "m"
```

Index overflow:

```
  path: m/2147483648'/0'/0'
  expected result: reject
  reason: component index 2147483648 is outside the accepted pre-hardened range 0..
2147483647
```

Unsupported script:

```
  coin selector: Dogecoin
  script selector: P2TR
  expected result: reject
  reason: Dogecoin Taproot output is not supported by v1.6.0
```

Malformed ARC V2 salt:

```
  salt length: 31 bytes
  expected result: reject
  reason: ARC V2 salt must be exactly 32 bytes
```

Malformed ARC V2 nonce:

```
  nonce length: 23 bytes
  expected result: reject
  reason: ARC V2 nonce must be exactly 24 bytes
```

Malformed ARC V2 MAC:

```
  mac length: 63 bytes
  expected result: reject
  reason: ARC V2 HMAC-SHA512 tag must be exactly 64 bytes
```

Weak ARC V2 iteration count:

```
iterations: 599999
expected result: reject
reason: import floor is 600000 iterations
```

ARC V2 MAC mismatch:

```
mutation: flip bit 0 of decoded mac_b64 byte 0
expected result: reject
reason: constant-time HMAC comparison fails
```

ARC V2 ciphertext mutation:

```
mutation: flip bit 0 of decoded ciphertext_b64 byte 0 without recomputing MAC
expected result: reject
reason: ciphertext bytes are included in the MAC transcript
```

ARC V2 plaintext mismatch:

```
mutation: decrypts to mnemonic with 12 words but word_count field is 24
expected result: reject
reason: decrypted word_count must match normalized mnemonic word count
```

Compliance Checklist

A test harness for v1.6.0 should verify all of the following conditions:

- TV1 mnemonic normalization produces the exact 93 bytes listed above.
- TV1 BIP39 PBKDF2 produces the exact 64-byte seed.
- TV1 secp256k1 BIP32 root IL, IR, xprv, and xpub match exactly.
- Bitcoin Auto m/0' produces P2PKH addresses and the listed key material.
- Bitcoin P2WPKH m/0' changes only address encoding and SLIP132 display keys, not child scalar bytes.
- Bitcoin Cash uses CashAddr output for the same m/0' child key bytes.
- Litecoin Auto at m/84'/2'/0' uses P2WPKH and the ltc Bech32 HRP.
- Dogecoin Auto at m/44'/3'/0' uses P2PKH and Dogecoin WIF versioning.
- Ethereum and ERC-20 rows use the same EIP-55 account address.
- XRP rows use XRPL classic addresses and never derive destination tags.
- Solana and Stellar use SLIP-0010 Ed25519 hardened derivation only.
- Monero emits spend/view key pairs and a standard mainnet address.
- Cardano emits the Shelley base address from payment and staking keys.
- ARC V2 fixed encryption emits the exact ciphertext, MAC, and armor lines.
- Every negative test rejects without returning partial key material.

No address-only comparison is sufficient. Private key bytes, public key bytes, chain codes, paths, script types, and encoded display forms are all normative.

Python CLI v1.6.0 Parity Checks

The Python CLI is a v1.6.0 scripted derivation surface. A Python parity harness must run with APP_VERSION equal to 1.6.0 and must verify at least the first receiving row for every upgraded coin. The canonical mnemonic for the commands below is TV1:

```
abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon
about
```

Version check:

```
python Arculus_Recovery.py --version
```

Expected stdout:

```
Arculus Recovery 1.6.0
```

Bitcoin Cash command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin bitcoincash --output-format json
```

Required JSON fields:

```
coin: bitcoincash
accounts[0].account_script_type_used: p2pkh
accounts[0].receiving[0].address:
  bitcoincash:qppj3tdvu4q89ngxn2l3pruhe7qyyzep9v3l5ke649
```

Solana command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin solana --output-format json
```

Required JSON fields:

```
coin: solana
accounts[0].account_script_type_used: solana
accounts[0].receiving[0].address:
  GjJyeC1r2RgkuoCWMyPYkCWSGSGLcz266EaAkLA27AhL
```

Stellar command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin stellar --output-format json
```

Required JSON fields:

```
coin: stellar
accounts[0].account_script_type_used: stellar
accounts[0].receiving[0].address:
  GB3JDWCQJCWMJ3IILWIGDTQJJC5567PGVEVXSCVPEQOTDN64VJBDQBYX
accounts[0].receiving[0].stellar_secret_seed:
  SBUV3MRWKNS6AYKZ6E6MOUVF2OYMON3MIUASWL3JLY5E3ISDJFELYBRZ
```

Monero command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin monero --output-format json
```

Required JSON fields:

```
coin: monero
accounts[0].account_script_type_used: monero
accounts[0].receiving[0].address:

44jKQv6ZKMd5ecLLmkNJGi7azgSptEq8ki7TFiat1TfLfdDQ1tQ7ZYa3cRh7X2uRwvLDjddWh97ajeyhR2seKSECQeDx
1WR
accounts[0].receiving[0].private_spend_key_hex:
  8f2d521d4334f4d5d174c47aacb346e7633516416fae089d74ecfd0c389a6b08
```

Cardano command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin cardano --output-format json
```

Required JSON fields:

```
coin: cardano
accounts[0].account_script_type_used: cardano
accounts[0].receiving[0].address:

addr1qy8ac7qqy0vtulyl7wntmsxc6wex80gvcyjy33qffrhm7sh927ysx5sftuw0dlft05dz3c7revpf7jx0xnlcjz3
g69mq4afdhv
```

Bitcoin regression command:

```
python Arculus_Recovery.py --mnemonic "<TV1>" --coin bitcoin --derivation "m/0'" --script-
type p2wpkh --count 1 --output-format json
```

Required JSON field:

```
accounts[0].receiving[0].address:
  bc1qgv52mt89gpev6p56huggl970sppqkgftxakv7f
```

The Python harness must parse JSON and compare exact strings. It must not compare terminal formatting, object key order, or address-only output. For secret fields, compare exact lowercase hexadecimal byte encodings.